Cresta Senior ML Engineer

Onsite interview question bank

01. Cresta Senior MLE Interview Loop: Format, Rounds, Culture and the Senior Bar

- Format notes
- Questions
- References

02. Coding: Reported Cresta Problems and Practical ML-Adjacent Python

- Format notes
- A. Sequence decoding (the reported MLE screen problem)
- B. Streaming text and transcripts
- C. Practical systems coding (reported practical prompts and close variants)
- D. Data structures and LeetCode-medium style (reported bar: medium)
- E. Object-oriented and low-level design coding
- F. ML-from-scratch coding in Python/NumPy
- G. Data processing and text normalization
- H. AI-assisted ("vibe") coding round
- I. Concurrency, performance and production hygiene
- References

03. ML/LLM Depth for Contact-Center AI: NLP, Speech, Fine-Tuning, Evaluation, RAG, Agents and Latency

- Format notes for this round
- Decoding and LLM fundamentals
- NLP for conversations
- Speech and ASR
- LLM fine-tuning and adaptation
- Distillation and model compression
- Retrieval-augmented generation
- Agents and tool use
- Evaluation
- Latency optimization and serving
- ML system design prompts (depth-heavy)
- Research, papers and ML foundations
- References

04. ML System Design: Real-Time Agent Assist, AI Agents, Conversation Intelligence, QA Automation, Knowledge Assist and Multi-Tenant LLM Serving

- Format notes for this round
- Questions
- References

05. Project Deep Dive and Hiring Manager Round: Defending the Architecture of ML Systems You Shipped

- Format notes for this round
- A. Opening: draw the system
- B. Data and labeling in your project
- C. Modeling decisions
- D. Evaluation and quality
- E. Serving, latency and cost
- F. Production, monitoring and incidents
- G. Impact and ownership
- H. Transfer to Cresta's systems (whiteboard follow-ups)
- I. Hiring manager: ML ownership and judgment
- J. Questions to ask the hiring manager
- References

07. Cresta Product and Research Context: ML System Design Questions Built on Agent Assist, AI Agent, Conversation Intelligence and Ocean Models

Format notes
Questions
References

08. ML System Design Reference Diagrams: Agent Assist, Voice AI Agent, Agent Evaluation, Model Deployment, Knowledge Assist RAG, Conversation Intelligence and Multi-Tenant LoRA Serving

Format notes for this round
Questions
References

09. Capacity Estimation and Latency Budgets: Back-of-Envelope Numbers for Real-Time Voice, LLM Serving, Knowledge Retrieval and Conversation Storage

Format notes for this round
Questions
References

10. Tagging System and Source Index: What Each Tag Means, Every Reported ML System Design Prompt by Portal, and Where Each Likely Prompt Comes From

Research limits (stated once, applies to the whole bank)
Format notes for this round
Part 1. The tags and their evidence thresholds
Part 2. Every [Reported] prompt, by portal
Part 3. [Likely] design prompts mapped to their sources
Part 4. Questions to confirm the loop with your recruiter
References

01. Cresta Senior MLE Interview Loop: Format, Rounds, Culture and the Senior Bar

> **How to read this chapter.** This chapter puts the most weight on the ML system design round, as requested. It also covers the rounds around it: screens, coding, AI-assisted coding, deep dive and behavioral.
>
> **Tagging system.** Every question carries one of two tags:
>
> - **[Reported] [n]**: a candidate report or question bank attributes this question, or this round type, to Cresta. The number points to the References list. Where the report is for another role (SWE, FDE), the tag says so.
> - **[Likely]**: an inference from Cresta's job postings, products and engineering blog. No candidate has reported it. A citation on a [Likely] item points to the product or posting the question is based on, not to a candidate report.
>
> **Research limits.** None of the cited pages could be opened during research or during this review. The egress proxy refused connections to Glassdoor, Blind, 1point3acres, PracHub, Greenhouse, CoderPad and cresta.com, and the web-search budget for this review run was used up. Every [Reported] item therefore rests on search-result excerpts gathered while this bank was being researched (this chapter and chapters 02-07), not on reading the full pages. One reference that no excerpt supported has been replaced (see the end of the References). Confirm the loop with your recruiter.

Format notes

What candidates and sources report

- **Senior MLE loop (Glassdoor, 2025-26):** the stages were a recruiter call, a LeetCode-medium technical round, a **"vibe coding" round using an LLM tool**, and a **systems design round**. The sample design prompt was "How would you deploy a model in production?" [1]
- **Live coding setting:** reviewers describe live rounds of about an hour in Python on CoderPad over Zoom, with a software engineer, camera on, and conceptual AI questions mixed in [1].
- **Phone-screen pace:** a 2026 Senior SWE report describes three LeetCode-medium questions in 70 minutes. The reviewer felt the interviewer wanted a specific approach, not just any working solution [31].
- **Senior/Staff SWE loop (Blind, 2024):** one hour of coding, one hour of system design and a hiring-manager chat [3].

- **Mid-level SWE loop (Blind):** one technical screen, three coding interviews and three behavioral rounds, with a mix of LeetCode-style and object-oriented questions. Another candidate's recruiter email said the interview would also cover "key concepts and fundamentals around LLMs, GenAI, RAG, and AI Agents" [3][4].
- **MLE phone screen, AI Agent direction (1point3acres, 2025):** greedy and beam-search decoding over a next-token probability dictionary [28][10].
- **Async first step:** Cresta adopted an asynchronous CoderPad technical assessment as the first engineering filter and runs its technical hiring on CoderPad [11]. A Cresta engineering posting describes a self-paced CoderPad assessment after resume review [26]. Blind posters mention an online assessment (OA) before the recruiter screen [5] [6]. One SWE online challenge mixed data structures and algorithms with SQL joins [32].
- **AI-allowed practical round:** a Forward Deployed Engineer (FDE) candidate described a 1.5-hour practical round with AI tools allowed [5]. This matches the "vibe coding" round in the senior MLE report [1].
- **System design with an agentic or eval flavor:** PracHub lists a Cresta ML system design question, "Design an Evaluation Platform for AI Agents Across Prompt and Model Changes" [8][9].
- **Other Cresta engineering loops:** reports include a project presentation round [2], a customer-delivery round plus a leadership round for a Senior FDE [2], an OOP pool/orchestrator coding question with round-robin scheduling [1], and a job dispatcher/executor phone screen (SWE, 2026) [29].
- **Timeline:** about 25 days on average across roles, according to Glassdoor [2].
- **Policy:** Cresta postings say candidates may not record interviews or use AI transcription during them, and that recruiting email comes only from @cresta.ai [12].

Round-by-round summary

Stage	Duration	Interviewer	Status	Senior bar assessed
Online assessment (CoderPad, async)	Timed	Automated	Reported [11][26] [5]	Clean, correct code under time pressure
Recruiter screen	~30 min	Recruiter	Reported [1]	Motivation, team match (Agentic Assist, Agent and System Quality, Voice/ASR, Insights) [12][16]
Technical round (LC-medium, CoderPad + Zoom)	~60-70 min	Engineer	Reported [1][31]	DS&A fluency; for ML, decoding or metric implementations [10][28]
AI-assisted ("vibe coding") practical	60-90 min	Engineer	Reported [1][5]	Directs the LLM tool well, verifies its output, ships working end-to-end code
ML system design	~60 min	Senior or staff MLE	Reported [1][8][9]	End-to-end LLM/ML system: requirements, eval, serving latency, rollout, monitoring
Hiring manager / behavioral	30-60 min	EM	Reported [3]	Ownership, pace, customer orientation, mentoring
Project presentation / deep dive	~60 min	Panel	Reported for SWE [2]; Likely for MLE	Depth on one shipped system; measured impact

What the senior bar looks like (from postings and blog)

- **Strong ML foundations from before LLMs, combined with deep LLM and prompting expertise.** The Agentic Assist track names both as requirements, along with "a proven ability to move research into production-grade systems" [16].
- **Evaluation of non-deterministic systems.** Postings name offline benchmarking, online experimentation and LLM-as-a-judge, plus specific failure modes to diagnose: hallucinations, retrieval errors, tool misuse, context drift, prompt brittleness and multi-step reasoning breakdowns [14][12].
- **Quality metrics.** Postings list accuracy, faithfulness, task completion, latency, cost and robustness [14].
- **Scale and production constraints.** Postings ask candidates to design for scalability, latency, cost efficiency, security and maintainability. The Voice posting covers real-time inference and streaming pipelines [15].
- **Culture.** Third-party pages cite these operating principles: "Mission is the boss", "Do too much", "Have the courage to be disliked" and "patient in strategy, impatient in execution". Reviewers describe a high technical bar, ownership, little politics and a fast pace [25][4].

Questions

A. ML system design: LLM agents and evaluation (highest priority)

1. Design an evaluation platform for AI agents that detects regressions across prompt changes and model changes. Its users are the teams that build agents. [Reported] [8] [9]
    - How do you version prompts, models, tools and test sets together so that a run is reproducible?
    - Which checks are deterministic, and which use an LLM judge? How do you calibrate the judge against human experts?
    - How do you score the agent's reasoning and tool-use path separately from whether the final outcome was correct?
    - How do you handle non-determinism? How many reruns per scenario, and what statistical test decides that something is a regression?
    - How do you gate a release in CI? What is the blocking threshold?
2. How would you deploy a model in production? [Reported] [1]
    - What does your rollout look like: shadow, canary, A/B, then full?
    - What do you monitor in the first 24 hours, and what triggers a rollback?
    - How do you serve it within a real-time latency budget?
    - How do you handle model versioning and per-customer configuration?
3. Design a customer simulator that generates realistic synthetic callers to test an AI agent before launch. [Likely] [17][27]
    - How do you give simulated callers objectives, personalities, account data and deviations from the standard path?
    - How do you mine real conversations so that the test set covers what production actually sees?
    - How do you stop the simulator and the agent from sharing the same blind spots?
4. Design a layered ("Swiss-cheese") reliability strategy for a production AI agent: pre-launch tests, human review and production monitoring. [Likely] [18]

    - Which failure modes does each layer catch, and which does it miss?
    - How do you turn production failures back into regression tests?
5. Design an LLM-as-a-judge system for scoring millions of conversations, with calibration to expert labels. [Likely]
    - How do you measure agreement between the judge and humans, and track it over time?
    - How do you detect judge drift when you upgrade the judge model?
    - What does the judge cost per conversation, and how do you cut it?
6. Design an AI agent that fully handles customer voice or chat for an enterprise, including tool calls (order lookup, refunds) and escalation to a human. [Likely]
    - How do you constrain tool use so the agent cannot make unauthorized commitments?
    - How do you detect and recover from tool misuse or context drift in multi-step flows?
    - How do you design the handoff to a human agent, including the context you pass along?
7. Design a multi-agent orchestration layer, in which a router hands off to specialized sub-agents. [Likely]
    - When does a multi-agent design beat a single agent with many tools?
    - How do you trace and debug a failure that spans several agents?
8. Design a guardrail system against prompt injection and policy violations in a customer-facing agent. [Likely]
    - What does it add to latency, and where in the pipeline does it run?
9. Design an "agent for building agents": a copilot that turns a business owner's natural-language description of a workflow (for example, flight rebooking) into an agent configuration, then tests it. [Likely] [34]
    - How do you validate the generated configuration before it touches real customers?
    - How do you keep a human in the loop without making the copilot pointless?
10. Design a grounding checker that flags agent responses unsupported by the knowledge base or by tool outputs, before or right after they are sent. [Likely] [35]
    - Claim extraction plus entailment, or a single judge call? What does each cost in latency?
    - What do you do when the checker fires in the middle of a live voice turn?

B. ML system design: real-time and voice

11. Design real-time Agent Assist: listen to a live call and surface next-best-action or knowledge suggestions to the human agent. [Likely] [21]
    - What is the end-to-end latency budget from audio to suggestion, and how is it split across ASR, retrieval and LLM?
    - How do you handle partial versus final transcripts? (Cresta pushes both about once per second [21].)
    - How do you decide when to show a suggestion and when to stay silent?
    - How do you measure the effect on agent outcomes, separating it from agent skill?
12. Design a voice AI agent pipeline that responds in under a second: telephony, streaming ASR, LLM and TTS. [Likely] [19]

    - Which per-component metrics do you track (LLM first-token latency, TTS first-byte latency), and how do you measure end-to-end latency?
    - How do you detect end-of-turn and handle barge-in?
    - Which networking choices matter (connection reuse, streaming APIs, keeping DNS off the critical path)?
13. Design an inference graph that runs several models (ASR, redaction, intent, sentiment, summarization) per conversation, with customer-specific policies. [Likely] [20]
    - How do you express dependencies and skip models a customer has not enabled?
    - How do you isolate a slow model so it cannot block real-time outputs?
14. Design an ASR evaluation and quality-monitoring program for contact-center audio across accents, noise and domain vocabulary. [Likely] [13]
    - How does WER differ from downstream task quality, and which do you optimize?
    - How do you get ground-truth transcripts cheaply?
    - How do you detect ASR regressions for a single customer?
15. Design per-customer ASR adaptation for product names, account IDs and jargon (contextual biasing, custom vocabularies or fine-tuning). [Likely] [13]
    - How do you stop biasing toward one customer's terms from raising errors on ordinary speech?
    - How do you roll out an adapted model to one tenant safely?
16. Design a real-time sentiment or escalation-risk detector for live calls. [Likely]
    - How do you combine acoustic and lexical signals?
    - How do you set alert thresholds per customer?

C. ML system design: RAG, knowledge and analytics

17. Design Knowledge Assist: RAG over a customer's knowledge base during live conversations. [Likely] [23][37]
    - How do you reformulate ambiguous conversational queries before retrieval?
    - Would you fine-tune embeddings for the domain, and how would you mine hard negatives?
    - How do you adapt answers by channel (voice, chat, email)?
    - How do you handle stale or contradictory documents?
18. Design topic discovery over millions of unlabeled conversations: an LLM-built taxonomy with up to three levels, which admins can merge, split and rename. [Likely] [24]
    - How do you keep cluster assignments stable when the taxonomy is edited?
    - How do you overlay outcomes (resolution, sales) to rank problem areas?
19. Design a system that infers conversation outcomes (resolved or not, customer experience) without surveys or disposition codes. [Likely] [24]
    - How do you validate inferred outcomes without ground truth?
20. Design a research agent ("AI Analyst") that answers open-ended business questions over a customer's conversation corpus. [Likely] [24]
    - How do you combine per-conversation LLM extraction with aggregate statistics?
    - How do you stop the agent from presenting a weak correlation as a finding?
21. Design automated QA scoring for 100% of calls against a customer-defined rubric. [Likely]
    - How do you turn a free-text rubric item into a reliable judgment?

    - How do you stop agents from gaming the scorer?
22. Design a real-time trend detector that flags emerging issues before dashboards would show them. [Likely]
    - How do you separate a true spike from seasonality or noise?
23. Design post-call summarization that writes into each customer's CRM format, using fine-tuned models rather than a frontier API. [Likely] [36]
    - How do you evaluate summaries when every customer wants different fields?
    - When does fine-tuning beat prompting a larger model for this task?

D. ML system design: models, serving, experiments and cost

24. Design the domain adaptation of a base LLM for contact centers, in the spirit of Cresta's Ocean-1: data, SFT, preference tuning and serving. [Likely] [22]
    - Self-instruct and RLHF/DPO: what data do you need, and how do you avoid overfitting to one customer?
    - How do you prove it beats a frontier API on quality, latency and cost?
25. Design a migration from a frontier LLM API to a smaller self-hosted model for one high-volume feature. [Likely]
    - Which quality bar must be met, and how is it measured?
    - Distillation or fine-tuning: which, and on what data?
26. Design multi-tenant serving: one base model with per-customer adapters or prompts. [Likely] [38]
    - How do you serve hundreds of LoRA adapters efficiently?
    - How do you prevent data leakage across tenants?
27. Design an LLM serving stack for real-time features with p99 latency targets. [Likely]
    - What do batching, KV cache, quantization and speculative decoding each trade off?
    - How do you autoscale when CPU-based autoscaling (HPA) is meaningless for GPUs?
28. Design monitoring for production LLM features across hundreds of enterprise customers. [Likely]
    - Which label-free signals catch quality regressions?
    - How do you catch a regression that affects only one customer?
29. Design the feedback flywheel: agent accept, edit and ignore signals on suggestions feed model improvement. [Likely]
    - What biases does this implicit feedback introduce, and how do you correct them?
30. Design a PII redaction pipeline for audio and transcripts that runs before storage and training. [Likely]
    - How do you trade recall against precision, and what does a miss cost?
31. Design an online experiment for a new Agent Assist model when a handful of large customers dominate traffic and agents work in teams. [Likely]
    - Randomize by conversation, by agent or by team? What interference do you expect?
    - Which outcome metric moves fast enough to decide within two weeks?
32. Design the training-data pipeline for customer-specific models under contractual data limits: opt-outs, retention windows and deletion requests. [Likely]

    - How do you prove a deleted customer's data no longer influences a deployed model?

E. Coding rounds (online assessment, technical screen and onsite)

33. Implement greedy decoding given a Python dictionary that maps each token to its candidate next tokens and their probabilities, starting from a start token and stopping at an end token or a maximum length. Then extend it to beam search with beam width k. [Reported] [10][28]
    - How do you add length normalization?
    - How would you add top-k / top-p sampling and temperature?
    - What is the time and space complexity in beam width and vocabulary size?
34. Design the decoder as a reusable component: what interface lets greedy and beam search share code? [Reported] [30]
35. Walk through how an AI agent generates a sequence step by step. Where can the decoding strategy change the agent's behavior? [Reported] [28]
36. Solve LeetCode-medium problems in the technical round, possibly three in 70 minutes. [Reported] [1][31]
    - Can you state the complexity and test edge cases before running the code?
    - If the interviewer is steering toward a specific approach, how do you find out early?
37. Implement a worker pool and an orchestrator that assigns tasks round-robin (reported for the FDE AI Agent role); a SWE variant asks for a job dispatcher that hands jobs to a pool of executors. [Reported] [1][29]
    - How do you handle worker failures and retries?
    - How do you make it thread-safe?
38. Implement a rate limiter in your language of choice. [Reported] [1]
    - Extend it to per-tenant limits for shared LLM API calls. Token bucket or sliding window, and why?
39. Call a public API (Wikipedia was used in a reported SWE phone screen), then process and return the results. [Reported] [7]
    - How do you handle pagination, rate limits and errors?
40. Implement autocomplete: given a list of words and a prefix, return the matching suggestions (reported for SWE). [Reported] [32]
    - How do you rank suggestions by frequency, and what changes at a million words?
41. Convert a flat list of records with parent references into a tree (reported for SWE). [Reported] [32]
42. Solve a grid or matrix problem framed as a puzzle game in a timed online challenge (reported for SWE). [Reported] [32]
43. Write SQL with joins in the online challenge, for example: from `conversations` and `annotations` tables, find agents whose suggestion-acceptance rate dropped week over week. [Reported for SQL joins; the example prompt is Likely] [32]
44. Solve object-oriented design problems mixed with LeetCode-style problems (mid-level SWE loop). [Reported] [3]
45. Implement word error rate (edit distance) between a reference transcript and a hypothesis. [Likely]

46. Given a stream of utterances with speaker labels and timestamps, merge them into turns and emit partial results in real time. [Likely]
47. Implement precision, recall and F1 from scratch for multi-label intent classification, plus micro and macro averages. [Likely]
48. Implement cosine-similarity top-k retrieval over an embedding matrix in NumPy, without loops. [Likely]
    - Which ANN index would you choose at 100M vectors?
49. Implement a dynamic batching queue that maximizes GPU throughput under a max-latency constraint. [Likely]
50. Implement a streaming chunker that splits a live transcript into token-budgeted segments without cutting sentences. [Likely]
51. Count the top-k most frequent intents over a sliding time window on an unbounded stream, with bounded memory. [Likely]

F. AI-assisted ("vibe coding") practical round

52. Build a small working feature end to end with an LLM coding tool, within the time limit. [Reported] [1][5]
    - How do you break the task down before prompting the tool?
    - How do you verify generated code? Which tests do you write yourself?
    - When do you stop prompting and write the code by hand?
53. Build a minimal RAG or agent prototype (for example, an agent that answers from a document set or calls a mock API). [Likely]
    - How do you evaluate it in the time available?
54. Build a tool-calling loop for a flight-rebooking agent against mock flight and booking APIs, including a confirmation step before any change. [Likely] [34]
55. Build a small evaluation harness: run an LLM over a labeled set of transcripts, enforce a JSON output schema, and report accuracy with confidence intervals. [Likely]
56. Debug an LLM-generated pipeline that passes its tests but gives wrong results. [Likely]

G. LLM / GenAI fundamentals (flagged by recruiters)

57. Explain the key concepts and fundamentals of LLMs, GenAI, RAG and AI agents. [Reported] [3][4]
58. What is RAG, and how do you prevent hallucination? [Reported for an FDE technical screen] [33]
    - In what ways does RAG still fail?
    - How do you measure faithfulness or groundedness automatically?
59. What is an AI agent, as opposed to a chain or a workflow? When would you not use an agent? [Likely]
60. How does function calling / tool use work, and how do you test multi-turn agent trajectories? [Likely]
61. What drives LLM inference latency and cost, and which levers reduce them? [Likely]
62. How do you handle conversations longer than the context window? [Likely]
63. Compare SFT, DPO and RLHF. When is each worth its cost? [Likely]

64. How do you make prompts robust across model versions, and how do you regression-test them? [Likely]
65. Which ML fundamentals from before LLMs still matter here (calibration, class imbalance, active learning, experiment design)? [Likely] [16]
    - Why does calibration matter for a confidence-gated suggestion?
66. How do ASR errors propagate into downstream NLP, and how do you make classifiers robust to them? [Likely]
67. How do you evaluate a generative model when there is no single correct answer? [Likely]
    - How do you design a rubric for human raters and measure agreement between them?
68. QA labels from different customers' reviewers disagree systematically. How do you detect and model annotator bias before training on the labels? [Likely]

H. Project deep dive / presentation

69. Present a significant project you built end to end. [Reported for SWE] [2]
    - What was the baseline, and how did you prove the improvement?
    - What did you personally own?
70. Walk through an LLM or ML system you shipped to production. What broke first? [Likely]
71. Tell me about a model that worked offline but failed online. How did you find the gap? [Likely]
72. Describe the evaluation you built for your project. What did your metrics miss? [Likely]
73. How did you handle latency, cost or scaling constraints? What did a request cost, and how did you bring it down? [Likely]
74. Describe a decision where you chose a simpler model over a more sophisticated one. [Likely]
75. Tell me about a time you took a research idea or paper into a production system. What had to change on the way? [Likely] [16]
76. If we gave you this problem in the contact-center domain, what would change? [Likely]

I. Hiring manager, behavioral and culture

77. Walk through your background, focusing on ML you shipped. Why Cresta and contact-center AI? [Likely]
78. Which track appeals to you: Agentic Assist, Agent and System Quality, Voice/ASR, or Insights? [Likely] [12]
    - Placement depends on your strengths and on business needs. Where would you have the most impact?
79. Tell me about a time you "did too much": went well beyond scope to get something shipped. [Likely] [25]
80. Tell me about a time you had the courage to be disliked: pushing back on a PM, a customer or a leader with data. [Likely] [25]
81. Tell me about a time you were patient on strategy but impatient on execution. [Likely] [25]

82. Describe working directly with customers or forward-deployed teams to define what an ML feature should do. [Likely]
    - How did you turn a vague ask into measurable requirements?
83. Describe a disagreement over quality versus shipping speed. How was it resolved? [Likely]
84. Tell me about mentoring engineers or raising your team's technical bar. [Likely]
85. Tell me about changing another team's roadmap or technical direction without having authority over them. [Likely]
86. Tell me about the hardest production incident you were involved in, and what the postmortem changed. [Likely]
87. How do you decide which new LLM research is worth adopting? [Likely]
88. If evaluation for a key model turned out to be weak, what would you do in your first 90 days? [Likely]
89. When would you build a capability in-house rather than use a third-party model provider? [Likely]
90. How would you explain a model regression to an enterprise customer? [Likely]

J. Questions to ask your interviewers

91. Which team would I join, and what are its goals for the next two quarters? [Likely]
92. How does the team gate LLM changes before they reach customers? [Likely]
93. What is the split between self-hosted models and third-party APIs, and where is it heading? [Likely]
94. How much direct customer contact do MLEs have? [Likely]
95. How is the AI-assisted coding round scored: on the final product, or on how I direct and check the tool? [Likely]
96. What separates Senior from Staff MLE here? [Likely]

References

1. Glassdoor, "Cresta (CA) Interview Questions" (Senior Machine Learning Engineer loop with vibe-coding and system design rounds; FDE reviews; CoderPad and Zoom Python round; rate limiter), 2024-2026. https://www.glassdoor.co.in/Interview/Cresta-CA-Interview-Questions-E3180909.htm
2. Glassdoor, "Cresta (CA) Interview Questions" (25-day average; SWE project presentation; Senior FDE loop), 2025-2026. https://www.glassdoor.ca/Interview/Cresta-CA-Interview-Questions-E3180909.htm
3. Blind, "Cresta interview process", 2024-2025. https://www.teamblind.com/post/cresta-interview-process-jmfnzjoa
4. Blind, "Anyone interview with Cresta lately?", 2025. https://www.teamblind.com/post/anyone-interview-with-cresta-lately-zvj0uhq2
5. Blind, "Interview for forward deployment engineer at Cresta", 2025. https://www.teamblind.com/post/interview-for-forward-deployment-engineer-at-cresta-ngswbj44
6. Blind, "Cresta senior backend SWE interview", 2025. https://www.teamblind.com/post/cresta-senior-backend-swe-interview-6v1tmjld
7. 1point3acres, "cresta interview questions" company page (SWE/MLE phone screen and online test reports, including the API phone screen), 2024-2026. https://www.1poi

nt3acres.com/interview/company/cresta
8. PracHub, "Cresta ML System Design Interview Questions" ("Design an Evaluation Platform for AI Agents Across Prompt and Model Changes"), 2026. https://prachub.com/companies/cresta/categories/ml-system-design
9. PracHub, "Cresta Software Engineer Interview Experience: System Design Round: Build an Eval Platform for AI Agents". https://prachub.com/interview-experiences/cresta-software-engineer-interview-experience-system-design-round-build-an-eval-platform-for-ai-agents
10. PracHub, "Cresta Machine Learning Interview Questions" ("Implement greedy and beam decoding"). https://prachub.com/companies/cresta/categories/machine-learning
11. CoderPad, "How Cresta consolidated all technical hiring on CoderPad to scale eng hiring" (case study). https://coderpad.io/case-study/how-cresta-consolidated-all-technical-hiring-on-coderpad-to-scale-eng-hiring/
12. Cresta on Greenhouse, "Senior Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4249943008
13. Cresta on Greenhouse, "Senior Machine Learning Engineer (ASR)". https://job-boards.greenhouse.io/cresta/jobs/5101011008
14. Cresta on Greenhouse, "Staff Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4741283008
15. Cresta on Greenhouse, "Senior Machine Learning Engineer, Voice Experience". https://job-boards.greenhouse.io/cresta/jobs/5155675008
16. Built In Vancouver, "Senior Machine Learning Engineer, Cresta" (Agentic Assist track). https://builtinvancouver.org/job/senior-machine-learning-engineer/8619914
17. Cresta Blog, "How Cresta Grades Its Agents: Evaluators in an LLM World". https://cresta.com/blog/how-cresta-grades-its-agents-evaluators-in-an-llm-world
18. Cresta Blog, "Why AI Agent Evaluations Fail, and How the Swiss-Cheese Model Prevails". https://cresta.com/blog/why-ai-agent-evaluations-fail----and-how-the-swiss-cheese-model-prevails
19. Cresta Blog, "Engineering for Real-Time Voice Agent Latency". https://cresta.com/blog/engineering-for-real-time-voice-agent-latency
20. Cresta Blog, "Understanding Cresta's Voice Platform: ML Services, Inference Graphs and Real-Time Intelligence", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-ml-services-inference-graphs-and-real-time-intelligence
21. Cresta Blog, "Understanding Cresta's Voice Platform: The Voice Stack", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-the-voice-stack
22. Cresta Blog, "Introducing Ocean-1: First Contact Center Foundation Model", 2023. https://cresta.com/blog/introducing-ocean-1-worlds-first-contact-center-foundation-model
23. Cresta Blog, "AI-Ready Knowledge for Contact Centers: Closing the Gap Between KBs and AI", 2025. https://cresta.com/blog/ai-ready-knowledge-for-contact-centers-closing-the-gap-between-kbs-and-ai
24. Cresta Blog, "Introducing the Next Generation of Cresta Insights", 2026. https://cresta.com/blog/introducing-the-next-generation-of-cresta-insights-a-complete-system-for-enterprises-to-understand-their-cx
25. JobsByCulture, "Cresta: Frontend Engineer" (culture and operating-principles summary; third-party). https://jobsbyculture.com/jobs-at/cresta/frontend-engineer
26. a16z Jobs, "Cresta: Senior Product Engineer" (posting describing the self-paced CoderPad assessment). https://jobs.a16z.com/jobs/cresta/5288477008--senior-product-engineer

27. Cresta Blog, "Cresta Launches Automated AI Agent Testing: Confidence in Every Conversation". https://cresta.com/blog/cresta-launches-automated-ai-agent-testing-confidence-in-every-conversation
28. 1point3acres, "Cresta Machine Learning Engineer Tech Phone Interview Experience" (MLE, AI Agent direction; greedy and beam-search decoding), 2025. https://www.1point3acres.com/interview/thread/1140481
29. 1point3acres, "Cresta Fulltime Software Engineer Tech Phone Screen: Job Dispatcher-Executor System Implementation" (title seen in a listing; full text not read), 2026. https://www.1point3acres.com/interview/thread/1172433
30. PracHub, "Cresta Interview Questions" (company page; "Design sequence decoding with greedy and beam search"), 2025. https://prachub.com/companies/cresta
31. Glassdoor, "Cresta (CA) Senior Software Engineer Interview Questions" (three LeetCode-medium questions in 70 minutes), 2026. https://www.glassdoor.ca/Interview/Cresta-CA-Senior-Software-Engineer-Interview-Questions-EI_IE3180909.0,9_KO10,34.htm
32. Glassdoor, "Cresta (CA) Software Engineer Interview Questions" (autocomplete, list-to-tree, grid puzzle and SQL-joins online challenge), 2024-2026. https://www.glassdoor.sg/Interview/Cresta-CA-Software-Engineer-Interview-Questions-EI_IE3180909.0,9_KO10,27.htm
33. Glassdoor, "Cresta (CA) Interview Experience and Questions" (FDE technical screen: RAG and hallucination), 2026. https://www.glassdoor.com/Interview/Cresta-CA-Interview-Questions-E3180909.htm
34. Cresta Blog, "Cresta Conductor: The Agent for AI Agent Development". https://cresta.com/blog/cresta-conductor-the-agent-for-ai-agent-development
35. Cresta Blog, "Grounding Reality: How Cresta Tackles LLM Hallucinations in Enterprise AI". https://cresta.com/blog/grounding-reality---how-cresta-tackles-llm-hallucinations-in-enterprise-ai
36. Cresta Blog, "How Fine-Tuned LLMs Power Knowledge Assist, Summarization and Chat Suggestions", 2024. https://cresta.com/blog/how-fine-tuned-llms-power-knowledge-assist-summarization-and-chat-suggestions
37. Fireworks AI Blog, "Story: Cresta Knowledge Assist", 2024. https://www.fireworks.ai/blog/story-cresta-knowledge-assist
38. Fireworks AI Blog, "Multi-LoRA", 2024. https://fireworks.ai/blog/multi-lora

Removed in review: the 1point3acres thread 1173094 (no search excerpt in this bank supported it; replaced by the company page, ref 7); the Blind company-posts page and the Scoutify guide (neither was cited, and Scoutify is a generic template).

02. Coding: Reported Cresta Problems and Practical ML-Adjacent Python

Format notes

- **Platform and setting.** Cresta runs its technical hiring on CoderPad, including an asynchronous CoderPad Screen step for engineering roles [11]. Candidates describe live rounds as about an hour on CoderPad and Zoom with a software engineer, with the camera on. The coding is live Python, and conceptual AI questions are mixed in [6].
- **MLE phone screen.** An August 2025 1point3acres report on the MLE (AI Agent) phone screen describes one problem in two parts: sequence decoding over a dictionary of next-token probabilities, first with greedy search and then with beam search, written in Python [1][2]. PracHub lists the same problem under two Cresta-tagged titles [4].
- **Loop shape.** A Glassdoor report for a Senior ML Engineer lists a recruiter call, a LeetCode-medium technical interview, a "vibe coding" interview (building something with an LLM coding tool) and a system design round [6]. SWE loops are described as one technical screen, three coding rounds and three behavioral rounds, mixing LeetCode-style and object-oriented problems [8]. A Senior/Staff loop is described as one hour of coding, one hour of system design and a hiring-manager chat [9].
- **Difficulty and bar.** A 2026 Toronto Senior SWE report describes a phone screen with three LeetCode-medium-level questions in 70 minutes [5]. One reviewer felt the interviewer wanted a specific approach, not just any working solution [5]. Other reported practical prompts include a rate limiter [6], a job dispatcher/executor system (SWE phone screen, 2026) [3], autocomplete over a word list, and turning a flat list into a tree [7]. FDE candidates report a 1.5-hour practical round where AI tools are allowed [10].
- **Senior-level emphasis (inferred).** Expect clean, testable Python, explicit complexity trade-offs, and extensions toward streaming and production concerns such as partial transcripts, latency and concurrency. These map to Cresta's real-time voice and agent stack [13][14][15] and its Senior MLE postings (Python/Go, PyTorch, Transformers, LLMs, model optimization) [16][17].

Tag legend: [Reported] means a source reports the question or a very close variant for Cresta (cited). [Likely] means it is inferred from the role, the team and Cresta's products; it is not reported.

A. Sequence decoding (the reported MLE screen problem)

A1. Core problem

1. [Reported] You are given a Python dict that maps each token to a dict of candidate next tokens and their probabilities. Starting from a start token, implement greedy decoding that produces the most likely sequence until an end token or a maximum length [1][4].
    - What do you do when a token has no outgoing edges, or when the graph has a cycle?
    - How do you break ties between equal probabilities so the output is deterministic?
    - What is the time complexity per step, and overall?
2. [Reported] Extend your solution to beam search with beam width k over the same next-token dictionary, and return the top sequence or the top-k sequences with their scores [1][4].
    - Why do you accumulate log-probabilities instead of multiplying probabilities?
    - How do you handle beams that reach the end token before the others do?
    - What is the complexity in terms of k, the vocabulary branching factor and the maximum length?
3. [Reported] Design the decoder as a reusable component (PracHub title: "Design sequence decoding with greedy and beam search"). What interface would you expose so that greedy and beam share code [4]?
    - How would you plug in a real model's next-token function instead of a static dict?
    - How would you unit-test the decoder?

A2. Likely extensions of the decoding problem

4. [Likely] Add length normalization to beam search (for example, dividing by length^alpha). Show a case where the unnormalized beam prefers a short sequence.
    - Does normalization change which hypotheses get pruned mid-search, or only the final ranking?
5. [Likely] Implement top-k sampling and nucleus (top-p) sampling with temperature over the same structure.
    - How do you make the sampler reproducible with seeds in tests?
    - What happens when p is so small that only one token survives?
6. [Likely] Add a constraint so that a banned set of tokens (for example, PII placeholders or forbidden phrases) never appears in decoded output.
    - How would you enforce "no repeated n-gram of size n"?
7. [Likely] Return the k best complete sequences without duplicates when different paths produce the same surface string.
8. [Likely] Implement early stopping: terminate beam search once no live beam can beat the best finished hypothesis.
    - Under what scoring assumption is this stopping rule correct?
9. [Likely] Batch the decoder so it decodes many prompts in parallel with a vectorized next-token function, using NumPy.
    - How do you handle sequences in the batch that finish at different steps (masking or padding)?
10. [Likely] Implement constrained decoding to one of a fixed set of allowed outputs (for example, intent labels) using a trie of allowed token sequences.

11. [Likely] Make the decoder stream tokens to a caller as a generator while beam search is still running.
    - Can you safely emit tokens before the beam has converged? Which prefix is stable?

B. Streaming text and transcripts

12. [Likely] You receive ASR results as a stream of events `(utterance_id, is_final, text, start_ms, end_ms)`. Partial transcripts can be revised until a final arrives. Maintain the current best transcript of the conversation. (Grounded in Cresta's partial-vs-final transcript pipeline [13][14].)
    - How do you handle a final that arrives after a newer partial for the same utterance?
    - What is your memory footprint for a 2-hour call?
13. [Likely] Given the transcript stream, fire a callback only when a partial transcript is "stable", for example when a prefix has not changed across N consecutive partials.
    - How do you avoid re-firing for the same stable prefix?
14. [Likely] Merge two ordered streams of utterances (agent channel and customer channel) into a single time-ordered conversation, interleaving by start time.
    - How would you generalize this to k speakers? What is the complexity with a heap?
    - How do you handle overlapping speech?
15. [Likely] Implement a sliding-window keyword detector over a stream of tokens: emit an event when any of M keyword phrases appears. (Cresta runs cheap keyword detection on partials [14].)
    - How does Aho-Corasick compare to naive matching as M grows?
    - What if keywords can span two transcript chunks?
16. [Likely] Implement a rolling context window that keeps the last N tokens (or last T seconds) of a conversation to feed an LLM prompt.
    - How do you always keep the system prompt and the first customer statement?
17. [Likely] Write a function that deduplicates overlapping chunks of transcript text from a streaming source (chunk i+1 may repeat the tail of chunk i).
    - What is the complexity of your overlap detection? Can you use KMP or rolling hashes?
18. [Likely] Compute talk-time, silence and interruption statistics per speaker from a list of timestamped segments.
    - How do you define and count an interruption?
    - How do you handle segments with missing end times?
19. [Likely] Implement real-time redaction of credit-card numbers and SSNs in a token stream where a number can be split across chunks. (Cresta's voice stack includes audio and transcript redaction [13].)
    - How much lookahead buffer do you need, and what does it cost in latency?
    - How would you validate card numbers (Luhn) without false positives on phone numbers?
20. [Likely] Given word-level ASR timestamps and a list of detected "moments" (character spans), map each moment back to audio start and end times.

21. [Likely] Implement a debouncer/throttler so that a downstream model is called at most once every X ms per conversation while still processing the latest partial.
22. [Likely] Implement an online running mean and variance (Welford) of per-turn sentiment scores per conversation, and flag turns that deviate by more than k standard deviations.

C. Practical systems coding (reported practical prompts and close variants)

23. [Reported] Implement a rate limiter in your language of choice [6].
    - Token bucket versus sliding-window log versus fixed window: which did you pick and why?
    - How would you make it thread-safe?
    - How do you make it per-customer (tenant) and per-endpoint?
24. [Reported] Implement a job dispatcher/executor system: a dispatcher accepts jobs and hands them to a pool of executors (reported for a 2026 SWE phone screen) [3].
    - How do you handle job failures and retries with backoff?
    - How do you support priorities, or cancellation of a queued job?
    - How do you guarantee at-least-once versus exactly-once execution?
25. [Reported] Implement an API client exercise: call a public API (reported as a Wikipedia API task in an SWE phone screen) and process the results [2].
    - How do you handle pagination, rate limits and transient errors?
    - How would you parallelize requests and cap concurrency?
26. [Likely] Implement an inference request batcher (dynamic batching): collect requests until either batch size B or wait time T, then call the model once and route results back to callers.
    - How do you choose B and T against a latency SLO? (Cresta's voice-agent target is about 1.5 s end to end [15].)
    - What happens when the model call raises for one item in the batch?
27. [Likely] Implement an LRU cache with TTL for embedding or LLM responses keyed by normalized prompt.
    - How do you make get and put O(1)?
    - How would you add a maximum size in bytes instead of in entries?
28. [Likely] Write a retry decorator with exponential backoff, jitter and a timeout, usable on both sync and async functions.
29. [Likely] Implement a simple inference-graph executor: given a DAG of model "nodes" with dependencies, run them in topological order and run independent nodes concurrently. (Cresta describes inference graphs that orchestrate ML services [14].)
    - How do you detect cycles?
    - How do you skip nodes whose conditions are not met, for example models that run only on final transcripts?
30. [Likely] Implement a circuit breaker around an external LLM or ASR provider call.
    - What state transitions exist, and how do you test them deterministically?
31. [Likely] Write an asyncio producer/consumer pipeline: ingest transcript events, run a classifier and write annotations, with bounded queues for backpressure.
    - What happens when the consumer is slower than the producer?

32. [Likely] Implement a top-K heavy-hitters tracker (for example, most frequent customer intents in the last hour) over a stream.
    - Exact (heap plus counter with time buckets) versus approximate (Count-Min Sketch): when would you choose each?
33. [Likely] Implement consistent hashing to route conversations to workers so that a conversation's events always land on the same worker.
    - What happens when a worker is added or removed?

D. Data structures and LeetCode-medium style (reported bar: medium)

34. [Reported] Expect LeetCode-medium algorithm problems in the technical/phone screen, sometimes three in 70 minutes [5][6].
    - Can you state complexity before coding and test edge cases out loud?
35. [Reported] Implement autocomplete: given a list of words and a prefix, return matching suggestions (reported for a SWE role) [7].
    - How do you return the top-k by frequency instead of alphabetically?
    - How do you support updates to frequencies as users type?
    - Implement the trie yourself with insert, prefix search and delete; how would you extend it to wildcard queries?
36. [Reported] Convert a flat list of records with parent references into a tree (reported for a SWE role) [7].
    - How do you detect orphans and cycles?
    - Can you do it in a single pass?
37. [Reported] A matrix or grid problem disguised as a puzzle game appeared in an online coding challenge (SWE) [7].
    - BFS or DFS: which and why? What is the memory cost?
38. [Likely] Given a conversation as a list of turns, find the longest contiguous segment where the customer's sentiment never drops below a threshold.
39. [Likely] Merge overlapping time intervals (speaker segments), then compute the total non-overlapping speech duration.
40. [Likely] Find all pairs of conversations whose embedding cosine similarity exceeds a threshold, efficiently for N = 100k.
    - What changes if you use approximate nearest-neighbor search?
41. [Likely] Implement word-level edit distance and use it to compute word error rate (WER) between a reference and an ASR hypothesis.
    - How would you return the alignment (substitutions, insertions, deletions)?
    - How do you normalize text (casing, punctuation, numbers) before scoring?
42. [Likely] Longest common subsequence between two transcripts of the same call from two different ASR vendors.
43. [Likely] Group near-duplicate customer utterances using MinHash/LSH or normalized string keys.
44. [Likely] Given a log of `(conversation_id, timestamp, event)` events, find conversations where event B happened within T seconds after event A.
45. [Likely] Design a data structure supporting `add(word)` and `most_frequent_in_last(seconds)` over a stream.

46. [Likely] Validate balanced spans: given nested annotation spans over text (entities inside sentences), check that they are properly nested.
47. [Likely] Tokenize text with a given byte-pair-encoding (BPE) merge table: apply the merges in order to a word.
    - What is the complexity of your naive merge loop, and how could you speed it up?

E. Object-oriented and low-level design coding

48. [Reported] Expect at least one object-oriented coding problem alongside LeetCode-style rounds (reported for SWE loops) [8].
    - How do you decide class boundaries, and what would you make an interface or abstract base class?
49. [Likely] Design the classes for a conversation-annotation system: Conversation, Turn, Annotation (moment versus action), and a registry of detectors that run on new turns. (Moment/action annotations come from [14].)
    - How do you add a new detector without modifying existing code?
50. [Likely] Implement a minimal tool-calling agent loop: given an LLM function that returns either text or a tool call, execute tools from a registry and feed results back until a final answer or a step limit.
    - How do you validate tool arguments against a schema?
    - How do you prevent infinite loops or repeated identical tool calls?
51. [Likely] Implement a conversation state machine for a booking or rebooking flow (states, transitions, guards) that an AI agent follows. (Cresta's agent material uses flight rebooking as an example scenario [12].)
    - How do you persist and resume state mid-conversation?
52. [Likely] Implement a plugin-style policy engine: rules such as "if intent X is detected and the agent has not said Y within 60 s, emit hint Z".
    - How do you represent time-based conditions efficiently across thousands of live calls?
53. [Likely] Implement a prompt template class with variable validation, truncation to a token budget and escaping of user content.
    - How do you guard against prompt injection from transcript text?

F. ML-from-scratch coding in Python/NumPy

54. [Likely] Implement logistic regression training with batch gradient descent in NumPy, then predict probabilities. (A third-party guide lists from-scratch ML algorithms as screen material for Cresta ML roles; it is not verified [12].)
    - How do you add L2 regularization and verify the gradient numerically?
    - How do you make it numerically stable for large logits?
55. [Likely] Implement numerically stable softmax and log-softmax, and cross-entropy loss.
56. [Likely] Implement k-means with k-means++ initialization [12].
    - How do you detect convergence, and what do you do about empty clusters?
57. [Likely] Implement precision, recall, F1 and a confusion matrix for a multi-class intent classifier from lists of labels.

    - Macro versus micro versus weighted averaging: when does each mislead?
58. [Likely] Compute ROC-AUC from scores and labels without sklearn.
    - How do you handle tied scores?
59. [Likely] Choose a decision threshold for a "send hint" classifier that maximizes recall subject to precision >= P, given scores and labels.
60. [Likely] Implement scaled dot-product attention (with a causal mask) in NumPy or PyTorch.
    - What shapes do Q, K and V have with batching and multiple heads?
    - Why scale by sqrt(d_k)?
61. [Likely] Implement a key-value cache for autoregressive decoding and show how it changes the per-step cost.
62. [Likely] Implement cosine-similarity top-k retrieval over a matrix of embeddings for a RAG knowledge-assist feature, in vectorized NumPy.
    - How do you handle a corpus that does not fit in memory?
63. [Likely] Implement a text chunker for RAG that splits documents into overlapping chunks on sentence boundaries under a token limit.
64. [Likely] Implement BM25 scoring over a small corpus and combine it with embedding scores (hybrid retrieval).
    - How do you normalize the two score distributions before combining?
65. [Likely] Implement stratified train/validation/test splitting by conversation, so that turns from one conversation never leak across splits.
66. [Likely] Implement a simple bootstrap confidence interval for the difference in an evaluation metric between two models.
67. [Likely] Write the training loop for a PyTorch text classifier: data loader, padding/collation, forward, loss, backward, gradient clipping and evaluation.
    - Where do you put `model.eval()` and `torch.no_grad()`, and why?
    - How would you add mixed precision?
68. [Likely] Implement Expected Calibration Error (ECE) for classifier probabilities.
69. [Likely] Implement an LLM-as-judge evaluation harness: run prompts against N test cases, parse structured verdicts, retry on parse failure and aggregate scores with per-category breakdowns. (Cresta's Senior MLE quality track focuses on evaluation frameworks and failure modes such as hallucination and tool misuse [17].)
    - How do you detect and handle judge position bias?

G. Data processing and text normalization

70. [Likely] Parse a JSONL file of conversations (possibly malformed lines) into turn-level training examples with labels, and report the counts of bad rows.
    - How do you process a 50 GB file without loading it into memory?
71. [Likely] Normalize ASR text for downstream NLP: number words to digits, collapse fillers ("um", "uh"), fix casing.
    - Which normalization steps are reversible, and why might you need reversibility?
72. [Likely] Given turn-level labels from multiple annotators, compute majority labels and Cohen's/Fleiss' kappa.
73. [Likely] Implement a group-by aggregation without pandas: average handle time per agent per day from a list of call records.
    - Then do it in pandas. Which operations dominate cost at scale?

74. [Likely] SQL: given `conversations` and `annotations` tables, find agents whose hint-acceptance rate dropped week-over-week. (SQL joins appeared in a Cresta online challenge [7].)
75. [Likely] Write a function to detect language-switching turns in a bilingual conversation using per-token language-ID scores and a smoothing window.
76. [Likely] Implement reservoir sampling to pick a uniform sample of k conversations from a stream of unknown length.
    - How do you make it weighted?
77. [Likely] Implement a sessionizer: from raw event logs with gaps, split events into sessions where a gap of more than G minutes starts a new session.

H. AI-assisted ("vibe") coding round

78. [Reported] Use an LLM coding tool to build a small working application within the interview time (reported for a Senior ML Engineer loop) [6]. FDE candidates report a similar 1.5-hour practical where AI is allowed [10].
    - How do you verify that LLM-generated code is correct? What tests do you write first?
    - When do you stop prompting and write the code yourself?
    - How do you explain the design choices the tool made?
79. [Likely] Build a small conversation summarizer service end to end with an LLM API: input a transcript, output a structured summary (reason for call, resolution, follow-ups) validated against a JSON schema.
    - How do you handle transcripts that exceed the context window?
80. [Likely] Build a toy customer-service agent for a scenario such as order status or flight rebooking with two tools (lookup and update), including guardrails.
    - How would you write automated tests for multi-turn behavior?
81. [Likely] Given an existing buggy repository (for example, a classifier service), use AI assistance to find and fix the bug, and add a regression test.

I. Concurrency, performance and production hygiene

82. [Likely] Make a Python preprocessing step that is too slow (pure-Python loops over tokens) 10x faster.
    - When do you vectorize, when do you use multiprocessing, and when does the GIL matter?
83. [Likely] Implement a thread-safe counter and per-key metrics registry (counts, latency histograms with percentiles).
    - How do you compute p50/p95/p99 over a stream without storing every value?
84. [Likely] Write an async function that calls three model services concurrently with an overall deadline and returns partial results if some time out.
85. [Likely] Write a timing harness that measures time-to-first-token and tokens/sec for a streaming LLM client.
86. [Likely] Implement graceful shutdown for a streaming worker: drain in-flight conversations, flush state and stop accepting new work.

87. [Likely] Given Go-like requirements (the Senior MLE role lists Python and Go [16]), sketch how you would implement your rate limiter or batcher with goroutines and channels.
    - What would change compared with your Python asyncio version?
88. [Likely] Add structured logging and tracing IDs that follow a conversation across ASR, orchestrator and model calls in your pipeline code.
89. [Likely] Write unit tests for the streaming transcript aggregator from question 12, including out-of-order and duplicate events.
    - How would you property-test it?

References

1. "Cresta MLE面经" / "Cresta Machine Learning Engineer Tech Phone Interview Experience" (MLE, AI Agent direction; greedy and beam search decoding). 1point3acres, 2025. https://www.1point3acres.com/interview/thread/1140481 (also https://www.1point3acres.com/bbs/thread-1140481-1-1.html)
2. "cresta interview questions (2026)": company page with Cresta SWE/MLE phone screen and online test reports, including an API-implementation phone screen. 1point3acres, 2024-2026. https://www.1point3acres.com/interview/company/cresta
3. "Cresta Fulltime Software Engineer Tech Phone Screen: Job Dispatcher-Executor System Implementation" (listing seen in the site's related-threads sidebar; full text not accessed). 1point3acres, 2026. https://www.1point3acres.com/interview/thread/1172433
4. "Cresta Interview Questions": Cresta-tagged "Design sequence decoding with greedy and beam search" and "Implement greedy and beam decoding". PracHub, 2025. https://prachub.com/companies/cresta
5. "Cresta (CA) Senior Software Engineer Interview Questions". Glassdoor, 2026. https://www.glassdoor.ca/Interview/Cresta-CA-Senior-Software-Engineer-Interview-Questions-EI_IE3180909.0,9_KO10,34.htm
6. "Cresta (CA) Interview Questions & Answers" (CoderPad+Zoom Python round, rate limiter, Senior ML Engineer loop with vibe-coding round). Glassdoor, 2024-2026. https://www.glassdoor.co.in/Interview/Cresta-CA-Interview-Questions-E3180909.htm
7. "Cresta (CA) Software Engineer Interview Questions" (autocomplete, list-to-tree, DS/algo plus SQL joins challenge). Glassdoor, 2024-2026. https://www.glassdoor.sg/Interview/Cresta-CA-Software-Engineer-Interview-Questions-EI_IE3180909.0,9_KO10,27.htm
8. "Cresta interview process". Blind (teamblind.com). https://www.teamblind.com/post/cresta-interview-process-jmfnzjoa
9. "Cresta senior backend SWE interview". Blind (teamblind.com). https://www.teamblind.com/post/cresta-senior-backend-swe-interview-6v1tmjld
10. "Interview for forward deployment engineer at Cresta". Blind (teamblind.com). https://www.teamblind.com/post/interview-for-forward-deployment-engineer-at-cresta-ngswbj44
11. "How Cresta Consolidated All Technical Hiring on CoderPad to Scale Eng Hiring". CoderPad case study. https://coderpad.io/case-study/how-cresta-consolidated-all-technical-hiring-on-coderpad-to-scale-eng-hiring/

12. "Cresta Interview Questions" (third-party guide; generic, unverified). Scoutify, 2026. https://scoutify.com/companies/cresta/interview/ ; Cresta's agent-design example (flight rebooking): "Cresta Conductor: The Agent for AI Agent Development". Cresta blog. https://cresta.com/blog/cresta-conductor-the-agent-for-ai-agent-development
13. "Understanding Cresta's Voice Platform: The Voice Stack". Cresta blog, 2025. https://cresta.com/blog/understanding-crestas-voice-platform-the-voice-stack
14. "Understanding Cresta's Voice Platform: ML Services, Inference Graphs, and Real-Time Intelligence". Cresta blog, 2025. https://cresta.com/blog/understanding-crestas-voice-platform-ml-services-inference-graphs-and-real-time-intelligence
15. "Engineering for Real-Time Voice Agent Latency". Cresta blog. https://cresta.com/blog/engineering-for-real-time-voice-agent-latency
16. "Senior Machine Learning Engineer (Chat Agent)". Cresta job posting, Lever. https://jobs.lever.co/cresta/172fe276-9415-4bae-9b0b-af72cf7cf014
17. "Senior Machine Learning Engineer" (agentic assistance and agent-quality tracks). Cresta posting via ZeroG Talent. https://zerogtalent.com/ai-jobs/cresta/senior-machine-learning-engineer-4249943008

03. ML/LLM Depth for Contact-Center AI: NLP, Speech, Fine-Tuning, Evaluation, RAG, Agents and Latency

Format notes for this round

- **Where depth gets tested.** A Glassdoor review of a Senior ML Engineer loop lists four steps: a recruiter call, a LeetCode-medium coding round, a round done with an LLM coding tool ("vibe coding"), and a system design round [1]. No source reports a standalone "ML deep dive" round for this role. Expect the depth questions below inside the phone screen, the system design round and the hiring-manager conversation.
- **Duration and interviewer.** The reported MLE phone screen (AI Agent direction) was a live Python decoding problem, with follow-ups on how an agent generates sequences [6][4][5]. Interviewers are probably ML engineers from the team you would join. This is an inference from the postings for the Voice Experience, ASR, Agentic Assist and Staff tracks [22][23][24][25][26].
- **Topics recruiters flag in advance.** A recruiter email to a candidate said the interviews would cover "key concepts and fundamentals around LLMs, GenAI, RAG, and AI Agents" [3].
- **Senior-level bar, as inferred from Cresta's postings.**
  - Debug hallucinations, retrieval errors, tool misuse, context drift, prompt brittleness and multi-step reasoning failures [22][23].
  - Evaluate with offline benchmarks, online experiments and LLM-as-a-judge [22][26].
  - Design and deploy enterprise RAG [22].
  - Make trade-offs across the real-time voice stack: ASR, turn detection, LLM and TTS [13][24].
- **Products and tech to anchor your answers.**
  - Ocean-1, Cresta's contact-center foundation model [11].
  - Fine-tuned models that power Knowledge Assist, summarization and chat suggestions [8][9].
  - Thousands of per-customer LoRA adapters served on one base-model cluster at Fireworks [9][10].
  - The voice stack and inference graphs [14][15].
  - LLM-judge evaluators, automated agent testing and the "Swiss-cheese" evaluation model [16][17][19].
  - Hallucination grounding [18], Cresta Conductor [20] and Insights [21].
- **Research caveat.** During this review, live page fetches failed for every cited domain (DNS and proxy errors) and the web-search budget was used up, so no URL could be re-opened. Each reference below was cross-checked against the reviewed reference lists in chapters 01, 04, 05 and 07 of this bank. Unverifiable or generic sources were

removed (see the note at the end of the References). Only questions a candidate source attributes to Cresta are tagged [Reported]. Everything else is [Likely], inferred from Cresta's published engineering work and postings. Confirm the round content with your recruiter.

Decoding and LLM fundamentals

1. Implement greedy decoding and beam search over a next-token probability table given as a Python dictionary (a directed token graph). [Reported] [4][5][6]
    - How do you handle sequences that hit an end token at different lengths? Would you apply length normalization?
    - What is the time and memory complexity in terms of beam width, vocabulary size and depth?
    - When does beam search do worse than greedy decoding for open-ended dialogue?
2. Walk through how an AI agent generates a sequence step by step. Where can decoding strategy change the agent's behavior? [Reported] [6]
    - How would you add constraints, such as forcing a valid tool-call JSON schema?
3. Explain the key concepts and fundamentals of LLMs, GenAI, RAG and AI agents. Then go one level deeper: what in the transformer architecture makes in-context learning and tool use possible? [Reported] [3]
    - Where do pretraining, instruction tuning and preference tuning each change model behavior?
4. Compare temperature, top-k, top-p and min-p sampling. Which would you use for agent-assist suggestions, and which for summaries? [Likely]
    - How do you make outputs reproducible for regression tests?
5. Explain speculative decoding. When would a draft model trained on contact-center data pay off? [Likely] [10]
    - What acceptance rate makes it worthwhile, and how does batch size change that?
6. Explain the KV cache and how its memory grows. How do paged attention and prefix caching apply to long call transcripts? [Likely]
    - Estimate the KV-cache memory for a 7B-class model with 8k context and 200 concurrent sessions. Which knobs (GQA, FP8 cache, context truncation) would you turn first?
7. How do rotary position embeddings work, and how would you extend a model's context window for hour-long calls? What quality risks come with context extension? [Likely]
8. Tokenizers often split account numbers, SKUs, phone numbers and dates badly. How does this hurt extraction and generation, and how would you mitigate it? [Likely] [12]

NLP for conversations

9. What is RAG, and how do you prevent hallucination? [Reported] [2] (reported for a Forward Deployed Engineer technical screen)
    - In what ways does RAG still fail?

    - How would you detect a hallucination that got past your safeguards?
10. How would you build intent or topic discovery over millions of unlabeled call transcripts? [Likely] [21]
    - How do you pick the number of clusters and keep them stable across retrains?
    - LLM labeling of every conversation is expensive at Insights scale. Where would you use embeddings and clustering, and where an LLM?
11. Design a classifier for conversation "moments", such as a missed upsell or a compliance phrase not said. Would you choose a fine-tuned encoder or a prompted LLM? [Likely] [8]
    - How do you handle severe class imbalance and labels that change by customer?
12. You need training labels for a new per-customer classifier and have only a few hundred human labels. How would you use LLM-generated (weak) labels, and how do you measure and correct their noise? [Likely] [8]
13. A model's confidence threshold works for one customer and over-fires for another. How do you calibrate scores per customer, and how do you set thresholds when you have few positives? [Likely]
14. How would you predict escalation, churn risk or negative sentiment early in a live conversation, from partial transcripts? [Likely] [14]
    - How do you trade earliness against precision, and how do you evaluate a prediction made at minute 3 of a 20-minute call?
15. How do transcription errors from ASR propagate into NLP models trained on clean text? How would you make the downstream models robust to them? [Likely] [15]
16. How would you do PII detection and redaction on streaming transcripts with low latency? [Likely]
    - What precision/recall trade-off would you choose, and why?
    - How do you align redaction spans back to audio timestamps, so the recording is redacted too?
17. How do you model speaker roles and turn structure in a two-party transcript for summarization or extraction? [Likely]
18. How would you handle multilingual calls, and calls that switch language mid-conversation? [Likely]
19. You fine-tuned a model on one customer's data and it looks great offline. How do you detect overfitting, leakage across time or agents, and memorization of that customer's PII? [Likely]

Speech and ASR

20. Explain CTC, RNN-T and attention encoder-decoder ASR. Which suits streaming contact-center audio, and why? [Likely] [25]
21. Streaming ASR returns partial transcripts every ~0.5 to 1.5 seconds and revises them. How should downstream models consume these unstable partials? [Likely] [15]
    - When do you trigger an LLM on partial text versus final text?
    - How do you upsert or reconcile results that were computed on revised partials?
22. How do you measure ASR quality beyond WER for downstream tasks such as entity capture (account numbers, names, SKUs)? [Likely] [25]
    - How would you build a domain-specific test set, and how often would you refresh it?

23. Spoken digits, dates, currency and spelled-out emails must become structured text. How would you design and evaluate inverse text normalization and entity formatting for ASR output? [Likely] [25]
24. How would you adapt an ASR model to customer-specific vocabulary: contextual biasing, fine-tuning, or LM rescoring? [Likely] [25]
25. You must choose between third-party ASR vendors and an in-house model (for example, a fine-tuned Whisper-class model). How would you run the bake-off? [Likely] [15][25]
    - How do you catch failure modes WER hides, such as hallucinated text during silence or hold music?
26. Design end-of-turn (endpointing) detection for a voice AI agent. Compare silence-based VAD, acoustic models and semantic models. [Likely] [13][24]
    - How do you measure false barge-ins versus added latency?
    - How would you distill a large turn-detection model into one small enough for real time?
27. Handling 8kHz telephony audio, noise and cross-talk: what augmentation and front-end choices matter? [Likely]
28. Speaker diarization on stereo versus mono call recordings: how do the approaches differ, and how do you evaluate them? [Likely]
29. How would you evaluate and choose a TTS system for a voice agent? Cover naturalness, pronunciation of names and alphanumerics, latency to first audio, and streaming behavior. [Likely] [13][24]
30. What changes when you use speech-to-speech models instead of a cascaded ASR, LLM and TTS stack? [Likely] [24]
    - How do you evaluate quality and controllability in each design?
31. The ASR WebSocket drops in the middle of a call. How would you replay or recover audio so the models don't lose context? [Likely] [15]

LLM fine-tuning and adaptation

32. Cresta fine-tuned its own models to match GPT-4 quality on Knowledge Assist at lower cost and latency. How would you set up that project end to end? [Likely] [8][9]
    - How do you decide a fine-tuned model is "as good as" the frontier model?
33. Ocean-1 is described as a contact-center foundation model. When would you do continued pretraining on domain conversations rather than only instruction tuning? How would you choose the data mix? [Likely] [11]
34. Compare full fine-tuning, LoRA, QLoRA and prompt tuning for per-customer adaptation. [Likely] [9][10]
    - How do you choose LoRA rank and target modules?
35. You serve thousands of per-customer LoRA adapters on one base model. What are the training, versioning and serving implications? [Likely] [10]
    - How do you handle a customer with too little data for its own adapter?
    - What breaks if you upgrade the base model?
    - How would you automatically evaluate and gate thousands of adapters before each release? [17]
36. How do you build SFT data from contact-center transcripts? Cover label sources, agent-quality filtering and deduplication. [Likely] [8]

37. When would you train on synthetic conversations generated by a stronger LLM? How do you keep synthetic data diverse, and how do you detect that it is degrading the model? [Likely] [17][20]
38. When would you use preference optimization (DPO, RLHF, or RLAIF) for agent-assist suggestions? Where would the preference signal come from? [Likely]
    - Can you use agent accept/reject clicks as rewards? What biases do they carry?
39. How do you prevent catastrophic forgetting of instruction following when you fine-tune on narrow domain data? [Likely] [11]
40. Walk through the GPU memory math for fine-tuning a 7B and a 70B model. When do you need FSDP or ZeRO, activation checkpointing or mixed precision? [Likely] [23]
41. How would you fine-tune an embedding model for contact-center terminology, product SKUs and policy language? [Likely] [12]
    - How do you mine hard negatives?
42. When would you call a frontier API rather than run a self-hosted fine-tuned model? Discuss cost, latency, data residency and quality. [Likely] [27]

Distillation and model compression

43. Explain knowledge distillation for LLMs: logit-level versus sequence-level (training on teacher outputs). [Likely]
    - Which would you use when the teacher is a closed API?
44. Design a pipeline that distills a large teacher into a small model for real-time summaries or suggestions. [Likely] [8]
    - How do you choose teacher prompts and filter teacher outputs?
    - How do you know when the student is good enough to ship?
45. An LLM judge is too slow and expensive to score every production conversation. How would you distill it into a small classifier for online monitoring, and how do you keep the two aligned over time? [Likely] [16]
46. Compare quantization methods (INT8, FP8, 4-bit GPTQ/AWQ) in terms of quality loss and throughput. How does quantization interact with LoRA adapters? [Likely] [10]
47. When would you prune or use a smaller architecture rather than distill? [Likely]
48. How do you detect regressions in rare but high-stakes behaviors (such as compliance disclosures) after compression? [Likely]

Retrieval-augmented generation

49. Design a RAG system for Knowledge Assist that answers agent questions mid-call from a customer's knowledge base. [Likely] [8][12][22]
    - How do you reformulate the query from a noisy, partial conversation?
    - How do you choose chunk size and structure for knowledge-base articles, FAQs and policy PDFs?
50. Knowledge Assist can surface answers before the agent asks. How do you decide when to trigger retrieval proactively, and how do you keep false triggers from distracting agents? [Likely] [9]
51. Compare hybrid retrieval (BM25 plus dense) with dense only for domain jargon and SKUs. [Likely] [12]

52. Where would you add a reranker (cross-encoder or LLM-based), and how do you fit it into a real-time latency budget? [Likely] [12][13]
53. How do you evaluate retrieval separately from generation? Which metrics, and where do labels come from? [Likely]
    - How do you detect knowledge-base gaps, where the answer simply doesn't exist?
54. Knowledge bases are often stale or contradictory. How do you make them "AI-ready", and how do you handle conflicting sources at answer time? [Likely] [12]
55. How do you keep the index fresh when articles change daily? What does upgrading the embedding model cost, and how do you re-index without downtime? [Likely] [12]
56. How do you enforce grounding and citation? When should the model refuse or fall back? [Likely] [18]
57. How do you adapt answer length and format by channel (voice, chat, email)? [Likely] [12]
58. Diagnose this: RAG answers look fluent but agents report they are wrong 15% of the time. Walk through your debugging. [Likely] [22]
    - How do you tell retrieval failures from generation failures?
59. When would you use long-context stuffing instead of retrieval? [Likely]
60. How do you handle multi-tenant isolation in a vector index, so no customer can retrieve another's documents? [Likely]

Agents and tool use

61. Explain function calling end to end. How do you make tool-call arguments reliable? [Likely] [22]
62. Tool calls fail, time out or are retried. How do you design tool schemas and error handling so the agent recovers, and so non-idempotent actions (refunds, cancellations) never run twice? [Likely] [23]
63. Design a customer-facing AI agent that authenticates a caller, looks up an order and processes a refund. [Likely]
    - Where do deterministic guardrails sit, and where does the LLM decide?
    - How do you prevent a refund from running without verified identity?
64. A customer's standard operating procedure has many branches. Would you encode it as an explicit workflow graph, a single long prompt, or a router over sub-agents? Which fails least in a regulated setting? [Likely] [26]
65. What are the most common agent failure modes in production? How would you detect each one (tool misuse, context drift, looping, multi-step reasoning breakdown)? [Likely] [22][23]
66. Explain AI agent frameworks and planning patterns (ReAct, plan-and-execute, state machines). Which fits a regulated contact center, and why? [Likely] [22]
67. How do you manage memory and context across a 30-minute call, including summarization of earlier turns? [Likely]
68. When do you hand off from the AI agent to a human, and how do you design and evaluate that escalation policy? [Likely]
69. How do you defend a voice or chat agent against prompt injection from customers? [Likely]

70. Cresta Conductor is described as an agent that helps build AI agents. How would you evaluate an LLM that writes or edits another agent's prompts and configuration? [Likely] [20]
    - What stops it from shipping an edit that passes tests but breaks real conversations?
71. Agentic assist (agent-facing) versus AI agent (customer-facing): how do the risk tolerance, latency budget and evaluation differ? [Likely] [22][24][26]

Evaluation

72. Design an evaluation framework for an AI agent: trajectory evaluation versus final-outcome evaluation. [Likely] [16]
    - What would you check deterministically, and what would you hand to an LLM judge?
73. Explain a layered ("Swiss-cheese") approach to agent evaluation. Which layers would you build first, and how do you know which failures slip through every layer? [Likely] [19]
74. How do you build and calibrate LLM-as-a-judge evaluators against human experts? [Likely] [16][17][18]
    - How do you measure judge-human agreement, and what threshold is acceptable?
    - How do you detect drift in a judge over time, or when you swap its model?
    - How do you control position, verbosity and self-preference bias?
75. A prompt change moves your eval score from 82% to 84% on 150 test conversations, and the agent is non-deterministic. Is that a real improvement? How would you decide, statistically, whether to ship? [Likely] [7]
76. Design a customer simulator to test AI agents before launch. How do you make simulated customers realistic and diverse? [Likely] [16][17]
    - Why run each simulated conversation several times? How do you report variance?
77. How do you measure the hallucination rate, and how do you use it to choose models and prompts? [Likely] [18]
78. How would you evaluate LLM-extracted structured fields (contact reason, outcome, sentiment) across millions of conversations, when there is no ground truth for most of them? [Likely] [21]
79. Offline metrics improved but online agent adoption fell. What could explain this, and how would you investigate? [Likely] [22]
80. How would you design an online A/B test for agent-assist suggestions in a contact center? Consider randomization unit, interference between agents, and business metrics such as AHT and CSAT. [Likely]
81. How do you evaluate summarization quality at scale: reference-based metrics, LLM judges, or human ratings? [Likely] [8]
82. How do you evaluate a voice agent end to end, including latency, interruptions and task success? [Likely] [13][24][25]
83. How do you build regression suites that stop a prompt or model change from breaking a customer's agent? [Likely] [7][17]

84. How would you monitor data drift and model quality in production for per-customer models? [Likely]

Latency optimization and serving

85. How would you deploy a model in production? [Reported] [1]
    - Go deeper for an LLM: which serving engine, precision and batching settings would you choose, and how would you canary and roll back a new model?
86. A voice agent must respond within about one second of the caller finishing. Break down the latency budget across telephony, ASR, endpointing, LLM, TTS and network. [Likely] [13]
    - Which components would you measure individually, and how would you run end-to-end tests?
87. How do you cut time-to-first-token and time-to-first-audio? Discuss streaming LLM output into TTS, filler phrases and prefetching. [Likely] [13]
88. How would you speculatively start LLM generation before end-of-turn is confirmed? What happens if the user keeps talking? [Likely] [13][24]
89. Explain continuous batching, and the trade-off between throughput and tail latency (p99) for real-time assist. [Likely]
90. Long transcripts make prefill expensive. Explain chunked prefill and prefill/decode disaggregation. When would each help a real-time assist workload? [Likely]
91. Design an inference graph that runs several models (ASR, intent, moment detection, suggestion LLM) per utterance. How do you orchestrate them under customer-specific policies? [Likely] [14]
    - How do you avoid redundant calls and handle partial failures?
92. Multi-LoRA serving: how do adapter swapping and batching across adapters affect latency? What limits remain, such as quantization compatibility? [Likely] [10]
93. Estimate the GPUs needed to serve real-time suggestions for 10,000 concurrent calls. State your assumptions about utterance rate, tokens per request and throughput per GPU. [Likely] [14]
94. How would you cut cost per conversation with model routing or cascades (a small model first, escalating to a larger one)? How do you choose the escalation signal? [Likely] [8][27]
95. How would you autoscale GPU inference for contact-center traffic with strong daily peaks? [Likely]
96. When would you cache LLM outputs or retrieval results in real-time assist, and what invalidates them? [Likely]

ML system design prompts (depth-heavy)

97. Design real-time agent assist: streaming audio in, suggestions and knowledge cards out. Draw the full pipeline. [Likely] [14][15]
    - Where does state live per call, and how do you scale to tens of thousands of concurrent calls?
98. Design automatic post-call summarization and disposition for every call, with quality guarantees. [Likely] [8]

99. Design a conversation analytics or "AI analyst" system that answers natural-language questions over millions of conversations. [Likely] [18][21]
    - How do you keep aggregate statistics accurate when an LLM produces them?
100. Design per-customer model customization (adapters, prompts, policies) with safe rollout and rollback. [Likely] [10][14]
101. Design an evaluation platform for AI agents that catches regressions across prompt and model changes. Go deep on the metrics, judges and statistical gates. [Reported] [7]
102. Design a feedback loop that turns agent accept/reject actions and supervisor reviews into training data. [Likely] [17]

Research, papers and ML foundations

103. Discuss a recent paper on LLM evaluation, distillation or speech that you found useful. What are its limitations for contact-center data? [Likely]
104. What are "strong pre-LLM ML foundations" good for in an LLM-first team? Give an example where a classical model beat an LLM. [Likely] [22]
105. Explain backpropagation through attention. Why does layer normalization placement matter for training stability? [Likely]
106. How do you explain to a customer why an AI suggestion or score was produced? [Likely]
107. How do you handle data privacy, retention and consent when you train on customer conversations? [Likely] [27]

References

1. Glassdoor, "Cresta (CA) Interview Questions" (Senior Machine Learning Engineer loop: recruiter call, LeetCode-medium, vibe-coding and system design rounds; "How would you deploy a model in production?"), 2025-2026. https://www.glassdoor.co.in/Interview/Cresta-CA-Interview-Questions-E3180909.htm
2. Glassdoor, "Cresta (CA) Interview Experience and Questions" (FDE technical screen: RAG and hallucination), 2026. https://www.glassdoor.com/Interview/Cresta-CA-Interview-Questions-E3180909.htm
3. Blind, "Anyone interview with Cresta lately?" (recruiter email: "key concepts and fundamentals around LLMs, GenAI, RAG, and AI Agents"), 2025. https://www.teamblind.com/post/anyone-interview-with-cresta-lately-zvj0uhq2
4. PracHub, "Cresta Interview Questions" (company page; "Design sequence decoding with greedy and beam search"), 2025. https://prachub.com/companies/cresta
5. PracHub, "Cresta Machine Learning Interview Questions" ("Implement greedy and beam decoding"). https://prachub.com/companies/cresta/categories/machine-learning
6. 1point3acres, "Cresta Machine Learning Engineer Tech Phone Interview Experience" (MLE, AI Agent direction; greedy and beam-search decoding), 2025. https://www.1point3acres.com/interview/thread/1140481
7. PracHub, "Cresta ML System Design Interview Questions" ("Design an Evaluation Platform for AI Agents Across Prompt and Model Changes"), 2026. https://prachub.com/companies/cresta/categories/ml-system-design

8. Cresta Blog, "How Fine-Tuned LLMs Power Knowledge Assist, Summarization and Chat Suggestions", 2024. https://cresta.com/blog/how-fine-tuned-llms-power-knowledge-assist-summarization-and-chat-suggestions
9. Fireworks AI Blog, "Story: Cresta Knowledge Assist", 2024. https://www.fireworks.ai/blog/story-cresta-knowledge-assist
10. Fireworks AI Blog, "Multi-LoRA", 2024. https://fireworks.ai/blog/multi-lora
11. Cresta Blog, "Introducing Ocean-1: World's First Contact Center Foundation Model", 2023. https://cresta.com/blog/introducing-ocean-1-worlds-first-contact-center-foundation-model
12. Cresta Blog, "AI-Ready Knowledge for Contact Centers: Closing the Gap Between KBs and AI", 2025. https://cresta.com/blog/ai-ready-knowledge-for-contact-centers-closing-the-gap-between-kbs-and-ai
13. Cresta Blog, "Engineering for Real-Time Voice Agent Latency". https://cresta.com/blog/engineering-for-real-time-voice-agent-latency
14. Cresta Blog, "Understanding Cresta's Voice Platform: ML Services, Inference Graphs and Real-Time Intelligence", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-ml-services-inference-graphs-and-real-time-intelligence
15. Cresta Blog, "Understanding Cresta's Voice Platform: The Voice Stack", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-the-voice-stack
16. Cresta Blog, "How Cresta Grades Its Agents: Evaluators in an LLM World". https://cresta.com/blog/how-cresta-grades-its-agents-evaluators-in-an-llm-world
17. Cresta Blog, "Cresta Launches Automated AI Agent Testing: Confidence in Every Conversation". https://cresta.com/blog/cresta-launches-automated-ai-agent-testing-confidence-in-every-conversation
18. Cresta Blog, "Grounding Reality: How Cresta Tackles LLM Hallucinations in Enterprise AI". https://cresta.com/blog/grounding-reality---how-cresta-tackles-llm-hallucinations-in-enterprise-ai
19. Cresta Blog, "Why AI Agent Evaluations Fail, and How the Swiss-Cheese Model Prevails". https://cresta.com/blog/why-ai-agent-evaluations-fail----and-how-the-swiss-cheese-model-prevails
20. Cresta Blog, "Cresta Conductor: The Agent for AI Agent Development". https://cresta.com/blog/cresta-conductor-the-agent-for-ai-agent-development
21. Cresta Blog, "Introducing the Next Generation of Cresta Insights: A Complete System for Enterprises to Understand Their CX", 2026. https://cresta.com/blog/introducing-the-next-generation-of-cresta-insights-a-complete-system-for-enterprises-to-understand-their-cx
22. Cresta on Greenhouse, "Senior Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4249943008
23. Cresta on Greenhouse, "Staff Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4741283008
24. Cresta on Greenhouse, "Senior Machine Learning Engineer, Voice Experience". https://job-boards.greenhouse.io/cresta/jobs/5155675008
25. Cresta on Greenhouse, "Senior Machine Learning Engineer (ASR)". https://job-boards.greenhouse.io/cresta/jobs/5101011008
26. Built In Vancouver, "Senior Machine Learning Engineer, Cresta" (Agentic Assist track). https://builtinvancouver.org/job/senior-machine-learning-engineer/8619914
27. CX Today, "Cresta CEO Ping Wu: Many Contact Centers Misunderstand the Value of LLMs". https://www.cxtoday.com/contact-center/cresta-ceo-ping-wu-many-contact-cent

ers-misunderstand-the-value-of-llms/

Removed or replaced in review: the Blind Cresta AI SWE AMA thread (it no longer supported any question here). The Blind company-posts page (its "know GenAI/RAG stuff" quote could not be confirmed, and chapter 01 had already dropped it). The Scoutify guide (a generic template, not candidate reports). The startup.jobs and Built In Bristol reposts, replaced by the original Greenhouse postings (refs 24 and 25). The opaque `cresta.com/?p=30805` link, replaced by the canonical voice-stack post (ref 15). Generic filler was removed: "implement logistic regression or k-means from scratch" (a coding item covered in chapter 02), "design an ML pipeline with a feature store" (too generic), and "walk through a project you led" (covered in chapter 05). The bias-variance question was rewritten into a senior-level overfitting and leakage question (Q19).

04. ML System Design: Real-Time Agent Assist, AI Agents, Conversation Intelligence, QA Automation, Knowledge Assist and Multi-Tenant LLM Serving

Tagging system.

- **[Reported] [n]**: a candidate report or question bank attributes this question, or a close variant, to Cresta. [n] points to the References list.
- **[Likely]**: inferred from Cresta's products, engineering blog and ML job postings. No candidate has reported it. Where a [Likely] item scales up a coding question reported for another round, the reference points to that coding report.

Diagram prompts. Most design questions include a "Sketch" follow-up. It names the diagram the interviewer is likely to ask you to draw on the whiteboard. The prompt names the boxes and arrows to include. It does not give the answer.

Research limits. In this review the egress proxy refused every cited domain (cresta.com, prachub.com, Glassdoor, Blind, 1point3acres, Greenhouse, Built In, fireworks.ai), and the shared web-search budget was already used up, so no cited page could be opened and no new candidate reports could be searched for. Instead, each reference was checked against the reviewed reference lists and excerpts in chapters 01, 03 and 07 of this guide. A citation that no excerpt supported was removed (see the note after the References). The only Cresta ML system design prompts on record are still the agent evaluation platform and "How would you deploy a model in production?". Two prompts from adjacent Cresta rounds (RAG and hallucination for FDE; decoder component design) were added as [Reported] variants, and their tags name the round. All other new questions are [Likely]. Confirm the round format with your recruiter.

Format notes for this round

- **Round exists and lasts about an hour.** A 2025-26 Senior MLE report lists a systems design round after a recruiter call, an LC-medium round and a "vibe coding" round. The sample prompt was "How would you deploy a model in production?" [1]. Senior/Staff SWE reports describe one hour of system design next to one hour of coding and a hiring-manager chat (SWE loop) [3].
- **Prompt flavor: LLM agents and evaluation.** PracHub lists "Design an Evaluation Platform for AI Agents Across Prompt and Model Changes" as a Cresta ML system

design question. A linked Cresta SWE interview experience describes building an eval platform for AI agents [5][6].

- **Adjacent reported prompts.** A 2026 FDE technical screen asked "What is RAG, and how do you prevent hallucination?" [26]. PracHub lists "Design sequence decoding with greedy and beam search" for Cresta [28]. A rate-limiter question and a worker-pool/orchestrator question appear in Cresta coding rounds [1], and so does a job dispatcher/executor question (SWE, 2026) [29]. Design interviewers may scale any of these up.
- **Adjacent rounds.** A 2025 Cresta MLE phone screen focused on greedy and beam decoding [7], so design interviewers may probe generation and decoding choices. Other Cresta engineering loops have included a project-presentation round, and the process takes about 25 days on average [2].
- **Interviewer.** Most likely a senior or staff MLE from the target track: Agentic Assist, Agent and System Quality, Voice/ASR or Insights. This is inferred from postings [8][9][10][11].
- **Senior bar (from postings).**
  - End-to-end ownership from research to production [11].
  - Evaluation of non-deterministic systems: offline benchmarks, online experiments and LLM-as-a-judge [9].
  - Diagnosing hallucination, retrieval errors, tool misuse, context drift and prompt brittleness [8][9].
  - Designing for latency, cost, scalability and security, including real-time streaming inference for voice [10].
  - Recruiters have flagged LLM, RAG and agent fundamentals as in scope [4].
- **What to expect (inferred).** Requirements, then an architecture sketch, then one or two deep dives: latency budget, evaluation, multi-tenancy or rollout. Expect probes on how you measure quality without labels and how you ship safely to many enterprise customers.

Questions

A. Reported prompts (this round, plus close variants from adjacent Cresta rounds)

1. Design an evaluation platform for AI agents that catches regressions when prompts or models change. [Reported] [5][6]
   - Sketch: how do the test-set store, scenario simulator, agent-under-test runner, judge/evaluator layer, results store and CI gate connect?
   - How do you version prompts, model IDs, tools, knowledge snapshots and test sets so that a run is exactly reproducible?
   - Which checks are deterministic (tool-call arguments, policy rules), and which need an LLM judge?
   - How many reruns per scenario do you need given non-determinism? What statistical test declares a regression?
   - How do you score the trajectory (tool calls, reasoning steps) separately from the final outcome?

    - How would you scale it to thousands of scenarios across hundreds of customer-specific agents? What does a full run cost?
    - Who consumes the results (ML engineers, forward-deployed engineers, customers), and how does that change the UX?
2. How would you deploy a model in production? [Reported] [1]
    - Sketch: what path does a model take from the registry through shadow, canary and full rollout, with monitoring and rollback?
    - What would you do differently for a real-time model (assist suggestions) and a batch model (post-call QA scoring)?
    - What signals in the first 24 hours trigger an automatic rollback?
    - How do you manage per-customer model versions and configuration?
3. What is RAG, and how do you prevent hallucination? (Reported as a conceptual question in a Forward Deployed Engineer technical screen. Expect a senior MLE design round to extend it: build RAG over an enterprise knowledge base for a customer-facing agent and show where hallucinations are caught.) [Reported] [26]
    - Sketch: ingestion, chunking, index, query rewriting, retrieval, re-ranking, generation with citations, groundedness check, fallback ("I don't know" or hand off to a human).
    - At which stage does each hallucination type start (retrieval miss, stale article, generator going beyond the retrieved text), and which check catches it?
    - What do you do when the groundedness check fails mid-conversation on a voice call?
4. Design sequence decoding with greedy and beam search as a reusable component. (PracHub title; the matching 2025 MLE phone screen was for the AI Agent team.) [Reported] [28][7]
    - What interface lets greedy, beam and sampling strategies share code and plug into a serving loop?
    - How does beam width trade quality against latency and memory when the component runs inside a real-time service?
    - How would you add stop conditions, length penalties and constraints (banned phrases, required schema) without forking the code?

B. Real-time Agent Assist (live guidance for human agents)

5. Design real-time Agent Assist: listen to a live call and show the human agent next-best-action hints, behavior prompts and suggested responses. [Likely]
    - Sketch: telephony/CCaaS audio fork, streaming ASR, transcript bus, inference graph, suggestion service, agent desktop UI. Where are the queues?
    - What is the end-to-end latency budget from spoken words to on-screen hint? How do you split it across ASR, models and UI?
    - How do you handle partial transcripts compared with final ones? Do you trigger on partials? (Cresta publishes both, about once per second [16].)
    - How do you decide when to show a hint and when to stay quiet, so agents don't get alert fatigue?
    - How do you measure impact on handle time, CSAT or conversion when top agents already behave well?
6. Design the "moment detection" layer that fires guidance triggers (for example "customer mentioned cancellation" or "agent forgot disclosure") from a live transcript.

[Likely]
    - Which triggers are rules, which are small classifiers, and which are LLM calls? How do you decide?
    - How do you let customers define new triggers without a training run?
    - How do you evaluate trigger precision and recall per customer with few labels?
7. Design suggested-response (smart compose) generation for chat agents. [Likely]
    - Sketch: which context window builder, retrieval, generator, ranker and UI feedback loop would you draw?
    - How do you personalize to a customer's tone and policies: prompting, retrieval of good past replies, or fine-tuning? (See [20].)
    - What offline metric best predicts agent acceptance rate?
    - How do you avoid suggesting something that violates compliance?
8. Design live call summarization that is ready the moment the call ends (auto-notes and dispositions). [Likely]
    - Do you summarize incrementally during the call or once at the end? What does each cost in latency and money?
    - How do you map the summary to the customer's CRM fields and disposition codes?
    - How do you measure factual consistency of summaries at scale?
9. Design an inference graph that runs many models per conversation (ASR, PII redaction, intent, sentiment, hints, summarization) with customer-specific configuration. [Likely]
    - Sketch: a DAG of model nodes with dependencies. Where do streaming nodes end and batch nodes begin? (See [15].)
    - How do you skip nodes a customer has not enabled, and how do you add a node without redeploying the graph?
    - How do you stop one slow node from blocking real-time outputs (timeouts, fallbacks, partial results)?
    - How would you trace a single conversation through the graph for debugging?
10. Design a real-time sentiment and escalation-risk detector for live calls. [Likely]

- How do you combine acoustic signals (prosody, overlap, silence) with lexical ones?
- How do you set per-customer alert thresholds and keep them calibrated over time?

11. Design supervisor live-monitoring: a dashboard that shows which of 2,000 concurrent calls need attention right now. [Likely]

- Which scoring and ranking would you use, and how often do you refresh it?
- How do you scale the fan-out of updates to many supervisor screens?

12. How would you A/B test a new Agent Assist model when agents, queues and customers differ widely? [Likely]
    - Do you randomize by agent, by call or by team? What about spillover between agents?
    - How long must the test run to detect a 2% handle-time change?
13. Design the delivery layer that pushes live hints, transcripts and knowledge cards to tens of thousands of agent desktops (browser or CCaaS-embedded widget). [Likely]
    - WebSockets, server-sent events or polling? How do you handle reconnects, ordering and duplicate hints?

    - How do you keep the hint tied to the correct call when an agent transfers, holds or conferences?
14. Design assist for chat agents who handle three to five conversations at once: which conversation needs the agent's attention next, and what to suggest there. [Likely]
    - Which signals rank the conversations (customer wait time, sentiment, SLA, open question)?
    - How do you evaluate the ranking offline?
15. Design an offline evaluation of a new Agent Assist hint model by replaying historical calls, before any live A/B test. [Likely]
    - How do you judge whether a hint shown at time t would have helped, when the agent never saw it?
    - How do you combine replay metrics (precision of hint timing, coverage) with a small human review?
16. In suggested-response generation, would you use greedy decoding, beam search or sampling of n candidates followed by a ranker? Justify the choice on latency, diversity and acceptance rate. [Likely] [7]
    - How does the choice change for a fine-tuned small model compared with a frontier API?
17. Design zero-shot intent and moment detection that works for a new customer on day one, then improves as data arrives. [Likely]
    - How do you start with LLM prompts over customer-provided descriptions and move to cheaper models once labels accumulate?
    - How do you detect the point at which the cheap model is good enough to replace the LLM?
18. Design real-time translation for a live call, where the agent works in English and the customer speaks another language. [Likely]
    - Where does translation sit relative to ASR, hints and the agent's reply (text or synthesized speech)?
    - How do you stop translation errors from triggering wrong hints or compliance alerts?

C. Voice AI agents (autonomous customer-facing agents)

19. Design a voice AI agent that handles inbound customer calls end to end, with responses in under a second. [Likely]
    - Sketch: SIP/telephony, VAD and endpointing, streaming ASR, dialog/LLM orchestrator with tools, streaming TTS, and back. Label the latency of every hop.
    - Which component metrics do you track (ASR final latency, LLM time-to-first-token, TTS time-to-first-byte), and how do you measure end-to-end latency? (See [14].)
    - How do you detect end of turn without cutting off slow speakers?
    - How do you handle barge-in when the customer interrupts mid-sentence?
    - Which network-level choices matter: co-location, connection reuse, streaming APIs, warm pools?
20. Design the tool-use layer for an AI agent that can look up orders, issue refunds and update accounts in customer systems. [Likely]

    - How do you constrain tools so the agent cannot make unauthorized commitments (refund limits, identity verification first)?
    - How do you handle slow or failing downstream APIs during a live voice call? What does the agent say while waiting?
    - How do you test tool calls safely without touching production customer systems?
21. Design the handoff from an AI agent to a human agent. [Likely]
    - What signals trigger escalation, and how do you avoid escalating too early or too late?
    - What context (summary, verified identity, attempted actions) do you pass to the human, and in what format?
22. Design an agent-building system in which a customer's business process (SOPs, flows, policies) becomes a working AI agent. [Likely]
    - Should the agent be a structured flow, a free-form LLM with tools, or a hybrid? What trade-offs does each bring?
    - How can an "agent that builds agents" draft the configuration from historical conversations? (See [25].)
    - How do you keep the agent compliant when the SOP changes?
23. Design guardrails for a customer-facing LLM agent: prompt injection, off-topic requests, PII leakage and policy violations. [Likely]
    - Where in the pipeline do guardrails run (input, planning, output), and what latency does each add?
    - How do you ground responses and detect hallucinations before they are spoken? (See [23].)
    - Which guardrails are small classifiers, and which are LLM checks?
24. Design a multi-agent architecture with a router and specialized sub-agents (billing, tech support, sales). [Likely]
    - When does this beat one agent with many tools?
    - How do you keep shared conversation state consistent across handoffs?
    - How do you debug a failure that spans several agents?
25. How would you keep a voice agent's latency stable during peak traffic (Monday morning, outages that cause call spikes)? [Likely]
    - What degrades first, and what is your graceful-degradation plan (smaller model, cached responses, queueing)?
26. Design multilingual support for a voice agent (language ID, code-switching, per-language ASR and TTS). [Likely]
    - How do you evaluate quality in languages your team does not speak?
27. Design context management for a long call (30-60 minutes, many tool calls) so the voice agent stays coherent without blowing the latency or token budget. [Likely]
    - What do you keep verbatim, what do you summarize, and what do you store as structured state (verified identity, open tasks)?
    - How do you test for context drift late in a long conversation? (See [8][9].)
28. Design identity verification inside a voice AI agent (knowledge-based questions, one-time passcodes, account lookup) before it can take sensitive actions. [Likely]
    - How do you enforce "verify first" in code rather than trusting the prompt?
    - How do you handle ASR errors on spelled names, dates of birth and account numbers?
29. Design detection of social-engineering and fraud attempts against a customer-facing AI agent (account takeover, refund abuse, repeated probing). [Likely]

    - Which signals are per-call, and which need cross-call aggregation per account or phone number?
    - What does the agent do when risk is high but not certain?
30. Design an outbound voice AI agent for campaigns (payment reminders, appointment confirmations), including answering-machine and IVR detection. [Likely]
    - How do you classify human, voicemail and IVR within the first seconds of the call?
    - How do calling-window rules, consent records and retry policies enter the design?
31. Design memory across conversations for returning customers, so an AI agent can pick up an issue from last week. [Likely]
    - What do you store, for how long, and how do you respect retention and deletion rules?
    - How do you stop stale or wrong memories from steering the agent?
32. Design an AI agent for asynchronous messaging channels (SMS, WhatsApp, web chat) where one conversation can span hours or days. [Likely]
    - How do you persist and resume conversation state, and how do you handle a customer who replies after their session has expired?
    - What changes in latency targets, evaluation and handoff compared with voice?

D. Agent evaluation, simulation and quality

33. Design a customer simulator that generates synthetic callers to test an AI agent before launch. [Likely]
    - Sketch: how do the persona/goal generator, simulated customer LLM, agent under test, conversation transcript and evaluators connect?
    - How do you seed personas and goals from real conversation data so coverage matches production? (See [24].)
    - How do you stop the simulator and the agent from sharing the same blind spots (same base model)?
34. Design a layered ("Swiss-cheese") reliability program for a production AI agent. [Likely]
    - What does each layer catch: unit tests on tools, simulated conversations, human review, production monitors? What does each miss? (See [13].)
    - How do production failures become new regression tests automatically?
35. Design an LLM-as-a-judge evaluator service used for both agent testing and production scoring. [Likely]
    - How do you calibrate judges against expert labels, and what agreement level is good enough? (See [12].)
    - How do you detect judge drift when you upgrade the judge model?
    - How do you stop position bias, verbosity bias and self-preference from skewing results?
    - How do you keep per-conversation judging cost manageable at millions of conversations?
36. Design production monitoring for AI agents across hundreds of enterprise deployments. [Likely]
    - Which label-free signals indicate quality problems (escalation rate, repeated questions, abandonment, tool error rates)?

    - How do you catch a regression that affects only one customer or one intent?
    - How do you alert without paging on noise?
37. How do you measure whether an AI agent "resolved" a call when there is no survey and no disposition code? [Likely]
    - How would you validate an inferred-outcome model without ground truth?
38. Design the release process for a prompt change that ships to 50 customer agents at once. [Likely]
    - Which tests gate it, and how do you stage the rollout across customers?
    - What is your rollback story when a customer has customized the prompt?
39. Design a "shadow mode" launch in which a new AI agent runs silently beside human agents on live traffic, to estimate containment and risk before it speaks to customers. [Likely]
    - How do you compare the agent's proposed actions with what the human actually did?
    - Which disagreements are agent errors, and which are human errors or policy differences?
40. Extend the customer simulator to voice: synthetic callers with TTS voices, accents, background noise, interruptions and silence. [Likely] [24]
    - Which failures appear only in audio (endpointing, barge-in, ASR on spelled data) and never in a text simulation?
    - How do you keep voice simulation cheap enough to run on every release?
41. Design a test harness that measures how robust the LLM layer is to ASR errors. [Likely] [27]
    - How do you inject realistic ASR noise (substitutions on entities, dropped words, wrong partials)?
    - When ASR errors hurt the outcome, do you fix ASR, the prompt, or add a confirmation step?
42. Design the curation pipeline for "golden" evaluation sets taken from production conversations. [Likely]
    - How do you sample (stratified by intent, failure-weighted, new-scenario detection) and redact PII before engineers see the data?
    - How do you keep the set from going stale or leaking into training data?
43. Design an audit and replay system that can reproduce exactly why an AI agent said or did something on a call three months ago, for a regulated customer. [Likely]
    - What must you log (prompt version, model version, retrieved documents, tool inputs and outputs, guardrail decisions), and at what storage cost?
    - How do you replay when the model or the knowledge base has changed since then?

E. Conversation intelligence and insights

44. Design topic discovery over millions of unlabeled contact-center conversations, producing a hierarchical taxonomy that admins can edit. [Likely]
    - Sketch: a batch pipeline from conversations to embeddings or LLM extraction, clustering, LLM labeling, the taxonomy store, and the assignment service.
    - How do you keep assignments stable when admins merge, split or rename topics? (See [19].)
    - How do you overlay outcomes (resolution, sales, churn) to rank problem areas?

45. Design an "AI analyst" that answers open-ended business questions over a customer's conversation corpus ("Why did cancellations rise last week?"). [Likely]
    - How do you combine per-conversation LLM extraction with SQL-style aggregation?
    - How do you stop it from presenting a weak correlation as a causal finding?
    - How do you bound cost when a question would touch ten million conversations?
46. Design a driver-analysis system that finds which agent behaviors correlate with good outcomes (the basis for coaching and assist hints). [Likely]
    - How do you control for confounders like call type, customer mood and queue?
    - How do you turn the findings into real-time guidance triggers?
47. Design a real-time emerging-issue detector that flags new spikes (an outage, a billing bug) within minutes. [Likely]
    - How do you separate a true spike from seasonality and noise?
    - How do you name the new issue automatically?
48. Design structured extraction from conversations (reason for call, products mentioned, competitor mentions) into a customer's data warehouse. [Likely]
    - How do you enforce schemas and handle extraction failures?
    - How do you backfill when the schema changes?
49. How would you design the storage and indexing layer for searching billions of utterances by keyword, semantic meaning and metadata filters? [Likely]
    - Which parts use a vector index, which use an inverted index, and how do you do hybrid ranking?
    - How do you enforce per-tenant isolation and retention policies in the index?
50. Design the event pipeline that turns live and post-call conversation events into analytics tables, with late-arriving transcripts and reprocessing when a model changes. [Likely]
    - How do you get idempotent, exactly-once results per conversation?
    - How do you recompute a year of a derived metric after an extraction model upgrade without double counting?

F. QA automation and coaching

51. Design automated QA that scores 100% of calls against each customer's rubric. [Likely]
    - Sketch: how do the rubric definition UI, rubric compiler (prompt or classifier per item), batch scoring workers, evidence highlighting and the QA analyst review queue connect?
    - How do you turn a free-text rubric item ("agent showed empathy") into a reliable, auditable judgment?
    - How do you show evidence (timestamps, quotes) so supervisors trust the score?
    - How do you handle disputes, and feed corrections back into the scorer?
52. How do you measure and improve agreement between automated QA scores and human QA analysts? [Likely]
    - What do you do when the human analysts don't agree with each other?
53. Design a system that lets a customer add a new QA criterion and have it live across past and future calls within an hour. [Likely]
    - Few-shot prompting, a small classifier, or both? How do you validate before going live?

    - What does a backfill over a year of calls cost?
54. How would you stop agents from gaming the automated QA scorer (for example, saying keywords without the real behavior)? [Likely]
55. Design an automated coaching system that picks which behavior each agent should work on and tracks improvement. [Likely]
    - How do you attribute improvement to coaching and not to seasonality or call mix?
56. Design compliance monitoring (required disclosures, prohibited statements) with near-zero miss tolerance in regulated industries. [Likely]
    - How do you trade precision against recall when a miss means regulatory risk?
    - Which checks run in real time, and which run post-call?
57. Design speaker attribution for QA scoring when some customers only provide mono (single-channel) recordings. [Likely]
    - How do diarization errors flow into rubric scores ("agent stated the disclosure"), and how do you guard against them?
    - When would you fall back to an LLM to infer the speaker from content?

G. Knowledge Assist (RAG in live conversations)

58. Design Knowledge Assist: surface answers from a customer's knowledge base to an agent during a live conversation. [Likely]
    - Sketch: KB connectors, ingestion and chunking, embedding and indexing, query formulation from conversation context, retrieval and re-ranking, grounded answer generation, citation UI, feedback.
    - How do you turn messy, implicit conversation context into a retrieval query? (See [18][20].)
    - When should the system answer proactively, and when should it wait for the agent to ask?
    - What latency is acceptable, and which steps would you cache?
59. How would you serve Knowledge Assist with a fine-tuned model at low latency and cost? [Likely]
    - What are the trade-offs of fine-tuning compared with prompting a frontier model for this task? (See [20][21].)
    - How do you evaluate answer quality and groundedness offline?
60. Design the ingestion pipeline that makes enterprise knowledge "AI-ready": duplicate, stale, contradictory and poorly structured KB articles. [Likely]
    - How do you detect conflicting articles and surface them to the content owner? (See [18].)
    - How do you handle PDFs, tables, and screenshots inside articles?
    - How do you re-index incrementally when articles change?
61. How do you detect and handle knowledge gaps: questions customers ask that the KB cannot answer? [Likely]
    - How would you mine gaps from conversations and propose new articles?
62. Design retrieval evaluation for Knowledge Assist when you have no labeled query-document pairs. [Likely]
    - How would you bootstrap labels with LLMs, and how far can you trust them?
    - Would you fine-tune embeddings per domain? Where would you get hard negatives?

63. How do you adapt one retrieved answer to channel constraints: a short spoken answer for voice, a formatted answer for chat? [Likely]
64. Plan an embedding-model upgrade for Knowledge Assist and conversation search that re-embeds billions of vectors across tenants without downtime or quality loss. [Likely]
    - How do you run the old and new indexes side by side, compare retrieval quality, and cut over per tenant?
    - What does the re-embedding job cost, and how do you schedule it around real-time traffic?

H. Multi-tenant LLM serving, models and cost

65. Design multi-tenant LLM serving: one shared base model with per-customer adapters, prompts and policies. [Likely]
    - Sketch: API gateway with tenant auth, request router, adapter registry, GPU serving pool with multi-LoRA batching, autoscaler and per-tenant metering.
    - How do you serve hundreds of LoRA adapters on shared GPUs without per-adapter replicas? (See [22].)
    - How do you prevent cross-tenant data leakage in caches, logs and training data?
    - How do you enforce per-tenant rate limits and noisy-neighbor isolation?
66. Design an LLM serving stack that meets p99 latency targets for real-time features. [Likely]
    - What do continuous batching, KV-cache management, quantization and speculative decoding each trade off?
    - Why is CPU-based autoscaling a poor fit for GPU inference, and what signal would you scale on?
    - How do you keep cold-start times acceptable?
67. Design a model router that chooses between a self-hosted small model and a frontier API per request. [Likely]
    - Which features drive the routing decision (task, customer tier, input length, confidence)?
    - How do you prove the router does not hurt quality?
68. Design the domain adaptation of a base LLM for contact centers, in the spirit of Ocean-1. [Likely]
    - Which data would you use (self-instruct, real transcripts, preference data), and how do you handle customer data rights? (See [17].)
    - How do you show it beats a frontier API on quality, latency and cost for your tasks?
    - How do you keep it current as new open base models come out?
69. Plan the migration of one high-volume feature from a frontier LLM API to a distilled, self-hosted model. [Likely]
    - What quality bar must the new model meet, and how is it measured?
    - What is the break-even volume where self-hosting is cheaper?
70. Design per-customer fine-tuning as a product feature: training, evaluation, deployment and rollback for each tenant. [Likely]
    - How do you schedule training jobs fairly and cap cost?
    - How do you stop one customer's adapter from regressing silently after a base-model upgrade?

71. Design a cost-accounting and budgeting system for LLM usage per customer and per feature. [Likely]
    - How do you attribute GPU cost on shared, batched infrastructure?
    - What would you do when one customer's usage spikes 10x?
72. How would you design for data residency and isolation (EU customers, regulated industries) on a shared LLM platform? [Likely]
    - What changes in serving, storage and training pipelines?
73. Design the prompt and configuration management service for hundreds of customer-specific agents and assist features. [Likely]
    - How do base prompts, vertical templates and per-customer overrides inherit from each other?
    - How do you version, review, diff and roll back a configuration, and tie each production response to the exact version that produced it?
74. Scale up the reported rate-limiter coding question: design per-tenant rate limiting and quota enforcement in front of shared LLM serving and third-party LLM APIs. [Likely] [1]
    - Token-based or request-based limits? How do you protect real-time traffic from a tenant's batch backfill?
    - How do you enforce limits consistently across regions and replicas?
75. Design scheduling that shares a GPU fleet between real-time inference (live assist, voice agents) and batch work (post-call QA, insights backfills, evaluation runs). [Likely]
    - How do you pre-empt batch work when live traffic spikes, without losing progress?
    - How do you plan capacity for diurnal traffic across time zones?
76. Design a distillation pipeline in which a frontier LLM labels data and small, fast models serve real-time classifiers and triggers. [Likely]
    - How do you choose which tasks to distill, and how do you refresh the student when the teacher or the customer's taxonomy changes?
    - How do you catch cases where the student confidently disagrees with the teacher?
77. How would you guarantee valid structured outputs (tool-call arguments, CRM fields, QA scores) from the LLM in your serving stack? [Likely] [28]
    - Constrained decoding against a JSON schema, validate-and-retry, or fine-tuning? What does each cost in latency?
    - How does constrained decoding interact with batching and speculative decoding?

I. Speech and ASR systems

78. Design a streaming ASR service for contact-center audio at thousands of concurrent calls. [Likely]
    - Sketch: audio ingestion (dual-channel), VAD, streaming encoder workers, partial and final hypothesis emitter, and the transcript event bus.
    - How do you balance accuracy and latency (chunk size, lookahead)?
    - How do you pack many streams onto one GPU?
79. Design customer-specific vocabulary adaptation for ASR (product names, account terms). [Likely]

    - Contextual biasing, fine-tuning, or post-correction with an LLM: which and why? (See [27].)
80. Design an ASR quality program across accents, noise and domains. [Likely]
    - How does WER relate to downstream task quality, and which do you optimize?
    - How do you get ground-truth transcripts cheaply and detect a regression for one customer?
81. Design PII redaction for audio and transcripts that runs before anything is stored or used for training. [Likely]
    - How do you redact both the transcript and the matching audio segment?
    - What does a missed credit-card number cost, and how does that set your recall target?

J. Data, feedback loops and platform

82. Design the feedback flywheel: agent accept, edit and ignore signals on suggestions, flowing back into model improvement. [Likely]
    - What biases does implicit feedback introduce (position, presentation, agent skill), and how do you correct for them?
    - How do you respect customer consent and data-usage terms when you train on their data?
83. Design the training-data platform for many enterprise tenants: collection, labeling, consent, lineage and deletion. [Likely]
    - How do you honor a deletion request for data that has already been used in training?
84. Design a labeling program that combines LLM pre-labels with expert human review for new tasks. [Likely]
    - How do you decide which examples go to humans (active learning, disagreement sampling)?
85. Design the experimentation platform for LLM features (prompt variants, model variants) across enterprise customers who may refuse experiments. [Likely]
    - How do you get statistical power when each customer is small?
86. Design an observability and tracing system for LLM pipelines (prompts, retrieved documents, tool calls, outputs) that is safe for PII. [Likely]
    - What do you sample and store, and for how long?
87. Design onboarding for a new enterprise customer, so that their deployment reaches production quality in weeks instead of months. [Likely]
    - What can be automated from their historical conversations (intents, KB, SOPs, QA rubric)?
88. Scale up the reported worker-pool and job-dispatcher coding questions: design the orchestrator that dispatches AI-agent and evaluation jobs across a worker fleet. [Likely] [1][29]
    - Round-robin, least-loaded or priority scheduling? How do you handle stuck or failed jobs, retries and idempotency?
    - How do you give each tenant a fair share of the fleet?
89. Design tool onboarding: turn a customer's API spec (for example OpenAPI) into tools an AI agent can call, with a sandbox and tests. [Likely] [25]
    - How do you write tool descriptions the model uses correctly, and detect tool misuse after launch?

- How do you mock customer systems for simulation without drifting from the real APIs?

K. Cross-cutting deep-dive probes (expect several in any design)

90. What are your functional and non-functional requirements, and which one dominates the design? [Likely]
91. Give a back-of-envelope estimate: concurrent calls, utterances per second, tokens per day, GPUs needed. [Likely]
92. Where is the latency budget spent, and which component would you optimize first? [Likely]
93. What is your offline evaluation, and how do you know it predicts online results? [Likely]
94. Which online metrics would you put on the launch dashboard, and which one is the north star? [Likely]
95. What happens when the LLM provider has an outage or rate-limits you mid-call? [Likely]
96. How does the design change for chat compared with voice, and for 10 customers compared with 1,000? [Likely]
97. Where could the system hallucinate or mislead a human agent, and how do you contain the blast radius? [Likely]
98. How do you handle a customer who demands that their data is never used for training? [Likely]
99. What would make you choose a classic ML model (gradient boosting, small classifier) over an LLM for one component? [Likely]
100. How would you explain a quality regression in this system to an enterprise customer's VP of operations? [Likely]
101. What are the top three ways this system fails in production, and how would you detect each? [Likely]

References

1. Glassdoor, "Cresta (CA) Interview Questions" (Senior Machine Learning Engineer review with systems design round; "How would you deploy a model in production?"), 2025-2026. https://www.glassdoor.co.in/Interview/Cresta-CA-Interview-Questions-E3180909.htm
2. Glassdoor, "Cresta (CA) Interview Questions" (about 25-day process; SWE project presentation), 2025-2026. https://www.glassdoor.ca/Interview/Cresta-CA-Interview-Questions-E3180909.htm
3. Blind, "Cresta interview process", 2024-2025. https://www.teamblind.com/post/cresta-interview-process-jmfnzjoa
4. Blind, "Anyone interview with Cresta lately?", 2025. https://www.teamblind.com/post/anyone-interview-with-cresta-lately-zvj0uhq2
5. PracHub, "Cresta ML System Design Interview Questions" ("Design an Evaluation Platform for AI Agents Across Prompt and Model Changes"), 2026. https://prachub.com/companies/cresta/categories/ml-system-design

6. PracHub, "Cresta Software Engineer Interview Experience: System Design Round: Build an Eval Platform for AI Agents". https://prachub.com/interview-experiences/cresta-software-engineer-interview-experience-system-design-round-build-an-eval-platform-for-ai-agents
7. 1point3acres, "Cresta Machine Learning Engineer Tech Phone Interview", 2025. https://www.1point3acres.com/interview/thread/1140481
8. Cresta on Greenhouse, "Senior Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4249943008
9. Cresta on Greenhouse, "Staff Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4741283008
10. Cresta on Greenhouse, "Senior Machine Learning Engineer, Voice Experience". https://job-boards.greenhouse.io/cresta/jobs/5155675008
11. Built In Vancouver, "Senior Machine Learning Engineer, Cresta" (Agentic Assist track). https://builtinvancouver.org/job/senior-machine-learning-engineer/8619914
12. Cresta Blog, "How Cresta Grades Its Agents: Evaluators in an LLM World". https://cresta.com/blog/how-cresta-grades-its-agents-evaluators-in-an-llm-world
13. Cresta Blog, "Why AI Agent Evaluations Fail, and How the Swiss-Cheese Model Prevails". https://cresta.com/blog/why-ai-agent-evaluations-fail----and-how-the-swiss-cheese-model-prevails
14. Cresta Blog, "Engineering for Real-Time Voice Agent Latency". https://cresta.com/blog/engineering-for-real-time-voice-agent-latency
15. Cresta Blog, "Understanding Cresta's Voice Platform: ML Services, Inference Graphs and Real-Time Intelligence", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-ml-services-inference-graphs-and-real-time-intelligence
16. Cresta Blog, "Understanding Cresta's Voice Platform: The Voice Stack", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-the-voice-stack
17. Cresta Blog, "Introducing Ocean-1: World's First Contact Center Foundation Model", 2023. https://cresta.com/blog/introducing-ocean-1-worlds-first-contact-center-foundation-model
18. Cresta Blog, "AI-Ready Knowledge for Contact Centers: Closing the Gap Between KBs and AI", 2025. https://cresta.com/blog/ai-ready-knowledge-for-contact-centers-closing-the-gap-between-kbs-and-ai
19. Cresta Blog, "Introducing the Next Generation of Cresta Insights", 2026. https://cresta.com/blog/introducing-the-next-generation-of-cresta-insights-a-complete-system-for-enterprises-to-understand-their-cx
20. Cresta Blog, "How Fine-Tuned LLMs Power Knowledge Assist, Summarization and Chat Suggestions", 2024. https://cresta.com/blog/how-fine-tuned-llms-power-knowledge-assist-summarization-and-chat-suggestions
21. Fireworks AI Blog, "Story: Cresta Knowledge Assist", 2024. https://www.fireworks.ai/blog/story-cresta-knowledge-assist
22. Fireworks AI Blog, "Multi-LoRA", 2024. https://fireworks.ai/blog/multi-lora
23. Cresta Blog, "Grounding Reality: How Cresta Tackles LLM Hallucinations in Enterprise AI". https://cresta.com/blog/grounding-reality---how-cresta-tackles-llm-hallucinations-in-enterprise-ai
24. Cresta Blog, "Cresta Launches Automated AI Agent Testing: Confidence in Every Conversation". https://cresta.com/blog/cresta-launches-automated-ai-agent-testing-confidence-in-every-conversation

25. Cresta Blog, "Cresta Conductor: The Agent for AI Agent Development". https://cresta.com/blog/cresta-conductor-the-agent-for-ai-agent-development
26. Glassdoor, "Cresta (CA) Interview Experience and Questions" (FDE technical screen: RAG and hallucination), 2026. https://www.glassdoor.com/Interview/Cresta-CA-Interview-Questions-E3180909.htm
27. Cresta on Greenhouse, "Senior Machine Learning Engineer (ASR)". https://job-boards.greenhouse.io/cresta/jobs/5101011008
28. PracHub, "Cresta Interview Questions" (company page; "Design sequence decoding with greedy and beam search"), 2025. https://prachub.com/companies/cresta
29. 1point3acres, "Cresta Fulltime Software Engineer Tech Phone Screen: Job Dispatcher-Executor System Implementation" (title seen in a listing; full text not read), 2026. https://www.1point3acres.com/interview/thread/1172433

Changed in review: The Blind thread "Cresta senior backend SWE interview" was removed. The only excerpt of it in this guide supports an online assessment, not the one-hour system design round it was cited for here, and ref 26 now points to the Glassdoor FDE report instead. References 1 (now also cited for the reported rate-limiter and worker-pool coding questions), 7, 28 and 29 match excerpts that chapters 01-03 already reviewed. Three generic probes were removed as filler: "what would you build in week one", "what would you hand to a vendor" and "twice or half the budget". Note on [Reported] tags: only questions 1-4 carry the tag. Questions 1 and 2 are reported for this round. Question 3 was reported for an FDE screen, and question 4 for an MLE phone screen. No question was retagged, because every [Reported] attribution matches an excerpt in the sibling chapters. Page contents could not be re-read live.

05. Project Deep Dive and Hiring Manager Round: Defending the Architecture of ML Systems You Shipped

> **Scope.** You asked for ML system design content, so this chapter treats the deep dive as an ML system design round about your own past work. Expect to draw your shipped system end to end, covering data, training, serving, evaluation and monitoring, and then defend each box and arrow. Hiring-manager questions are kept only where they probe how you designed, scoped or owned ML systems.

Format notes for this round

- **What candidates report.**
  - Cresta software-engineering loops have included a **project presentation** round [2].
  - A Senior FDE loop included a separate **leadership round** [2].
  - A Senior/Staff backend loop is described as one hour of coding, one hour of system design and a **hiring-manager chat** [4].
  - A mid-level SWE loop reportedly had **three behavioral rounds** alongside the coding rounds [3].
  - The reported Senior MLE loop was a recruiter call, a LeetCode-medium round, a "vibe coding" round with an LLM tool, and a system design round ("How would you deploy a model in production?") [1]. A recruiter email to another candidate said interviews would cover "key concepts and fundamentals around LLMs, GenAI, RAG, and AI Agents" [5].
  - An MLE (AI Agent) phone screen covered greedy and beam-search decoding [25]. PracHub lists "Design an Evaluation Platform for AI Agents Across Prompt and Model Changes" as a Cresta ML system design question [22][26]. An FDE technical screen covered RAG and hallucination [31].
- **What is not confirmed.** No source reports a standalone project deep dive for the Senior MLE role. Assume it may be folded into the hiring-manager chat or the system design round, so prepare one project you can draw as an architecture diagram in 5 minutes.
- **Duration and interviewer (inferred).** Expect 45-60 minutes with an engineering manager or a senior or staff MLE from the target track: Agentic Assist, Agent and System Quality, Voice/ASR, Chat Agent, or Insights [7][8][9][10][11][30].
- **Senior bar (inferred from postings).**
  - You owned an ML system end to end, from problem framing to production.
  - You can diagnose LLM failure modes: hallucination, retrieval errors, tool misuse, context drift, prompt brittleness, and multi-step reasoning breakdowns [7][11].

    - You evaluate with offline benchmarks, online experiments and LLM-as-a-judge [7].
    - You make deliberate latency and cost trade-offs [17][18].
    - You can turn vague customer asks into measurable ML requirements. Cresta runs forward-deployed engineering loops, so expect MLEs to work alongside customer-facing teams [6].
- **Culture signals.** A third-party summary cites these principles: "Mission is the boss", "Do too much", "Have the courage to be disliked", and "patient in strategy, impatient in execution" [21]. Hiring-manager questions below are framed around them.
- **Research caveat.** During this review, every cited page failed to load (DNS errors for Glassdoor, Blind, Greenhouse, PracHub, 1point3acres, cresta.com and fireworks.ai), and the web-search budget for the run was used up. So no page was re-read and no new candidate reports could be found. Each source is kept only where search-result excerpts gathered for this bank (chapters 01-07) support it. [Reported] marks only questions a source attributes to Cresta. Everything else is [Likely], inferred from the role, Cresta's products and its engineering blog. All added questions are [Likely].

A. Opening: draw the system

1. Present a significant project you built end to end. [Reported format for SWE: project presentation round] [2]
    - What did you personally own, and what did others build?
    - What was the baseline, and how did you prove the improvement?
2. Draw the architecture of the most complex ML system you shipped, from raw data to the user-facing output. [Likely]
    - Which box on this diagram was hardest to build, and which failed most in production?
    - Which component would you delete if you rebuilt it today?
3. How would you deploy the model from your project in production? Walk through the path from checkpoint to live traffic. [Reported, as a system design prompt for Senior MLE] [1]
    - How did you roll out (shadow, canary, A/B), and what triggered a rollback?
    - How did you version models, prompts and configs together?
4. Walk through one request through your system with timings at each hop. [Likely]
    - Where did p99 latency come from, and what did you do about it?
5. What was the one-sentence problem statement, and who was the user? [Likely]
    - How did the business metric map to your model metric?
6. Was this an LLM system, a classical ML system, or a hybrid? Why that split? [Likely]
    - Which parts did you leave as rules or heuristics on purpose?
7. Redraw the diagram as it looked at v1, then as it looks now. What forced each change? [Likely]
8. Which non-functional requirement (latency, availability, cost, data residency, compliance) shaped the architecture most, and which box exists only because of it? [Likely] [7]

B. Data and labeling in your project

9. Where did your training and evaluation data come from, and how was it labeled? [Likely]
    - How did you measure label quality and annotator agreement?
10. How did you handle PII or sensitive data in the pipeline? [Likely] [7]
    - Would your approach work for contact-center transcripts with card numbers and health data?
11. Could a customer opt out of training or ask for its data to be deleted? How would you remove that data's influence from a model already in production? [Likely] [23]
12. How did you detect data drift or a shift in the input distribution after launch? [Likely]
13. Did you use synthetic data or LLM-generated labels? How did you validate them? [Likely]
    - What biases did the generator add?
14. How did you split train, validation and test data to avoid leakage (for example by customer, by time, by conversation)? [Likely]
15. How did you version datasets so that any production model could be traced to the exact data and code that trained it? [Likely]
16. If your inputs came from ASR or OCR, how did upstream errors affect labels and model quality, and what did you do about it? [Likely] [9]
17. If you had to build the same dataset for a new enterprise customer in two weeks, what would you do? [Likely] [12]

C. Modeling decisions

18. Why did you choose this model family or base model over the alternatives? [Likely]
    - What did you try that did not work?
19. Did you fine-tune, prompt, or use retrieval? What evidence drove the choice? [Likely] [12][13]
    - What would make you switch to another approach?
20. If you fine-tuned, walk through the setup: data size, method (full, LoRA, adapters), hyperparameters, and compute. [Likely] [13]
    - How did you stop catastrophic forgetting or regressions on general ability?
21. Cresta reports that fine-tuned smaller models matched GPT-4-level quality on Knowledge Assist at lower cost and latency. Did you make a similar trade in your project? Show the numbers. [Likely] [12][13]
22. How did you handle per-customer customization: one model, per-customer adapters, or per-customer prompts? [Likely] [13][27]
    - How would your design scale to thousands of customers?
23. Describe a decision where you chose a simpler model over a more sophisticated one. [Likely]
24. How did you choose decoding settings (greedy, beam, sampling, temperature) for your generation task, and what did each do to quality, diversity and latency? [Likely; decoding was tested in an MLE phone screen] [25]
25. Did you use structured outputs or constrained decoding for extraction or tool calls? How did you handle schema violations at runtime? [Likely]
26. How did you handle inputs longer than the context window, such as hour-long calls or long chat histories? [Likely]

27. When the base model was upgraded or the API provider changed versions, what broke in your prompts, and how did you regression-test the upgrade? [Likely]
28. If you used RAG, draw the retrieval pipeline: chunking, embedding, index, reranking and grounding. [Likely] [7][31]
    - How did you measure retrieval quality apart from generation quality?
    - What did you do when the knowledge base was stale or contradictory?
29. If your system was an agent with tools, how did you define the tool schemas and control the loop? [Likely] [11]
    - How did you stop runaway loops or wrong tool calls?

D. Evaluation and quality

30. Describe the evaluation you built. What did your metrics miss? [Likely]
31. Tell me about a model that worked offline but failed online. How did you find the gap, and how did you change the offline evaluation afterwards? [Likely]
32. Did you use LLM-as-a-judge? How did you calibrate it against humans? [Likely] [15]
    - How did you catch judge bias such as position, length or self-preference?
33. Who wrote the rubrics for human raters, and how did you measure and improve inter-rater agreement? [Likely] [15]
34. How did you gate releases? Which tests had to pass before a prompt or model change shipped? [Likely] [20][22]
    - How fast did that gate run, and who could override it?
35. Redraw your project's evaluation stack as a platform for AI agents where prompt and model changes ship every week: datasets, judges, statistical gates and reporting. [Reported, as a Cresta ML system design prompt] [22][26]
    - How would it run across many customers whose agents use different prompts?
36. How large were your eval sets, and how did you decide that a metric difference was statistically significant rather than noise? [Likely]
37. How did you keep eval sets from going stale, or from leaking into training data and prompts? [Likely]
38. Cresta describes layering evaluations like a "Swiss-cheese model". How many independent layers of checks did your project have? [Likely] [16]
    - Which layer caught the most real issues?
39. How did you measure and reduce hallucinations? [Likely] [19][31]
    - What grounding or citation checks did you add?
40. How did you evaluate multi-turn conversations, not just single responses? [Likely] [15][20]
    - Did you simulate users? How realistic were the simulations?
41. What was your error analysis process? Show one taxonomy of failures you built. [Likely]
42. How did you test rare but high-stakes behaviors, such as compliance or legal disclosures? [Likely]

E. Serving, latency and cost

43. Draw your serving stack: hardware, inference server, batching, caching and autoscaling. [Likely] [13]
44. What was your latency budget, and how was it split across components? [Likely] [17]
    - For a real-time voice agent (ASR, turn detection, LLM, TTS), where would your system's budget go? [Likely] [17][18]
45. What did one request or conversation cost, and how did you bring it down? [Likely]
46. Did you use streaming, speculative decoding, quantization or distillation? What did each buy you? [Likely]
    - What quality did you lose, and how did you measure it?
47. How did your system behave under a traffic spike or a provider outage? [Likely]
    - What was your fallback path?
48. How did you handle provider rate limits and model deprecations when you depended on third-party APIs? [Likely]
49. At what request volume did self-hosting an open model beat a hosted API on cost for your project? How did you calculate the break-even? [Likely] [13][23]
50. How did you serve multiple models or adapters on shared hardware? [Likely] [13][27]
    - With thousands of per-customer LoRA adapters, how would you handle adapter loading, cold starts and GPU memory?
51. If your inference ran as a graph of ML services, how did you order and parallelize the nodes? [Likely] [18]

F. Production, monitoring and incidents

52. What did you monitor after launch: system metrics, model metrics and business metrics? [Likely]
    - Which alert fired most often, and was it useful?
53. Describe the hardest production incident on this system, and what the postmortem changed. [Likely]
54. How did you detect silent model degradation without labels? [Likely]
55. How did you collect feedback from users or agents and feed it back into training? [Likely] [12]
    - How did you keep the feedback loop from reinforcing bias?
56. How did you run A/B tests in a B2B setting with a small number of large customers? [Likely]
    - How did you handle customers who refused experiments?
57. How did you roll a change out customer by customer, and how did you roll back one customer without affecting the rest? [Likely]
58. What audit trail did you keep so you could explain, weeks later, why the system produced a specific output for a specific customer? [Likely]
59. What would break first if traffic grew 10x? [Likely]

G. Impact and ownership

60. What was the measurable impact: revenue, handle time, conversion, CSAT, or cost? How confident are you in that number? [Likely]
    - How did you separate your system's effect from seasonality or other launches?

61. What did you personally decide, and where did you defer to others? [Likely]
62. What would you do differently if you started this project again? [Likely]
63. What technical debt did you knowingly take on, and how did you pay it back? [Likely]
64. How did you scope the project to deliver value in weeks rather than quarters? [Likely]
65. If we gave you this problem in the contact-center domain, what would change in your diagram, and which parts would you reuse as they are? [Likely]

H. Transfer to Cresta's systems (whiteboard follow-ups)

66. Map your project onto real-time agent assist: what would the live-transcript-to-suggestion path look like? [Likely] [12][18]
    - How would your model consume partial, changing ASR transcripts rather than final text? [Likely] [29]
67. How would you adapt your evaluation setup to grade AI agents on thousands of customer conversations? [Likely] [15][22]
68. How would your data pipeline change for post-call analytics and topic discovery at enterprise scale? [Likely] [24]
69. Cresta built Ocean-1 as a contact-center foundation model. If you were extending a domain model like that, how would your project experience apply? [Likely] [14]
70. How would you design guardrails for an autonomous AI agent talking to end customers? [Likely] [19]
71. When should an AI agent hand off to a human, and what context should it pass along? [Likely]
72. Using your RAG experience, how would you make a customer's messy knowledge base "AI-ready": deduplication, conflict resolution and freshness? [Likely] [28]
73. Cresta Conductor builds AI agents from natural-language instructions. How would you test and version an agent built that way before it reaches customers? [Likely] [20] [32]
74. For a chat agent track: how would you evaluate an agent that completes multi-step tasks over a long chat, such as a refund that needs identity checks? [Likely] [30]
75. For a voice experience track: how would you reduce time to first audio in your architecture? [Likely] [10][17]
76. For an ASR track: how would you adapt ASR to customer-specific vocabulary such as product names and account terms? [Likely] [9]
77. Which metrics would you put on a dashboard for a customer's AI-agent deployment? [Likely]

I. Hiring manager: ML ownership and judgment

78. Walk through your background, focusing on ML systems you shipped. Why Cresta and contact-center AI? [Likely]
79. Which track appeals to you: Agentic Assist, Agent and System Quality, Voice/ASR, Chat Agent, or Insights? [Likely] [7][8][30]
    - Where would you have the most impact in your first six months?
80. Tell me about a time you "did too much" to get an ML system shipped. [Likely] [21]

81. Tell me about pushing back on a PM, customer or leader with data ("courage to be disliked"). [Likely] [21]
82. Tell me about a time you were patient on strategy but impatient on execution. [Likely] [21]
83. Describe working directly with customers or forward-deployed teams to define what an ML feature should do. [Likely] [6]
    - How did you turn a vague ask into measurable requirements?
84. A customer gives you an ambiguous request and a one-week deadline. How do you define success for the model yourself? [Likely]
85. Describe a disagreement over quality versus shipping speed for a model launch. [Likely]
86. How do you explain a model regression to an enterprise customer? [Likely]
87. Tell me about a technical bet you got wrong. How did you notice, and how fast did you change course? [Likely]
88. How do you decide which new LLM research or model release is worth adopting? [Likely]
89. How do you split your time between research exploration and shipping in a startup? [Likely]
90. When would you build a capability in-house rather than use a third-party model API? [Likely] [23]
91. How do you raise the ML bar of a team: design reviews, eval standards, or mentoring? [Likely]
92. Tell me about mentoring an engineer through their first production ML launch. [Likely]
93. Tell me about changing another team's technical direction without authority over them. [Likely]
94. If evaluation for a key model turned out to be weak, what would you do in your first 90 days? [Likely]
95. Tell me about a project you killed or pivoted. How did you decide? [Likely]
96. How do you prioritize between customer escalations and roadmap ML work? [Likely]
97. What separates a senior MLE from a staff MLE, in your view? Where are you? [Likely] [8]
98. How do you use AI coding tools in your own ML engineering work? [Likely] [1]
    - How do you check generated code before it reaches production?

J. Questions to ask the hiring manager

99. Which team would I join, and what ML systems does it own end to end? [Likely]
100. How does the team gate prompt and model changes before they reach customers? [Likely]
101. What is the split between self-hosted models and third-party APIs, and where is it heading? [Likely]
102. How much direct customer contact do MLEs have? [Likely]
103. What is the on-call expectation for MLEs, and who owns model incidents? [Likely]
104. What does success look like for this role at 6 and 12 months? [Likely]

References

1. Glassdoor, "Cresta (CA) Interview Questions" (includes Senior Machine Learning Engineer loop with vibe-coding and system design rounds; "How would you deploy a model in production?"), 2025-2026. https://www.glassdoor.co.in/Interview/Cresta-CA-Interview-Questions-E3180909.htm
2. Glassdoor, "Cresta (CA) Interview Questions" (SWE project presentation; Senior FDE leadership round), 2025-2026. https://www.glassdoor.ca/Interview/Cresta-CA-Interview-Questions-E3180909.htm
3. Blind, "Cresta interview process", 2024-2025. https://www.teamblind.com/post/cresta-interview-process-jmfnzjoa
4. Blind, "Cresta senior backend SWE interview", 2025. https://www.teamblind.com/post/cresta-senior-backend-swe-interview-6v1tmjld
5. Blind, "Anyone interview with Cresta lately?" (recruiter email on LLMs, GenAI, RAG and AI Agents), 2025. https://www.teamblind.com/post/anyone-interview-with-cresta-lately-zvj0uhq2
6. Blind, "Interview for forward deployment engineer at Cresta", 2025. https://www.teamblind.com/post/interview-for-forward-deployment-engineer-at-cresta-ngswbj44
7. Cresta on Greenhouse, "Senior Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4249943008
8. Cresta on Greenhouse, "Staff Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4741283008
9. Cresta on Greenhouse, "Senior Machine Learning Engineer (ASR)". https://job-boards.greenhouse.io/cresta/jobs/5101011008
10. Cresta on Greenhouse, "Senior Machine Learning Engineer, Voice Experience". https://job-boards.greenhouse.io/cresta/jobs/5155675008
11. Built In Vancouver, "Senior Machine Learning Engineer, Cresta" (Agentic Assist track). https://builtinvancouver.org/job/senior-machine-learning-engineer/8619914
12. Cresta Blog, "How Fine-Tuned LLMs Power Knowledge Assist, Summarization and Chat Suggestions", 2024. https://cresta.com/blog/how-fine-tuned-llms-power-knowledge-assist-summarization-and-chat-suggestions
13. Fireworks AI Blog, "Story: Cresta Knowledge Assist", 2024. https://www.fireworks.ai/blog/story-cresta-knowledge-assist
14. Cresta Blog, "Introducing Ocean-1: First Contact Center Foundation Model", 2023. https://cresta.com/blog/introducing-ocean-1-worlds-first-contact-center-foundation-model
15. Cresta Blog, "How Cresta Grades Its Agents: Evaluators in an LLM World". https://cresta.com/blog/how-cresta-grades-its-agents-evaluators-in-an-llm-world
16. Cresta Blog, "Why AI Agent Evaluations Fail, and How the Swiss-Cheese Model Prevails". https://cresta.com/blog/why-ai-agent-evaluations-fail----and-how-the-swiss-cheese-model-prevails
17. Cresta Blog, "Engineering for Real-Time Voice Agent Latency". https://cresta.com/blog/engineering-for-real-time-voice-agent-latency
18. Cresta Blog, "Understanding Cresta's Voice Platform: ML Services, Inference Graphs, and Real-Time Intelligence", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-ml-services-inference-graphs-and-real-time-intelligence
19. Cresta Blog, "Grounding Reality: How Cresta Tackles LLM Hallucinations in Enterprise AI". https://cresta.com/blog/grounding-reality---how-cresta-tackles-llm-hall

ucinations-in-enterprise-ai
20. Cresta Blog, "Cresta Launches Automated AI Agent Testing: Confidence in Every Conversation". https://cresta.com/blog/cresta-launches-automated-ai-agent-testing-confidence-in-every-conversation
21. JobsByCulture, "Cresta: Frontend Engineer" (culture and operating-principles summary; third-party). https://jobsbyculture.com/jobs-at/cresta/frontend-engineer
22. PracHub, "Cresta ML System Design Interview Questions" ("Design an Evaluation Platform for AI Agents Across Prompt and Model Changes"), 2026. https://prachub.com/companies/cresta/categories/ml-system-design
23. CX Today, "Cresta CEO Ping Wu: Many Contact Centers Misunderstand the Value of LLMs". https://www.cxtoday.com/contact-center/cresta-ceo-ping-wu-many-contact-centers-misunderstand-the-value-of-llms/
24. Cresta Blog, "Introducing the Next Generation of Cresta Insights", 2026. https://cresta.com/blog/introducing-the-next-generation-of-cresta-insights-a-complete-system-for-enterprises-to-understand-their-cx
25. 1point3acres, "Cresta Machine Learning Engineer Tech Phone Interview Experience" (MLE, AI Agent direction; greedy and beam-search decoding), 2025. https://www.1point3acres.com/interview/thread/1140481
26. PracHub, "Cresta Software Engineer Interview Experience: System Design Round: Build an Eval Platform for AI Agents". https://prachub.com/interview-experiences/cresta-software-engineer-interview-experience-system-design-round-build-an-eval-platform-for-ai-agents
27. Fireworks AI Blog, "Multi-LoRA", 2024. https://fireworks.ai/blog/multi-lora
28. Cresta Blog, "AI-Ready Knowledge for Contact Centers: Closing the Gap Between KBs and AI", 2025. https://cresta.com/blog/ai-ready-knowledge-for-contact-centers-closing-the-gap-between-kbs-and-ai
29. Cresta Blog, "Understanding Cresta's Voice Platform: The Voice Stack", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-the-voice-stack
30. Cresta on Lever, "Senior Machine Learning Engineer (Chat Agent)". https://jobs.lever.co/cresta/172fe276-9415-4bae-9b0b-af72cf7cf014
31. Glassdoor, "Cresta (CA) Interview Experience and Questions" (FDE technical screen: RAG and hallucination), 2026. https://www.glassdoor.com/Interview/Cresta-CA-Interview-Questions-E3180909.htm
32. Cresta Blog, "Cresta Conductor: The Agent for AI Agent Development". https://cresta.com/blog/cresta-conductor-the-agent-for-ai-agent-development

Removed or changed in review: the Blind "Cresta AI SWE AMA" thread (chapter 03's review found it no longer supported any question; the customer-facing claim now rests on the FDE thread). The Scoutify guide (a generic template, not candidate reports). Two duplicates were merged: the "offline vs online correlation" question went into "worked offline but failed online" (Q31), and "which part could be reused at Cresta" went into the contact-center transfer question (Q65). Q1 was retagged from "[Reported for SWE]" to "[Reported format for SWE]", because the source reports a project presentation round, not this exact wording. Eight sources already cited elsewhere in this bank (chapters 01-04 and 07) were added as refs 25-32. Their pages were not re-opened in this review.

07. Cresta Product and Research Context: ML System Design Questions Built on Agent Assist, AI Agent, Conversation Intelligence and Ocean Models

> **Scope.** You asked for ML system design content, so this chapter presents Cresta's products and research as design prompts an interviewer could build from them. Each design question has a **Sketch** follow-up that names the diagram you would draw: the boxes and arrows, not the answer. Questions about the product or the company appear only where they set up a design discussion.
>
> **Tagging system.**
>
> - **[Reported] [n]**: a candidate report or question bank attributes this question, or a close variant, to Cresta. [n] points to the References list.
> - **[Likely]**: inferred from Cresta's products, engineering blog, partner write-ups and ML job postings. No candidate has reported it. Where a [Likely] question builds on a specific Cresta post, that post is cited as context only; the citation does not mean anyone reported the question.
>
> **Research limits.** No cited page could be opened while this chapter was researched or reviewed. In the review, fetches of 17 cited URLs (cresta.com, fireworks.ai, prachub.com, Glassdoor, 1point3acres, Greenhouse, Built In, CX Today, Scoutify) were all refused by the egress proxy or failed DNS lookup, and the shared web-search budget was already used up, so no new candidate reports could be searched for. Every reference is therefore kept only where search-result excerpts gathered for this guide (chapters 01-07) support it, and nothing here comes from a full read of a page. No candidate has reported a question about Cresta's products as such. The Cresta ML system design prompts on record are the agent evaluation platform and "How would you deploy a model in production?" [1][5][6]. Four prompts from adjacent Cresta rounds (decoding in an MLE AI Agent phone screen, RAG and hallucination in an FDE screen, and the recruiter's LLM/RAG/agents prep note) are added as [Reported] variants, and each tag names its round. Facts about the founders are general background with no source cited here; check them before quoting them in an interview.

Format notes

- **Where this material appears.** No candidate report describes a separate "Cresta context" round. Product knowledge comes up inside the ML system design round, the

hiring-manager conversation and the project deep dive [1][2]. For senior MLEs, Glassdoor lists a recruiter call, a LeetCode-medium round, an AI-assisted ("vibe coding") round and a system design round [1].

- **Interviewer.** Most likely a senior or staff MLE, or an engineering manager, from the track you are matched to: Agentic Assist, Agent and System Quality, Voice/ASR or Insights. This is inferred from the job postings [9][10][12][13].
- **Duration.** About 60 minutes for system design, based on reports from comparable Cresta engineering loops [2]. Hiring-manager chats run 30-60 minutes [2].
- **Recruiter prep signal.** One recruiter email told the candidate to expect "key concepts and fundamentals around LLMs, GenAI, RAG, and AI Agents" [2][3].
- **What the senior bar assesses here.** First, whether you can turn a Cresta product into a measurable ML problem with a latency budget, an evaluation plan and a rollout plan. Second, whether your designs reflect real constraints: streaming voice, many enterprise tenants, non-deterministic LLM output, and customers who must be able to trust the system. The postings name hallucinations, retrieval errors, tool misuse, context drift and prompt brittleness as failure modes to diagnose, and accuracy, faithfulness, task completion, latency, cost and robustness as metrics [9][11][12][13].

Product map to anchor your designs

Product area	What it does (as Cresta and partners describe it)	Design themes it implies
Agent Assist / Knowledge Assist	Real-time guidance and knowledge answers for human agents during live calls and chats [15][16][18]	Streaming ASR, retrieval under latency, when to suggest something, measuring impact
AI Agent	Autonomous voice and chat agents that handle customer conversations, with testing and an agent-building assistant (Conductor) [14][19][20][24][25]	Tool use, guardrails, escalation, simulation-based testing, voice latency
Conversation Intelligence / Insights	Topic discovery, outcome inference, an AI analyst and QA over all conversations [21]	Batch LLM extraction, clustering, judges, causal pitfalls
Voice platform	Voice stack and inference graphs of ML services per conversation [17][18]	Model DAGs, per-customer policies, partial transcripts
Ocean models and fine-tuned LLMs	Ocean-1 contact-center foundation model; fine-tuned models matching GPT-4 on specific tasks; multi-LoRA serving at Fireworks [7][15][16][22]	Domain adaptation, distillation, multi-tenant adapter serving
Quality and evaluation	Evaluators, the Swiss-cheese reliability model, automated agent testing, grounding against hallucination [14][23][24][26]	LLM-as-a-judge, regression gating, groundedness

Questions

A. Agent Assist (real-time guidance for human agents)

1. Design Cresta's real-time Agent Assist. It listens to a live call and shows the human agent next-best-action hints, behavior reminders and knowledge cards. [Likely]
    - Sketch: telephony/CCaaS audio feed, streaming ASR (partial and final transcripts), conversation state store, trigger/moment detectors, suggestion generator, ranking and filtering, push channel to the agent's desktop, and a logging path for feedback.

    - What is the latency budget from the customer's last word to a card on screen, and how do you split it across ASR, detection, retrieval and generation?
    - Which components run on partial transcripts and which wait for final ones? (Cresta's voice-stack post describes pushing both [18].)
    - How do you decide when not to show a suggestion?
2. Cresta Agent Assist uses "moment" or hint triggers such as "customer mentions cancellation". How would you build a trigger system that a customer's admin can configure without ML expertise? [Likely]
    - Sketch: admin UI for trigger definitions, compiler that turns definitions into classifier or LLM prompts, real-time evaluator, per-tenant config store and alerting.
    - When do you use a trained classifier, a few-shot LLM or a keyword rule?
    - How do you estimate a new trigger's precision before it goes live?
3. How would you measure whether Agent Assist improves agent outcomes such as handle time, conversion or CSAT, rather than just correlating with good agents? [Likely]
    - Sketch: experiment assignment (agent-level or team-level), exposure logging, outcome joins, and the analysis pipeline.
    - Why randomize at agent or team level rather than per conversation?
    - How do you handle spillover when agents learn from suggestions and change their behavior?
4. Design the feedback loop that uses agents' accept, edit and ignore actions on suggestions to improve the models. [Likely]
    - Sketch: UI event capture, sessionization, label derivation, bias correction, training-data store and retraining trigger.
    - What does an ignored suggestion actually tell you?
    - How do you correct for position bias and for agents who click everything?
5. Design chat reply suggestions (smart compose) for chat agents, served from a fine-tuned model. [Likely] [15]
    - Sketch: chat event stream, context builder, candidate generator, safety filter, ranker, UI, and a feedback log.
    - How do you keep suggestions consistent with the brand's voice for each customer?
    - What offline metric best predicts how often agents accept a suggestion?
6. Design automatic after-call summarization that fills CRM fields when the call ends. [Likely] [15]
    - Sketch: final transcript, PII redaction, summarizer, field extractor with schema validation, CRM connector and a human edit loop.
    - How do you evaluate summaries against agents' manual notes?
    - What happens when the model invents a field value?
7. Design a real-time supervisor dashboard that flags live calls going badly (escalation risk, compliance misses) so a supervisor can step in. [Likely]
    - Sketch: per-call scoring stream, aggregation by team, ranking of calls needing attention, alerting with suppression.
    - How do you calibrate alert thresholds so supervisors are not flooded?
8. Partial ASR transcripts get revised as the speaker continues. Design the real-time detection layer so a hint or behavior check does not fire, retract and fire again as the transcript changes. [Likely] [18]

- Sketch: partial-transcript stream with revision IDs, stability window or debounce, detector, fire-once state per moment, and a correction path when a final transcript contradicts a hint already shown.
- How do you trade the extra latency of waiting for stable text against the cost of a wrong hint?

B. Knowledge Assist (RAG in live conversations)

9. Design Knowledge Assist, which answers an agent's question, or one detected in the conversation, from the customer's knowledge base during a live call. [Likely] [15][16] [20]
    - Sketch: KB ingestion (connectors, chunking, cleaning), embedding index per tenant, query reformulation from conversation context, retriever and reranker, answer generator with citations, and the channel formatter.
    - How do you turn a messy spoken exchange into a good retrieval query?
    - How do you keep one tenant's documents isolated from another's?
10. Cresta has written about the gap between existing knowledge bases and AI-ready knowledge. How would you build a pipeline that finds and fixes KB gaps (missing, stale or contradictory articles)? [Likely] [20]
    - Sketch: unanswered-question mining from conversations, clustering, coverage check against the KB, LLM-drafted articles, human approval and re-indexing.
    - How do you detect two articles that contradict each other?
11. Fireworks reports that Cresta moved Knowledge Assist to fine-tuned models that matched GPT-4 quality at lower cost and latency. How would you plan and run that migration? [Likely] [15][16]
    - Sketch: production traffic sampling, teacher labeling, student fine-tuning, offline evaluation against the teacher, shadow deployment, canary and cutover.
    - What quality bar must the student meet, and on which slices?
    - How do you keep the system working if the fine-tuned model fails?
12. How would you evaluate retrieval for Knowledge Assist when you have no labeled query-document pairs? [Likely]
    - How would you use LLM-generated queries, agent click signals and expert spot checks?
13. How would you make Knowledge Assist's answers grounded and stop it from answering when the KB has no answer? [Likely] [26]
    - Sketch: retrieval confidence, answerability classifier, generation constrained to citations, post-hoc groundedness check, and an "I don't know" path.
    - How do you measure the false-refusal rate as well as the hallucination rate?
14. What is RAG, and how do you prevent hallucination? Then extend it: draw RAG over a customer's knowledge base for Knowledge Assist and mark every point where a hallucination can be caught. [Reported] [27] (reported as a conceptual question in a Forward Deployed Engineer technical screen; the design extension is [Likely])
15. KB articles change during the business day (a new promotion, a policy update). Design ingestion so Knowledge Assist never serves a superseded answer. [Likely] [20]
    - Sketch: KB change feed or crawler, document versioning, incremental chunk and embed jobs, index alias swap, cache invalidation, and the answer log recording which article version was used.

    - How do you find answers already shown to agents that relied on an article that has since been retracted?
16. Design retrieval over past resolved conversations ("how did other agents solve this?") as a second knowledge source next to the KB. [Likely]
    - Sketch: resolved-case mining, PII scrubbing, case summarization, case index per tenant, retriever, and a ranking step that blends cases with KB articles.
    - How do you stop a wrong workaround that one agent used from spreading to every agent?

C. AI Agent (autonomous voice and chat agents)

17. Design Cresta's AI Agent: an autonomous voice agent that handles a full customer call for an enterprise, including authentication, tool calls (order lookup, refund) and handoff to a human. [Likely] [19][25]
    - Sketch: telephony, streaming ASR, turn-taking/endpointing, dialogue manager or LLM agent loop, tool gateway with policy checks, TTS, human handoff service, and the trace store.
    - Where are business rules enforced: in the prompt, in the tool gateway or in a separate checker?
    - What context goes to the human agent at handoff?
18. Cresta's latency post discusses engineering a voice agent for low end-to-end latency. Design the pipeline and its latency budget. [Likely] [19]
    - Sketch: audio in, voice activity detection, streaming ASR, endpointing, LLM (time to first token), streaming TTS (time to first byte), audio out, with a timing annotation on each arrow.
    - Which stages can overlap, for example speculative generation before the user finishes speaking?
    - How do you handle barge-in when the customer interrupts?
    - Which network choices matter (connection reuse, region placement, streaming APIs)?
19. Cresta Conductor is described as an agent that helps build AI agents. Design a system in which a business user describes a flow in natural language and receives a working, tested agent configuration. [Likely] [20][25]
    - Sketch: natural-language spec, plan/flow generator, tool and API binding, generated test scenarios, simulation runner, review UI and deploy.
    - How do you stop a generated agent from calling tools it should not have?
    - What does the human review step check?
20. Design the guardrail layer for a customer-facing AI agent: prompt injection, off-topic requests, unauthorized commitments and PII leakage. [Likely]
    - Sketch: input classifier, policy-aware tool gateway, output checker, escalation trigger and audit log.
    - How much latency can the guardrails add on a voice call?
21. How would you decide, per customer and per intent, which conversations the AI Agent should handle and which should go straight to humans? [Likely]
    - Sketch: intent classifier, containment-success predictor, routing policy, and a monitoring loop on outcomes.
    - How do you measure containment without rewarding the agent for hanging up on customers?

22. Design the shift from Agent Assist to AI Agent for one use case, so a customer can move gradually from human-assisted to fully automated conversations. [Likely]
    - Sketch: assist-mode logs, automation-candidate discovery, agent built from those logs, shadow mode, partial automation, full automation.
    - How do you use assist-mode data (what top agents say) to build the agent?
23. A voice AI agent gets an account number wrong because of an ASR error. How would you redesign the pipeline to make this less likely and to recover when it happens? [Likely]
    - What would you change in ASR biasing, confirmation turns, spelling modes and tool-side validation?
24. Walk through how Cresta's AI Agent produces one response step by step, from transcript to spoken output. Where can the decoding strategy (greedy, sampling, beam, constrained decoding) change the agent's behavior? [Reported] [4] (variant of a 2025 MLE phone-screen question for the AI Agent team)
25. Design sequence decoding with greedy and beam search as a reusable component. Then extend it to an AI agent that must emit valid tool-call JSON: when do you use constrained decoding, and when validation with a retry? [Reported] [4][28] (the decoder component is reported; the tool-call extension is [Likely])
    - Sketch: decoder interface (step function, scoring, stop rules), grammar or schema mask, validator, retry and fallback path, and a latency annotation for each.
26. AI agent tools have side effects (refunds, cancellations, address changes). Design the action layer so a retried LLM step, a dropped connection or a misheard confirmation never applies an action twice or to the wrong account. [Likely] [25]
    - Sketch: tool schema registry, intent-to-action confirmation turn, idempotency keys, a pending-action state machine, the customer system's API, and a compensation (undo) path.
    - Which actions need an explicit spoken confirmation, and who decides?
27. Cresta Conductor's examples describe agents built from defined flows with LLM steps inside them. Design a hybrid agent runtime that mixes a deterministic flow graph with free-form LLM nodes. [Likely] [25]
    - Sketch: flow definition (nodes, transitions, required slots), LLM node executor, slot store, policy checks on transitions, and a fallback to free-form handling or to a human.
    - Which parts of a flight-rebooking flow should be deterministic, and which should the LLM own?
28. A customer calls back the next day about the same issue. Design conversation memory for the AI Agent across sessions and channels (voice, chat, SMS). [Likely]
    - Sketch: customer identity resolution, interaction summaries, memory store with retention rules, retrieval at call start, and redaction.
    - What must the agent never remember, and how do you enforce that?

D. Agent quality: evaluators, testing and the Swiss-cheese model

29. Design an evaluation platform for AI agents that detects regressions across prompt and model changes. [Reported] [5][6]
    - Sketch: scenario store, versioned agent configs (prompt, model, tools), simulator, run orchestrator, deterministic checks, LLM judges, results store,

regression dashboard, CI gate.
    - How many reruns does each scenario need given non-determinism, and what statistical test calls a regression?
30. Cresta describes automated AI agent testing with simulated customers. Design the customer simulator. [Likely] [14][24]
    - Sketch: persona and goal generator mined from real conversations, simulator LLM, agent under test, conversation runner, outcome judge.
    - How do you make simulated customers behave realistically (interrupting, going off-script, being vague)?
    - How do you stop the simulator and the agent from sharing the same blind spots when both are LLMs?
31. Cresta's post on evaluators describes how it grades its agents in an LLM world. Design an evaluator service that scores every production conversation against a set of criteria. [Likely] [14]
    - Sketch: criteria registry, per-criterion judge prompts or classifiers, batch scoring jobs, calibration set with expert labels, agreement monitoring, and results surfaced to customers.
    - How do you measure and maintain agreement between judges and human experts?
    - What happens to historical scores when you upgrade the judge model?
32. Apply the Swiss-cheese model to an AI agent: which failures does each layer (pre-launch tests, human review, production monitoring) catch, and which slip through? [Likely] [23]
    - Sketch: the layers as slices, with failure types drawn as arrows that pass through or are stopped.
    - How do production failures become new pre-launch tests?
33. Design hallucination detection and grounding for agent and assist outputs. [Likely] [26]
    - Sketch: claims extraction, evidence lookup (KB, tool results, transcript), entailment check, scoring, and block-or-flag policy.
    - How do you handle a statement that is true but not supported by any available source?
34. Design a release process for prompt and model changes across hundreds of enterprise customers' agents. [Likely]
    - Sketch: change request, offline evaluation per tenant, staged rollout by tenant cohort, online metrics, automatic rollback.
    - How do you catch a regression that affects only one customer?
35. Design the expert-labeling platform used to calibrate evaluators and judges. [Likely] [14]
    - Sketch: sampling service (stratified and uncertainty-based), labeling UI with rubric, inter-annotator agreement tracking, adjudication, gold-set store, and judge-calibration reports.
    - How many expert labels per criterion do you need before you trust a judge for one customer?
36. Design automated red-teaming for a customer-facing voice agent: social engineering to get account access, prompt injection spoken by the caller, and attempts to make the agent promise refunds it cannot give. [Likely] [23][24]

    - Sketch: attack library, adversarial simulator persona, text-to-speech for spoken attacks, agent under test, policy-violation judge, and a findings tracker that feeds release gates.
    - How do you keep the red-team suite from going stale as the agent's prompts change?
37. Design a live-call quality monitor that scores an AI agent conversation while it is still running and can trigger a human takeover mid-call. [Likely] [14][23]
    - Sketch: turn-level signals (repetition, sentiment, tool failures, policy flags), streaming scorer, takeover policy with hysteresis, warm-transfer service, and outcome logging.
    - How do you set the takeover threshold so it rescues failing calls without doubling human workload?

E. Conversation Intelligence and Insights

38. Design topic discovery over millions of unlabeled conversations: an LLM-generated taxonomy up to three levels deep, which admins can merge, split and rename. [Likely] [21]
    - Sketch: conversation summarization, embedding, clustering, LLM labeling, hierarchy builder, admin edit layer, incremental assignment of new conversations.
    - How do you keep assignments stable when admins edit the taxonomy?
39. Design outcome inference: estimate whether each conversation was resolved, and the customer's experience, without surveys or disposition codes. [Likely] [21]
    - How do you validate inferred outcomes when you have survey data for only a few percent of calls?
40. Design Cresta's "AI Analyst": answer open-ended business questions ("Why did cancellations rise last week?") over a customer's conversation corpus. [Likely] [21]
    - Sketch: question parser, query planner, structured metadata queries, per-conversation LLM extraction over a sample, aggregation, statistical checks, answer with evidence links.
    - How do you avoid presenting a weak correlation as a cause?
    - How do you bound the cost of a question that needs LLM reads of 1M conversations?
41. Design automated QA scoring for all calls against a customer-written rubric. [Likely]
    - Sketch: rubric parser, per-item scorer (rule, classifier or LLM), evidence extraction, score aggregation, dispute and override flow.
    - How do you turn a vague rubric item ("showed empathy") into a reliable judgment?
42. Design a real-time trend detector that flags emerging issues, such as an outage that shows up as a spike in a topic, before dashboards would. [Likely]
    - How do you separate a real spike from seasonality and noise?
43. Design a system that finds what top-performing agents do differently from others, and turns it into coaching guidance. [Likely]
    - Sketch: performance labels, behavior extraction from transcripts, comparison model with controls, insight ranking, coaching content generator.
    - How do you control for call mix, so you do not just learn that top agents get easier calls?

44. Design the batch processing backend that runs dozens of LLM extractions over every conversation for thousands of tenants each day. [Likely]
    - Sketch: ingestion queue, per-tenant scheduling, model router (small model or large model), batching, retries, results store, cost metering.
    - How do you prioritize jobs when GPU capacity is short?
45. Design semantic search over a customer's conversation corpus for analysts ("find calls where the customer mentioned a competitor's price"). [Likely] [21]
    - Sketch: transcript chunking by turn, hybrid lexical and dense index per tenant, metadata filters, reranker, result snippets with audio timestamps.
    - How do ASR errors on brand and product names hurt recall, and how do you compensate?

F. Voice platform and inference graphs

46. Cresta describes its voice platform as running inference graphs of ML services for each conversation. Design that orchestration layer. [Likely] [17]
    - Sketch: a DAG with ASR, diarization, redaction, intent, sentiment, moment detection, summarization; a per-customer policy that enables nodes; an execution engine; and output sinks.
    - How do you express dependencies and skip models a customer has not enabled?
    - How do you isolate a slow node so it cannot delay real-time outputs?
47. Design the voice stack that ingests audio from many CCaaS platforms and telephony providers and produces speaker-separated streaming transcripts. [Likely] [18]
    - Sketch: connectors (SIPREC, media streams, vendor APIs), audio normalization, channel or speaker separation, streaming ASR, transcript bus.
    - How do you handle mono audio where agent and customer are mixed?
48. Design ASR adaptation for contact-center audio: noisy lines, accents and domain vocabulary such as product names. [Likely] [10]
    - Sketch: data collection, transcription/labeling, fine-tuning or contextual biasing, per-tenant vocabularies, and WER evaluation by slice.
    - When is word error rate the wrong metric for downstream tasks?
49. Design PII and PCI redaction for audio and transcripts that runs before storage and training. [Likely]
    - Sketch: streaming entity detection, audio muting aligned to timestamps, transcript masking, audit sampling.
    - What does a missed credit card number cost, and how does that set your recall target?
50. One recording can contain IVR prompts, hold music, a transfer and two different human agents. Design segmentation that labels these spans so downstream models see only the relevant parts. [Likely] [18]
    - Sketch: audio event detection, speaker change detection, role classifier (customer, agent, IVR, bot), transfer detection, and a span map attached to the transcript.
51. Design the evaluation and selection of text-to-speech for a voice agent: naturalness, correct pronunciation of brand names and numbers, and time to first audio byte. [Likely] [12][19]
    - Sketch: test-utterance set by slice, automatic metrics, listener ratings, pronunciation lexicon per tenant, latency harness, and A/B testing on live calls.

52. Your streaming ASR vendor or a GPU pool degrades in the middle of peak traffic. Design graceful degradation for the real-time inference graph. [Likely] [17]
    - Sketch: health checks per node, secondary ASR path, priority tiers that shed non-critical nodes first, circuit breakers, and backfill of skipped nodes after the call.
    - What does the agent's screen show while some features are degraded?

G. Ocean models, fine-tuning and serving

53. Ocean-1 was announced as a contact-center foundation model. Design the training pipeline for a domain-adapted LLM like it. [Likely] [7]
    - Sketch: data sources (licensed and consented conversations, synthetic instructions), cleaning and PII removal, continued pretraining or SFT, preference tuning, evaluation suite, release.
    - What data do you need to train instruction-following for contact-center tasks?
    - How do you show it beats a general frontier model on quality, latency and cost?
54. Design the system that adapts a shared base model to each customer: a per-customer fine-tune, a LoRA adapter, prompts, or retrieval. [Likely] [7][16][22]
    - Sketch: customer data intake, eligibility and consent checks, adapter training jobs, evaluation gate, adapter registry, deployment.
    - When is a per-customer adapter worth its cost compared with a better prompt?
55. Fireworks describes serving many LoRA adapters on one base-model deployment. Design multi-tenant serving for thousands of per-customer adapters. [Likely] [22]
    - Sketch: gateway with tenant auth, router, adapter cache (GPU, CPU, storage), mixed-adapter batching, autoscaler, per-tenant metering.
    - What happens to latency when a cold adapter is requested?
    - How does quantization of the base model interact with adapters?
56. Design the model-routing layer that chooses between a small fine-tuned model and a large frontier model per request. [Likely]
    - Sketch: request features, router (rules or learned), model pool, quality feedback, cost dashboard.
    - How do you train the router without labels for which model would have done better?
57. Design the LLM serving stack for Cresta's real-time features with a p99 latency target. [Likely]
    - What do continuous batching, KV-cache reuse, quantization and speculative decoding each trade off?
    - How do you autoscale GPUs for call-center traffic that peaks in business hours across time zones?
58. Design the data governance pipeline that decides which customer conversations can be used for training shared models and which can only be used for that customer. [Likely]
    - Sketch: consent and contract metadata, data tagging, access-controlled training sets, lineage tracking, deletion propagation.
    - What happens when a customer asks you to delete their data after it has been used for training?
59. Design the benchmark suite that compares Ocean models, open-weight models and frontier APIs on contact-center tasks each time a new model is released. [Likely] [7]

    - Sketch: task registry (summarization, extraction, reply suggestion, tool use), held-out tenant data with consent, metrics and judges per task, latency and cost harness, and a leaderboard with slice views.
    - How do you prevent benchmark data from leaking into the training data of the models you evaluate?
60. How would you turn agents' edits to suggested replies into preference pairs for DPO or similar preference tuning, and what can go wrong? [Likely] [15]
    - Sketch: suggestion and edit log, pair builder (suggested versus sent text), filters for trivial edits, per-tenant weighting, training job, and an evaluation gate.
    - How do you avoid learning one agent's personal style as if it were a preference?

H. Founders' research background and how it shapes the product

These questions draw on public background that is not sourced in this guide. Cresta was founded by Zayd Enam and Tim Shi, both from Stanford AI research; Tim Shi's earlier work includes research on web-based agents (World of Bits) during a period at OpenAI. Verify this before you mention it. The current CEO, Ping Wu, has spoken publicly about how contact centers misunderstand the value of LLMs [8].

61. Early research on web-based agents trained agents to complete tasks in browser environments. How would you apply that idea to an AI agent that operates a contact center's internal tools (CRM, billing) rather than calling APIs? [Likely]
    - Sketch: environment (UI or API), observation encoding, action space, policy (LLM), reward or success check, sandbox.
    - When is UI automation better than an API, and what does it cost in reliability?
62. Cresta's CEO has argued that many contact centers misunderstand where LLMs create value. Where would you put LLMs, and where would you keep smaller models or rules, in a contact-center stack? Draw the stack and defend each choice. [Likely] [8]

I. Cross-cutting design prompts an interviewer may use

63. How would you deploy a model in production? [Reported] [1]
    - Sketch: model registry, CI evaluation gate, shadow deployment, canary, A/B test, full rollout, monitoring and rollback.
    - What do you watch in the first 24 hours, and what triggers a rollback?
64. Design monitoring for LLM features across hundreds of enterprise customers when you have almost no labels. [Likely]
    - Sketch: per-tenant traffic metrics, output-distribution drift, judge-sampled quality, user-feedback signals, alert routing.
    - Which label-free signals catch quality regressions early?
65. Design the onboarding pipeline for a new enterprise customer: from first data access to Agent Assist live in production. [Likely]
    - Sketch: data connectors, historical conversation backfill, ASR and redaction, model adaptation, evaluation with customer stakeholders, launch.
    - What is the minimum data needed before launch?
66. Design a system that lets a customer's admin define a new metric in natural language (for example, "Did the agent offer the upgrade?") and get it computed over past and future conversations. [Likely]

    - Sketch: definition UI, LLM prompt compiler, sample-based validation with admin feedback, backfill job, streaming scorer.
    - How do you show the admin the metric's accuracy before they rely on it?
67. Design the trace and observability system for multi-step LLM agents, so engineers can debug a failed conversation. [Likely]
    - Sketch: span model (turn, LLM call, tool call, retrieval), trace store, search, replay with a modified prompt.
68. Design a cost model and budget controls for LLM usage per tenant. [Likely]
    - How do you prevent one tenant's batch backfill from starving real-time traffic?
69. Design an A/B testing framework for LLM prompt changes where outcomes (CSAT, resolution) arrive days later. [Likely]
    - Which proxy metrics do you use, and how do you validate them against delayed outcomes?
70. How would you design support for many languages in Agent Assist and AI Agent? [Likely]
    - Sketch: language identification, multilingual ASR, translation or native multilingual LLM, per-language evaluation.
71. Design a system that detects compliance violations (required disclosures not read, prohibited statements) in real time and after the call. [Likely]
    - How do you trade recall against precision when a miss means regulatory risk?
72. Design the data flywheel for the whole company: how do conversations flowing through assist, AI agents and analytics improve each product? [Likely]
    - Sketch: shared conversation store, labeling sources (agent actions, judge scores, outcomes), training pipelines per product, evaluation gates.
73. Explain the key concepts of LLMs, GenAI, RAG and AI agents, then draw how they fit together inside one Cresta product of your choice. [Reported] [2][3] (a recruiter told a candidate to expect these concepts; framing them around a Cresta product is [Likely])
74. A European bank requires its audio, transcripts and model calls to stay in the EU. Design multi-region deployment for the real-time and batch ML stack under data-residency rules. [Likely]
    - Sketch: regional data planes, a global control plane holding only configuration, region-pinned model serving and adapters, regional evaluation jobs, and cross-region telemetry with no conversation content.
    - Which shared models can you still improve with EU data, and how?

J. Product-sense questions that lead into design

75. Which Cresta product would you work on first, and what is its hardest ML problem? [Likely]
    - How would you sketch the current system and the change you would make?
76. What are the main ML differences between assisting a human agent and replacing one? [Likely]
    - How does each change the latency, accuracy and evaluation requirements?
77. What would you need to believe to build an in-house foundation model rather than fine-tune open models or call frontier APIs? [Likely] [7]
78. If a competitor's AI agent claims 80% containment, how would you check that claim, and how would you measure your own? [Likely]

79. What makes contact-center conversations harder for ML than generic chat data? [Likely]
    - How do ASR errors, long calls, multiple parties and hold music affect your pipeline design?
80. How would you prioritize between improving ASR accuracy and improving the LLM layer, given a fixed team? [Likely]
81. Which Cresta engineering blog post did you read, and what would you do differently in the design it describes? [Likely] [17][18][19]
82. How should a product like Cresta's explain its suggestions or scores to agents and supervisors, and what does that require from the system design? [Likely]
83. Where do you see the biggest risk of customer-visible failure in Cresta's AI Agent, and how would you design against it? [Likely]

K. Track-specific prompts (by team in the job postings)

84. Agentic Assist track: design an assist agent that can take actions for the human agent (fill forms, look up orders) rather than only suggest text. [Likely] [13]
    - How do you confirm actions with the human before they run?
85. Agentic Assist track: how would you combine pre-LLM ML (classifiers, ranking) with LLMs in one assist pipeline? [Likely] [13]
86. Agent and System Quality track: design the metric set and dashboards used to decide whether an AI agent is ready to launch for a customer. [Likely] [9][11]
    - Which metrics block a launch, and which are only tracked?
87. Agent and System Quality track: how would you diagnose context drift and multi-step reasoning failures in long conversations? [Likely] [11]
88. Voice/ASR track: design the evaluation of a new streaming ASR model against the current one, including latency and downstream task impact. [Likely] [10][12]
    - Sketch: test-set curation by slice, offline WER and latency, downstream replay through assist models, shadow traffic, rollout.
89. Voice Experience track: design natural turn-taking for a voice agent: endpointing, backchannels and interruption handling. [Likely] [12][19]
    - How do you evaluate turn-taking quality offline?
90. Insights track: design a pipeline that keeps conversation-level LLM extractions consistent when the underlying model is upgraded. [Likely] [21]
    - How do you backfill history, or version metrics, so customer dashboards do not jump?
91. Voice Experience track: design measurement of perceived response latency for live voice-agent calls, using the recordings rather than server logs. [Likely] [12][19]
    - Sketch: dual-channel recording, voice activity detection on both sides, gap computation per turn, attribution of each gap to pipeline stages using trace timestamps, and percentile dashboards per tenant.
92. Agentic Assist track: design an "autopilot" mode in chat that drafts a full reply and sends it after a short timeout unless the human agent edits or cancels it. [Likely] [13]
    - Sketch: draft generator, confidence and risk scorer, per-intent eligibility policy, countdown UI, send service, and audit log.
    - Which intents should never be eligible, and how do you decide that from data?
93. Agent and System Quality track: design automatic root-cause clustering of failed AI agent conversations, so engineers see a ranked failure taxonomy instead of single

transcripts. [Likely] [11]
    - Sketch: failure detection, trace featurization (step, tool, error type), LLM failure summaries, clustering, ranking by volume and severity, and links to replay.

L. Questions you can ask the interviewer about Cresta's systems

94. How is the inference graph configured per customer today, and what is the hardest part of operating it? [Likely] [17]
95. How much of the real-time stack runs on self-hosted fine-tuned models and how much on third-party APIs? [Likely] [16][22]
96. How do evaluators and simulated testing gate AI agent releases today? [Likely] [14] [24]
97. What is the current state of the Ocean model family, and how does it relate to the per-customer adapters? [Likely] [7]
98. Which parts of Conductor and AI agent building are owned by ML engineers, and which by product engineers? [Likely] [25]

References

1. Glassdoor, "Cresta (CA) Interview Questions" (Senior Machine Learning Engineer review: recruiter call, LeetCode-medium, vibe-coding and system design rounds; "How would you deploy a model in production?"), 2025-2026. https://www.glassdoor.co.in/Interview/Cresta-CA-Interview-Questions-E3180909.htm
2. Blind, "Cresta interview process", 2024-2025. https://www.teamblind.com/post/cresta-interview-process-jmfnzjoa
3. Blind, "Anyone interview with Cresta lately?", 2025. https://www.teamblind.com/post/anyone-interview-with-cresta-lately-zvj0uhq2
4. 1point3acres, "Cresta Machine Learning Engineer Tech Phone Interview Experience" (MLE, AI Agent direction; greedy and beam-search decoding, how an agent generates a sequence), 2025. https://www.1point3acres.com/interview/thread/1140481
5. PracHub, "Cresta ML System Design Interview Questions" ("Design an Evaluation Platform for AI Agents Across Prompt and Model Changes"), 2026. https://prachub.com/companies/cresta/categories/ml-system-design
6. PracHub, "Cresta Software Engineer Interview Experience: System Design Round: Build an Eval Platform for AI Agents". https://prachub.com/interview-experiences/cresta-software-engineer-interview-experience-system-design-round-build-an-eval-platform-for-ai-agents
7. Cresta Blog, "Introducing Ocean-1: World's First Contact Center Foundation Model", 2023. https://cresta.com/blog/introducing-ocean-1-worlds-first-contact-center-foundation-model
8. CX Today, "Cresta CEO Ping Wu: Many Contact Centers Misunderstand the Value of LLMs". https://www.cxtoday.com/contact-center/cresta-ceo-ping-wu-many-contact-centers-misunderstand-the-value-of-llms/
9. Cresta on Greenhouse, "Senior Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4249943008
10. Cresta on Greenhouse, "Senior Machine Learning Engineer (ASR)". https://job-boards.greenhouse.io/cresta/jobs/5101011008

11. Cresta on Greenhouse, "Staff Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4741283008
12. Cresta on Greenhouse, "Senior Machine Learning Engineer, Voice Experience". https://job-boards.greenhouse.io/cresta/jobs/5155675008
13. Built In Vancouver, "Senior Machine Learning Engineer, Cresta" (Agentic Assist track). https://builtinvancouver.org/job/senior-machine-learning-engineer/8619914
14. Cresta Blog, "How Cresta Grades Its Agents: Evaluators in an LLM World". https://cresta.com/blog/how-cresta-grades-its-agents-evaluators-in-an-llm-world
15. Cresta Blog, "How Fine-Tuned LLMs Power Knowledge Assist, Summarization and Chat Suggestions", 2024. https://cresta.com/blog/how-fine-tuned-llms-power-knowledge-assist-summarization-and-chat-suggestions
16. Fireworks AI Blog, "Story: Cresta Knowledge Assist", 2024. https://www.fireworks.ai/blog/story-cresta-knowledge-assist
17. Cresta Blog, "Understanding Cresta's Voice Platform: ML Services, Inference Graphs and Real-Time Intelligence", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-ml-services-inference-graphs-and-real-time-intelligence
18. Cresta Blog, "Understanding Cresta's Voice Platform: The Voice Stack", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-the-voice-stack
19. Cresta Blog, "Engineering for Real-Time Voice Agent Latency". https://cresta.com/blog/engineering-for-real-time-voice-agent-latency
20. Cresta Blog, "AI-Ready Knowledge for Contact Centers: Closing the Gap Between KBs and AI", 2025. https://cresta.com/blog/ai-ready-knowledge-for-contact-centers-closing-the-gap-between-kbs-and-ai
21. Cresta Blog, "Introducing the Next Generation of Cresta Insights: A Complete System for Enterprises to Understand Their CX", 2026. https://cresta.com/blog/introducing-the-next-generation-of-cresta-insights-a-complete-system-for-enterprises-to-understand-their-cx
22. Fireworks AI Blog, "Multi-LoRA", 2024. https://fireworks.ai/blog/multi-lora
23. Cresta Blog, "Why AI Agent Evaluations Fail, and How the Swiss-Cheese Model Prevails". https://cresta.com/blog/why-ai-agent-evaluations-fail----and-how-the-swiss-cheese-model-prevails
24. Cresta Blog, "Cresta Launches Automated AI Agent Testing: Confidence in Every Conversation". https://cresta.com/blog/cresta-launches-automated-ai-agent-testing-confidence-in-every-conversation
25. Cresta Blog, "Cresta Conductor: The Agent for AI Agent Development". https://cresta.com/blog/cresta-conductor-the-agent-for-ai-agent-development
26. Cresta Blog, "Grounding Reality: How Cresta Tackles LLM Hallucinations in Enterprise AI". https://cresta.com/blog/grounding-reality---how-cresta-tackles-llm-hallucinations-in-enterprise-ai
27. Glassdoor, "Cresta (CA) Interview Experience and Questions" (Forward Deployed Engineer technical screen: "What is RAG, and how do you prevent hallucination?"), 2026. https://www.glassdoor.com/Interview/Cresta-CA-Interview-Questions-E3180909.htm
28. PracHub, "Cresta Interview Questions" (company page; "Design sequence decoding with greedy and beam search"), 2025. https://prachub.com/companies/cresta

Changed in review: No cited page could be opened (proxy refusals and DNS failures for every domain), and no web search was possible, so citations were checked against the

search-result excerpts already reviewed in chapters 01-05. Removed as uncited or unsupported: the second Glassdoor regional mirror (glassdoor.ca), the Blind "Cresta senior backend SWE interview" thread (its excerpt supports only an online assessment), the Blind AMA thread (no excerpt in this guide supports it), the 1point3acres company page (uncited here) and the Scoutify guide (a generic template, not a candidate report). The 1point3acres thread title was corrected. Two references were added: the Glassdoor FDE report and the PracHub company page. Both [Reported] tags in the original chapter (agent evaluation platform; deploying a model in production) match reviewed excerpts and were kept. Four [Reported] variants from adjacent Cresta rounds were added, each naming its round. Two questions were removed as duplicates: "extract top agents' behaviors and generalize them" (same as the top-performer coaching design) and "human-in-the-loop assist-then-automate" (same as the assist-to-AI-Agent transition design). All other added questions are [Likely].

08. ML System Design Reference Diagrams: Agent Assist, Voice AI Agent, Agent Evaluation, Model Deployment, Knowledge Assist RAG, Conversation Intelligence and Multi-Tenant LoRA Serving

Why this chapter exists. Chapters 04 and 07 give a "Sketch" prompt for most design questions, but no drawn diagram. This chapter draws seven labeled box-and-arrow architectures for the core Cresta prompts. Under each diagram is a set of whiteboard questions an interviewer can ask about that drawing. The diagrams are reference skeletons to practice against. They are not Cresta's real architecture, and the questions have no written answers.

Tagging system (used on every diagram and question).

- **[Reported] [n]**: a candidate report or question bank attributes the prompt, or a close variant, to Cresta. [n] points to the References list. Where the report came from a different round or role, the tag says so.
- **[Likely]**: inferred from Cresta's products, engineering blog and ML job postings. No candidate has reported it.
- **Back-pointers**: "ch04 Q5" means question 5 in chapter 04 (ML System Design). "ch07 Q1" means question 1 in chapter 07 (Cresta Product and Research Context).

How to read the diagrams.

- Boxes are components. Arrows are numbered data flows: (1), (2) and so on.
- The **budget** on an arrow is an illustrative latency target written for practice. It is an assumption, not a number published by Cresta, unless a citation is attached. Interviewers usually want you to state your own budget and defend it.
- "async" marks a flow that is off the latency-critical path.
- The diagrams are plain-text (ASCII) so that they render the same in Markdown, in a terminal and in a PDF export.

Research limits. In the adversarial review of this chapter, 10 cited URLs were tried (PracHub ML system design list and eval-platform experience, three cresta.com blog posts, Fireworks Knowledge Assist story, Glassdoor, Blind, Built In Vancouver and 1point3acres). Every one failed: WebFetch returned DNS errors and the egress proxy refused the CONNECT (HTTP 403). The shared web-search budget

> was also used up, so no new candidate reports could be searched for. No URL could be shown to be dead, and none was shown to be live. Every citation is carried over from the reviewed excerpt lists in chapters 01, 04, 07 and 10. Only two Cresta ML system design prompts are on record: the agent evaluation platform and "How would you deploy a model in production?". Those two diagrams, and only their headline prompts, are tagged [Reported]. Follow-up questions under them are the guide's own extrapolation and are tagged [Likely]. All other diagrams are [Likely]. The Knowledge Assist diagram extends a [Reported] conceptual question from a Forward Deployed Engineer screen, and its tag says so.

Format notes for this round

- **Length and slot.** Senior/Staff loop reports describe one hour of system design alongside an hour of coding and a hiring-manager chat [2]. A 2025-26 Senior MLE report lists a systems design round after a recruiter call, an LC-medium round and a "vibe coding" round [1].
- **Reported prompts.** "How would you deploy a model in production?" (Senior MLE) [1]. "Design an Evaluation Platform for AI Agents Across Prompt and Model Changes" (PracHub ML system design list, with a linked Cresta SWE interview experience) [4] [5].
- **Recruiter-flagged scope.** A recruiter told one candidate the interviews would cover key concepts and fundamentals of LLMs, GenAI, RAG and AI agents [3]. A 2026 FDE screen asked "What is RAG, and how do you prevent hallucination?" [6].
- **Interviewer (inferred).** Most likely a senior or staff MLE from the target track: Agentic Assist, Agent and System Quality, Voice/ASR, Voice Experience or Insights [7] [8][9][10][11].
- **Senior bar (from postings).** End-to-end ownership from research to production [11]. Evaluation of non-deterministic systems with offline benchmarks, online experiments and LLM-as-a-judge [8]. Diagnosis of hallucination, retrieval errors, tool misuse and context drift [7][8]. Design for latency, cost and real-time streaming inference in voice [9].
- **Where diagrams matter (inferred).** Expect to draw the architecture within the first 10-15 minutes, then defend one or two arrows in depth: the latency budget, the evaluation loop, multi-tenancy or the rollout path. Practice drawing each diagram below from memory in under five minutes.

Questions

Diagram 1. Real-time Agent Assist path [Likely]

Points back to: ch04 Q5 (Agent Assist), Q6 (moment detection), Q9 (inference graph), Q13 (delivery layer); ch07 Q1 (Agent Assist), Q8 (partial transcripts), Q46 (inference graphs), Q47 (voice stack). Sources for the product shape: [12][13].



Drawing and data-flow questions

1. Draw the real-time Agent Assist path from live call audio to a hint on the agent's screen. Number every arrow and write a latency budget on each. [Likely]
    - Which arrow dominates end-to-end latency, and what evidence would show that?
    - Which arrows carry audio, which carry text events, and which carry UI pushes? How does the choice of transport differ for each?
    - Where would you place queues, and where must you avoid queueing altogether?
2. How do you get audio out of the customer's telephony or CCaaS platform (arrow 1) when each vendor exposes a different integration? [Likely] [12]
    - What breaks when a vendor gives you mixed mono audio instead of two channels?
    - How do you detect and recover from dropped or out-of-order media packets?

3. On arrow 3, which downstream nodes should consume partial transcripts, and which should wait for finals? [Likely] [12]
    - How does a detector avoid firing twice when a partial is revised?
    - What do you do when a final transcript contradicts a hint that is already on screen?
4. Where does diarization sit (arrow 3a), and what does each downstream model need from it? [Likely]
    - How would you label IVR prompts, hold music and transfers so they do not trigger hints?
    - What is the cost of a speaker-attribution error for a compliance hint versus a sales hint?
5. Why partition the transcript bus by conversation id, and what ordering guarantees does arrow 4 need? [Likely]
    - What happens to in-flight events when a consumer instance restarts mid-call?
    - How do you size the bus for thousands of concurrent calls?

Inference graph and guidance questions

6. Inside the inference graph executor, which nodes run on every utterance, which run on triggers, and which run only after the call? [Likely] [13]
    - How do per-tenant policies switch nodes on and off without separate deployments?
    - How do you stop one slow node from delaying all the others?
7. How would you meet a 20-80 ms budget for intent and hint classifiers while the suggestion LLM takes hundreds of milliseconds? [Likely]
    - Would you show a fast hint first and refine it later, or wait for the slow path?
    - What would you cache, and what is the cache key?
8. Design the guidance policy box: how does it decide which of several candidate hints to show, and when to stay silent? [Likely]
    - Which signals feed the alert-fatigue model?
    - How would a customer admin tune thresholds without breaking other tenants?
9. How does feedback (arrow 7) reach model training without slowing the hot path? [Likely]
    - Which feedback events are reliable labels, and which are biased by what was shown?
10. Which boxes in this diagram must be redundant for a live call, and what does the agent see when each one fails? [Likely]
    - What is the degradation order: which nodes do you shed first under load?
11. How would you change this diagram for chat Agent Assist, where there is no audio but an agent handles several conversations at once? [Likely]
    - Which boxes disappear, and which new ones (a conversation prioritizer, for example) appear?
12. Where would you add PII redaction in this diagram, and on which arrows must raw audio or text never travel? [Likely]

Session lifecycle and audit questions

13. Redraw Diagram 1 as a call-session state machine (ringing, connected, hold, transfer, conference, wrap-up). Which boxes hold per-call state, and what happens to that state on a warm transfer to another agent? [Likely] [12][13]

- Which hints must be suppressed while the customer is on hold or in the IVR?

14. Where would you attach provenance (model version, input utterance ids, policy rule) to each hint so that a supervisor or customer can audit why it was shown? [Likely] [13]
15. How would you replay a recorded call through this diagram offline, with realistic partial-transcript timing, to reproduce a hint bug deterministically? [Likely] [12][13]

- Which arrows must be recorded with timestamps for the replay to be faithful?

16. Add a supervisor branch to Diagram 1: one supervisor watches 30 live calls with real-time alerts. Which new arrows and fan-out does that add, and what is their latency budget? [Likely] [12]

Diagram 2. Voice AI agent loop [Likely]

Points back to: ch04 Q19 (voice agent), Q20 (tool layer), Q21 (handoff), Q23 (guardrails), Q25 (peak latency); ch07 Q17 (AI Agent), Q18 (latency pipeline), Q20 (guardrails), Q26 (side-effecting tools), Q89 (turn-taking). Sources for the product shape: [9][14][18][19].



Turn-taking and latency questions

17. Draw the voice agent loop from caller audio to agent audio and back, with a latency budget on each arrow that adds up to your end-to-end target. [Likely] [14]
    - Which budget would you cut first if the total came to 1.6 s?
    - Which stages can overlap (streaming) and which must run in sequence?
    - How do you measure the gap the caller actually hears, as opposed to the sum of component timings?
18. How does the VAD and turn-taking box decide that the caller has finished (arrow 2)? [Likely] [9]
    - What do you do for slow speakers, people reading out account numbers, and long pauses mid-thought?

    - Could a semantic end-of-turn model replace a fixed silence threshold? What would it cost in latency?
19. Walk through barge-in (arrow 2a): the caller interrupts while the agent is speaking. [Likely]
    - How does the planner learn which part of its answer the caller actually heard?
    - How do you tell a real interruption from a backchannel ("uh-huh") or background noise?
20. Where would you speculate (start the LLM before the endpoint is final, pre-synthesize filler), and how do you discard wasted work? [Likely]
21. How does an ASR error on arrow 3 (for example, a wrong account digit) spread through the rest of the loop, and where would you catch it? [Likely] [10]
    - Which confirmations would you add, and what do they cost in call time?

Planner, tools and guardrail questions

22. What state does the LLM planner hold, and where does it live so a crashed worker can resume the call? [Likely]
23. Draw the tool gateway (arrow 4a) for side-effecting actions such as refunds. [Likely]
    - How do idempotency keys and confirmation turns fit into the diagram?
    - What does the agent say while a slow backend API is running?
    - How would you test tools without touching the customer's production systems?
24. When does the planner call RAG (arrow 4b), and how does that change time to first token? [Likely] [19]
25. Where do guardrails run (input, planning, output), and what latency does each add? [Likely] [19]
    - Which checks are small classifiers and which are LLM calls?
    - A sentence fails the output check after earlier sentences have already been spoken. What happens?
26. Which signals trigger the human handoff branch (5b), and what context travels with the call? [Likely]
    - How do you avoid escalating too early or too late?
27. How would you change the diagram to keep latency stable during a 5x call spike after an outage? [Likely]
    - Which box degrades first, and what is the fallback for each?
28. What goes into the per-turn trace, and how is it used by the evaluation platform in Diagram 3? [Likely] [15]
29. How would you redraw this loop for a structured flow (nodes, transitions, slots) with LLM steps inside, as in Cresta's agent-building examples? [Likely] [18]
    - Which box owns the flow state, and how do you stop the LLM from skipping a required step?

Speech front-end and context questions

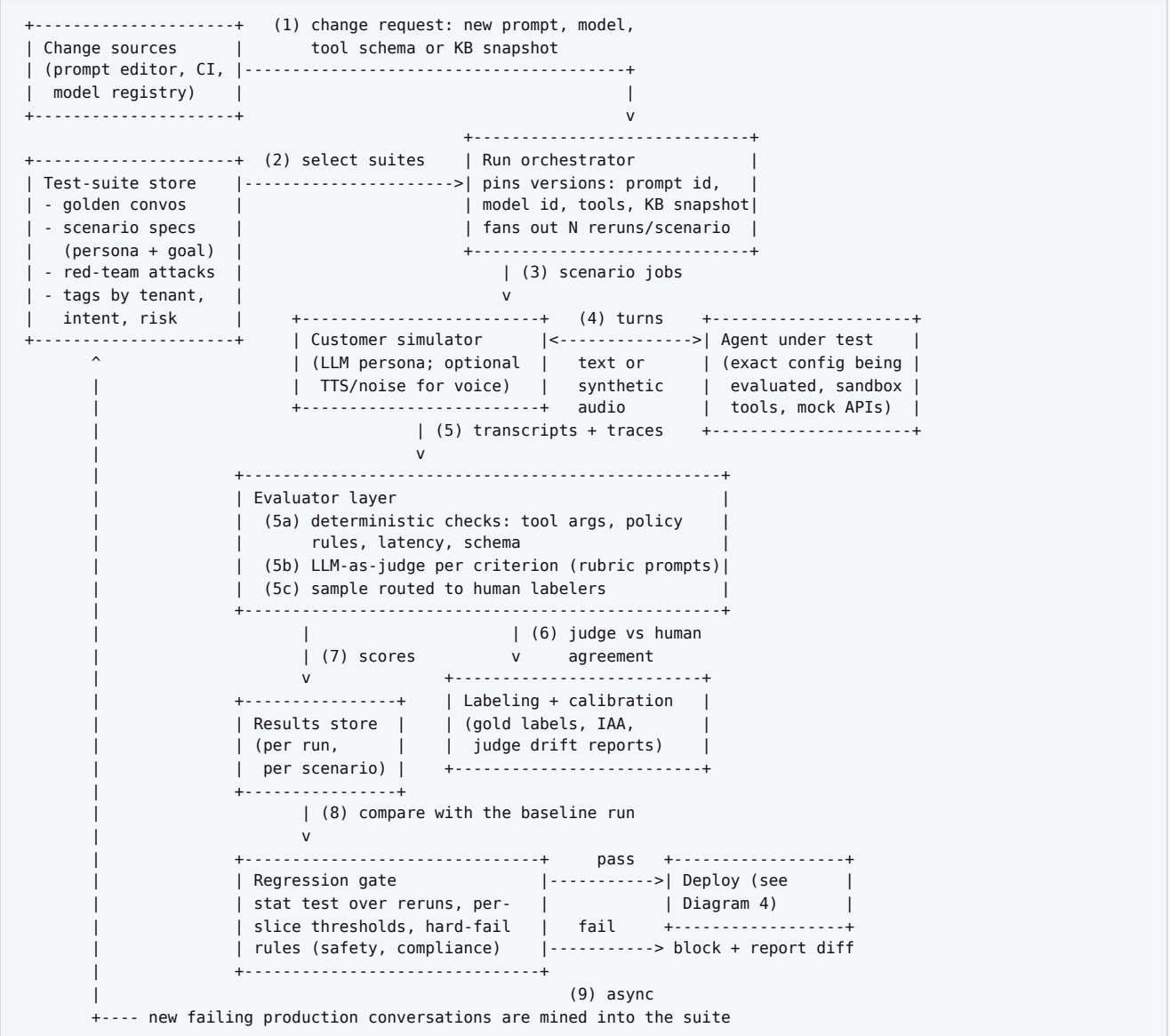
30. Where in Diagram 2 would language identification sit, and what happens to ASR, the planner and TTS when the caller switches language mid-call? [Likely] [9][10]
31. A caller must read out a card number or press DTMF keys for payment. Redraw the loop so those digits bypass the LLM, the trace store and recordings, and show which box resumes control afterward. [Likely] [12][19]

32. Draw what goes into the planner's prompt on turn 40 of a long call. Where do summarization of older turns and retrieval of earlier facts happen, and what do they add to time to first token? [Likely] [14]
33. How would the planner pass expected-entity hints (for example, "the next answer is an order number") back to the ASR box for contextual biasing, and which arrow carries them? [Likely] [10]

- How do you measure whether biasing helped entity accuracy without hurting general word error rate?

Diagram 3. Agent evaluation platform for prompt and model changes [Reported] [4][5]

Points back to: ch04 Q1 (reported eval platform), Q33 (customer simulator), Q35 (LLM-as-a-judge), Q38 (release of prompt changes), Q42 (golden sets); ch07 Q29 (reported eval platform), Q30 (simulator), Q31 (evaluators), Q34 (release process), Q35 (expert labeling). Product context: [15][16][17].



Pipeline and reproducibility questions

34. Draw an evaluation platform that catches regressions when a prompt or model changes, and name what flows on each arrow. [Reported] [4][5]
    - Who triggers a run (arrow 1): an engineer, CI, or a model-registry event?
    - Who reads the result: ML engineers, forward-deployed engineers or the customer? How does that change the report?
35. What must the run orchestrator pin so that a run can be reproduced exactly a month later? [Likely] (follow-up to the reported prompt [4][5]; the follow-up itself is not reported)
    - How do you snapshot a knowledge base and mock tool responses that change over time?
36. How many reruns per scenario do you need, and which statistical test turns noisy scores into a pass or fail at the gate (arrow 8)? [Likely]
    - How do you keep a flaky scenario from blocking every release?
37. What is in the test-suite store, and how is each scenario tagged so you can report by tenant, intent and risk? [Likely] [17]
    - How does a failing production conversation become a new scenario (arrow 9) without leaking PII?

Simulator and evaluator questions

38. How do you make the customer simulator (arrow 4) realistic enough that passing the suite predicts production behavior? [Likely] [17]
    - How would you mine personas and goals from real conversations?
    - How do you stop the simulator from being too cooperative?
    - What changes when the agent under test is a voice agent and the simulator must speak?
39. Which checks belong in the deterministic box (5a), and which genuinely need an LLM judge (5b)? [Likely] [15]
    - How do you score the trajectory (tool calls, steps) separately from the final outcome?
40. How do you calibrate the LLM judge against human labels (arrows 5c and 6), and what agreement level is good enough to gate a release? [Likely] [15]
    - What do you do when the judge model itself is upgraded?
    - How do you catch a judge that prefers the style of its own model family?
41. Apply the Swiss-cheese model to this diagram: which failure types does each layer stop, and which pass through every layer? [Likely] [16]
42. How do you scale to thousands of scenarios across hundreds of customer-specific agents, and what does a full run cost? [Likely] (follow-up to the reported prompt [4] [5])
    - Which runs are per-tenant, and which are shared across tenants?
    - How would you cut cost without losing the ability to detect a regression?
43. A change improves the average score but breaks one large customer's top intent. Where in the diagram is that caught, and who decides whether it ships? [Likely]
44. Where would red-team scenarios (prompt injection, social engineering) enter this diagram, and should they hard-fail the gate? [Likely]

Online evaluation and authoring-loop questions

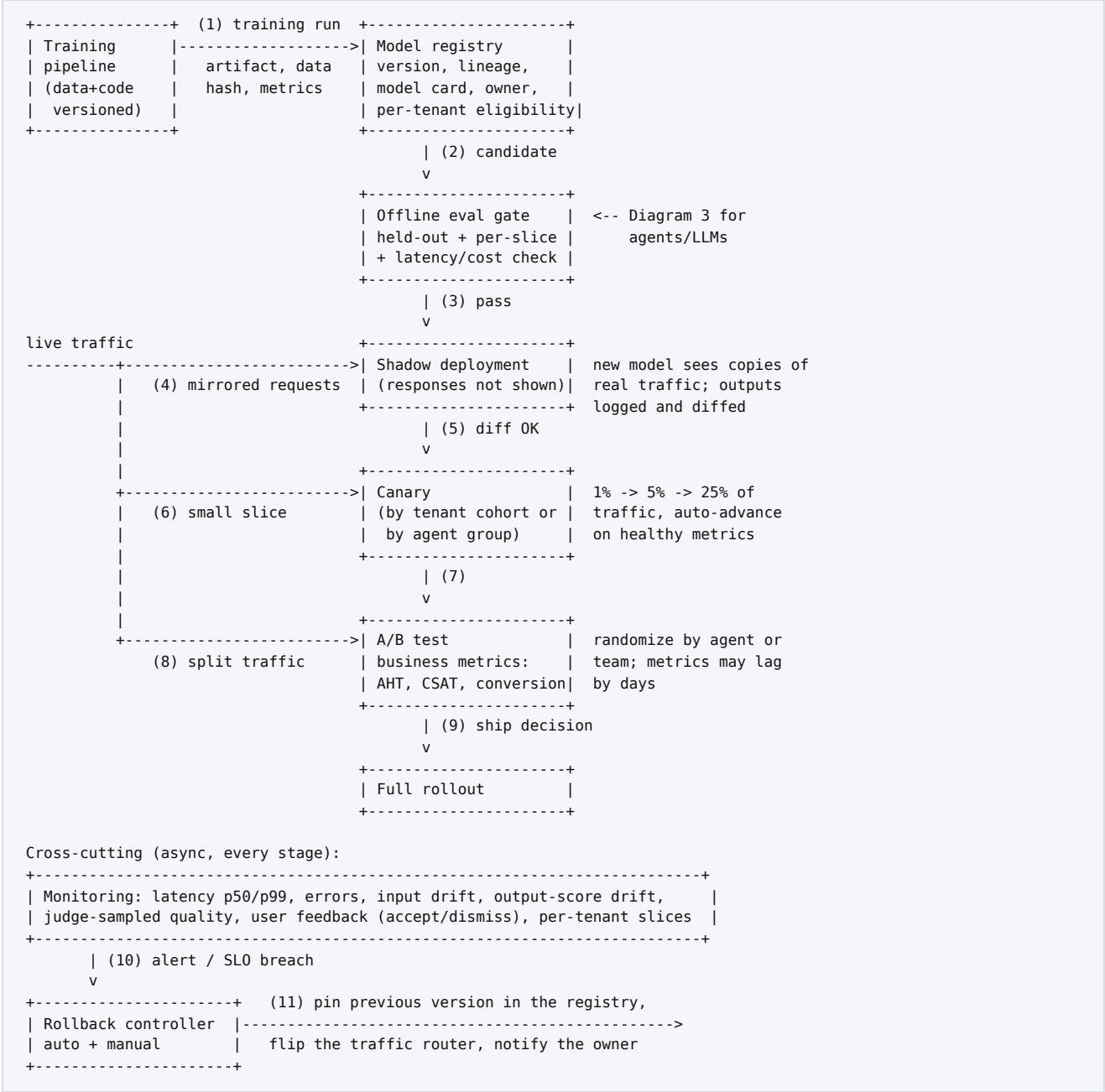
45. Draw the online counterpart of Diagram 3: production conversations sampled to the same evaluators and results store. What do you do when offline suite scores and online judge scores disagree? [Likely] [15][16]
46. An agent-building assistant (as in Cresta Conductor) proposes prompt edits and runs the suite automatically. Where does it plug into Diagram 3, and how do you stop it from overfitting the test suite? [Likely] [17][18]

- Which scenarios would you hold out from the assistant entirely?

47. Which evaluator boxes would you add for voice-only failures (response latency, talk-over, missed barge-in, mispronounced names), and where does their input come from? [Likely] [14][17]
48. When the gate fails, what does the diff report show the prompt author, and how does each failing scenario link back to its full trace? [Likely] [15]

Diagram 4. Deploying a model in production [Reported] [1]

Points back to: ch04 Q2 (reported "deploy a model in production"), Q38 (prompt change release), Q70 (per-customer fine-tuning); ch07 Q34 (release process), Q63 (reported deploy question), Q64 (monitoring).



Rollout path questions

49. Draw how a model goes from training to full production, with every gate and the rollback path. [Reported] [1]
    - What does the registry entry contain, and who is allowed to promote a version?
    - Which gates are automatic and which need a human sign-off?
50. When is shadow mode (arrow 4) worth its GPU cost, and what do you compare between the shadow and production outputs? [Likely]
    - How do you evaluate shadow outputs for a generative model when there is no single correct answer?
51. How do you choose the canary unit (arrow 6) when a few large customers dominate traffic? [Likely]
    - Why might you canary by tenant cohort rather than by random request?
52. How do you randomize the A/B test (arrow 8) when agents work in teams and business metrics arrive days later? [Likely]

    - What is your guardrail metric, as opposed to your success metric?
    - How long must the test run, and how do you deal with novelty effects on agents?
53. How does this diagram differ for a real-time model (assist hints) and a batch model (post-call QA scoring)? [Likely] (follow-up to the reported prompt [1])
    - For the batch model, what replaces canary traffic?

Monitoring and rollback questions

54. Which monitoring signals (box 10) trigger an automatic rollback in the first 24 hours, and which only page a human? [Likely] (follow-up to the reported prompt [1])
    - How do you detect a quality regression when you have almost no labels?
55. How does the rollback controller (arrow 11) undo a model that also changed a prompt template, a feature schema or an index? [Likely]
    - What must be versioned together so that rollback is one operation?
56. How do you manage per-customer model versions, so that one tenant can stay pinned while others upgrade? [Likely] (follow-up to the reported prompt [1])
    - How do you stop the number of live versions from growing without limit?
57. Where in this diagram would a prompt-only change enter, and which gates can it skip? [Likely]
58. How would you prove to an enterprise customer, after the fact, which model version produced a given suggestion on a given call? [Likely]

Dependencies and change-management questions

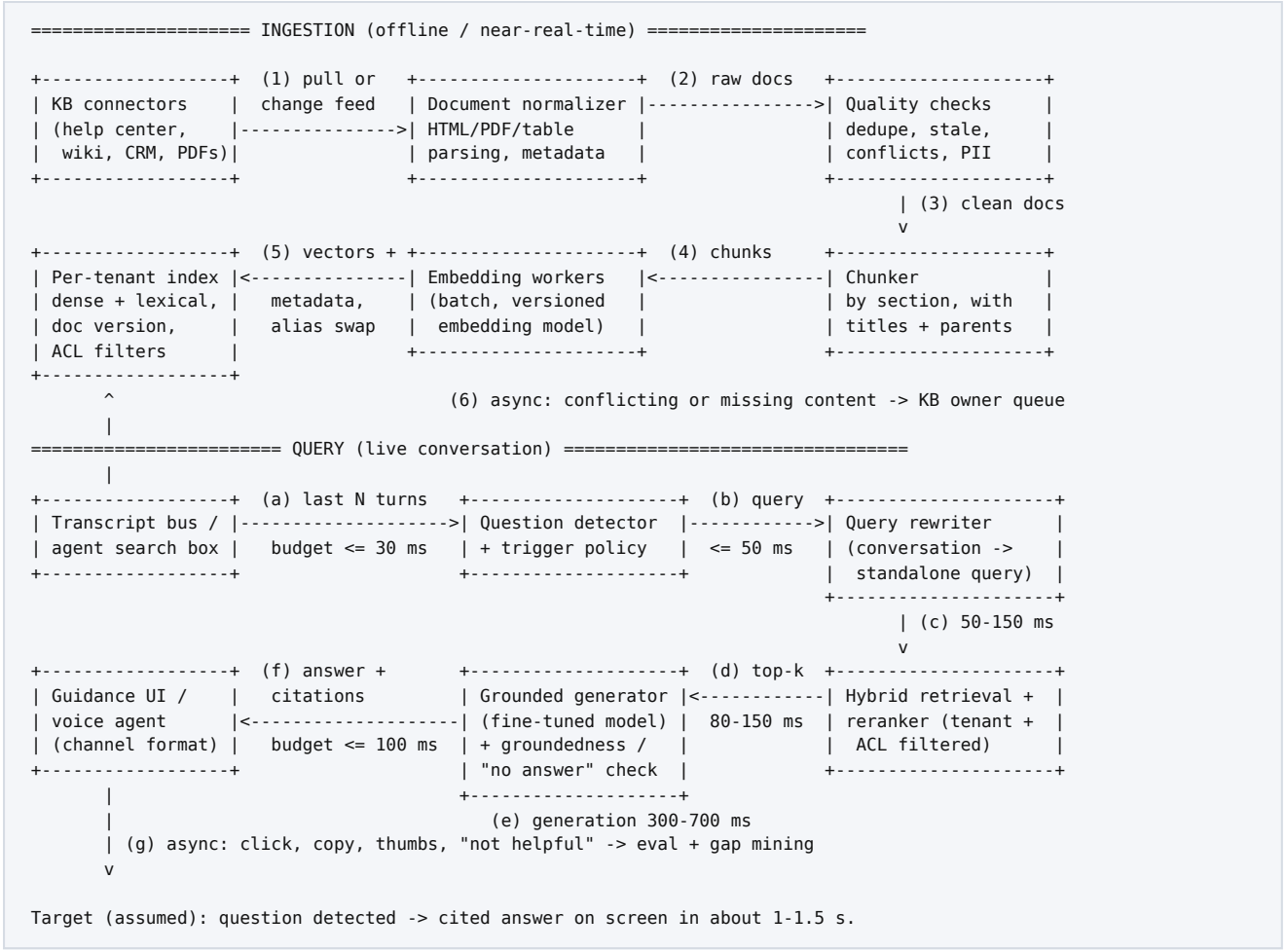
59. Where in Diagram 4 do you log the features and inputs a model saw at serving time, so the next training run does not suffer training-serving skew? [Likely]
60. Diagram 1 chains ASR into intent, hint and suggestion models. How do you canary a new ASR model when every downstream model was trained on the old ASR's output? [Likely] [10][13]

- Which downstream metrics would you gate on, in addition to word error rate?

61. An enterprise customer requires advance notice and approval before the model behind its agents changes. Where does that approval sit in Diagram 4, and how does it interact with shared base-model upgrades? [Likely] [24]

Diagram 5. Knowledge Assist: RAG ingestion and query [Likely] (extends a [Reported] FDE question [6])

Points back to: ch04 Q3 (reported RAG and hallucination, FDE screen), Q58 (Knowledge Assist), Q59 (fine-tuned serving), Q60 (AI-ready ingestion), Q62 (retrieval evaluation); ch07 Q9 (Knowledge Assist), Q11 (fine-tuned migration), Q13 (grounding), Q14 (RAG and hallucination), Q15 (fresh KB updates). Product context: [19][20][21][22].



Target (assumed): question detected -> cited answer on screen in about 1-1.5 s.

Ingestion questions

62. Draw the ingestion side of Knowledge Assist, from the customer's knowledge sources to a searchable per-tenant index. [Likely] [20]
    - How do you detect stale, duplicated and contradictory articles (box after arrow 2), and who fixes them?
    - How do you handle tables, PDFs and screenshots?
63. How do you keep the index fresh when an article changes during the business day (arrows 1 and 5)? [Likely]
    - What does the alias swap protect against, and what does it cost?
    - How do you record which article version an answer used?
64. How would you choose chunk boundaries for contact-center knowledge (procedures, eligibility rules, pricing tables)? [Likely]
65. Plan an embedding-model upgrade that re-embeds every tenant's index without downtime. Where in the diagram do the old and new indexes coexist? [Likely]

Query and grounding questions

66. Draw the query path from a live conversation to a cited answer, with a latency budget on each arrow. [Likely] [21][22]
    - Which step would you cache, and what is the cache key?
67. What is RAG, and where in this diagram do you prevent hallucination? [Reported] [6] (FDE technical screen; expect a senior MLE design round to ask for the diagram as

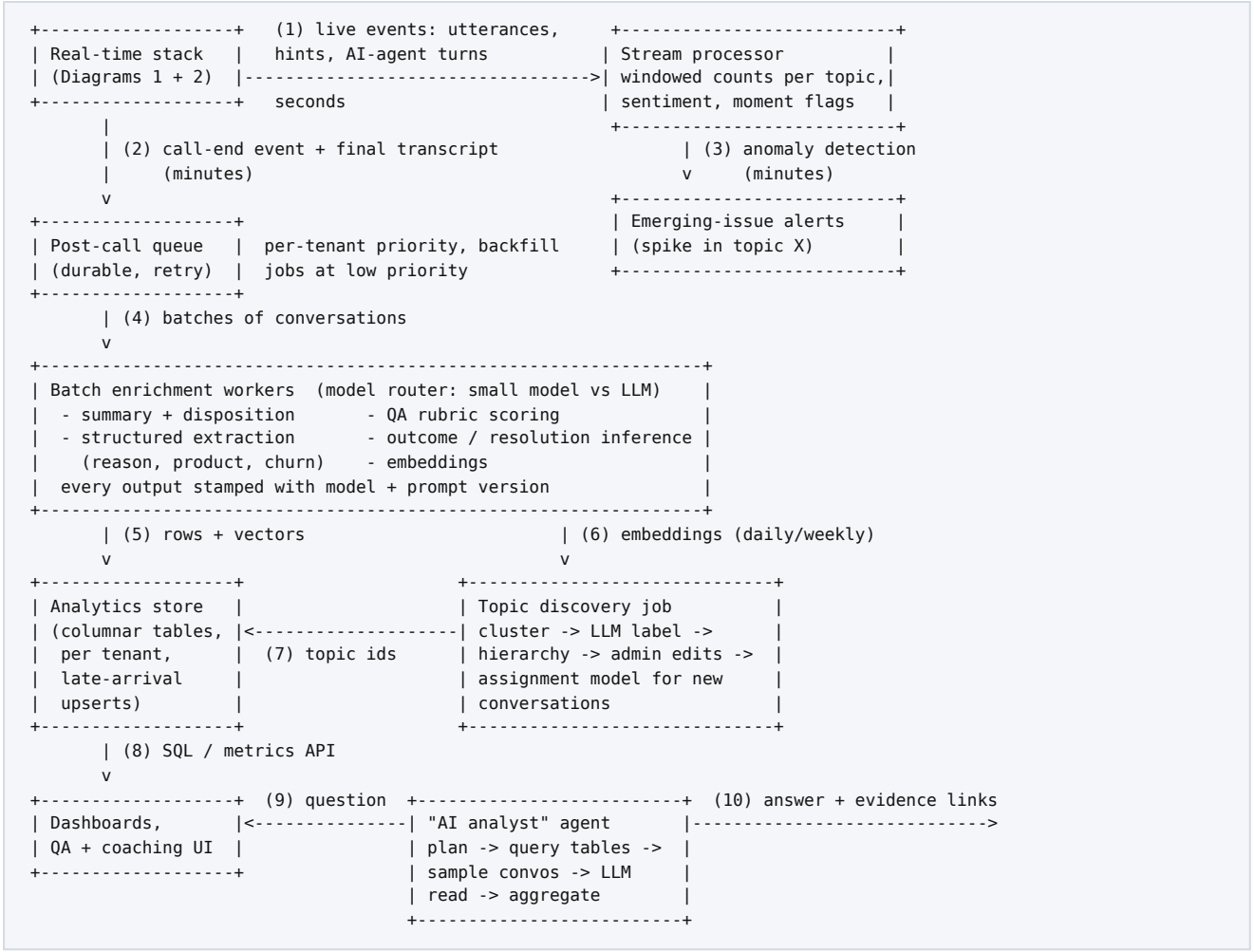
well.)
    - At which box does each failure type start: retrieval miss, stale article, generator going beyond the passages?
    - What does the "no answer" path show the agent, or say to the caller?
68. How does the question detector (arrow a) decide when to answer proactively and when to wait for the agent to ask? [Likely]
69. How does the query rewriter turn a noisy, multi-turn, partly mis-transcribed conversation into a good retrieval query? [Likely]
70. How would you evaluate retrieval and answers when there are no labeled query-document pairs? [Likely]
    - How far can you trust LLM-generated relevance labels?
71. Fireworks reports that Cresta moved Knowledge Assist to fine-tuned models. Where in this diagram does that change cost and latency, and how would you prove quality held? [Likely] [21][22]
72. How does the diagram change when the consumer is a voice AI agent (Diagram 2) rather than a human agent's screen? [Likely]
73. How do you enforce document-level access control so an agent never sees an article meant for another team or tenant? [Likely]

Routing and safety questions

74. Some questions need account-specific facts from a customer API rather than KB articles. Where in Diagram 5 does a router decide between retrieval and a tool call, and how is that decision evaluated? [Likely] [19][20]
75. How does Diagram 5 change when the conversation is in Spanish but most KB articles are in English? Which boxes need multilingual models? [Likely] [20]
76. A KB article contains text that tries to instruct the model ("ignore previous instructions..."). At which boxes in Diagram 5 do you defend against this indirect prompt injection? [Likely] [19]

Diagram 6. Conversation intelligence: batch and stream analytics [Likely]

Points back to: ch04 Q44 (topic discovery), Q45 (AI analyst), Q47 (emerging issues), Q48 (structured extraction), Q50 (event pipeline), Q51 (automated QA); ch07 Q38 (taxonomy), Q40 (AI Analyst), Q41 (QA scoring), Q42 (trend detector), Q44 (batch backend), Q90 (consistent extractions). Product context: [25].



Pipeline questions

77. Draw the analytics system that turns every conversation into structured data, showing which parts are streaming and which are batch. [Likely] [25]
    - Why does the emerging-issue detector (arrow 3) need the streaming path?
    - What is the freshness target for each output: seconds, minutes, hours?
78. How do you schedule batch enrichment (arrow 4) fairly across thousands of tenants while live traffic shares the same GPUs? [Likely]
    - What happens when a tenant asks to backfill a year of calls with a new extractor?
79. Every output is stamped with a model and prompt version. What do you do when the extractor model changes and last month's numbers no longer match this month's? [Likely]
    - When do you backfill, and when do you show a break in the series?
80. How do you handle late-arriving and corrected events in the analytics store (arrow 5)? [Likely]
81. Which enrichments would you run with a small fine-tuned model and which with a frontier LLM? How does the router decide? [Likely]
    - How do you attribute cost per tenant and per feature?

Topic discovery and AI analyst questions

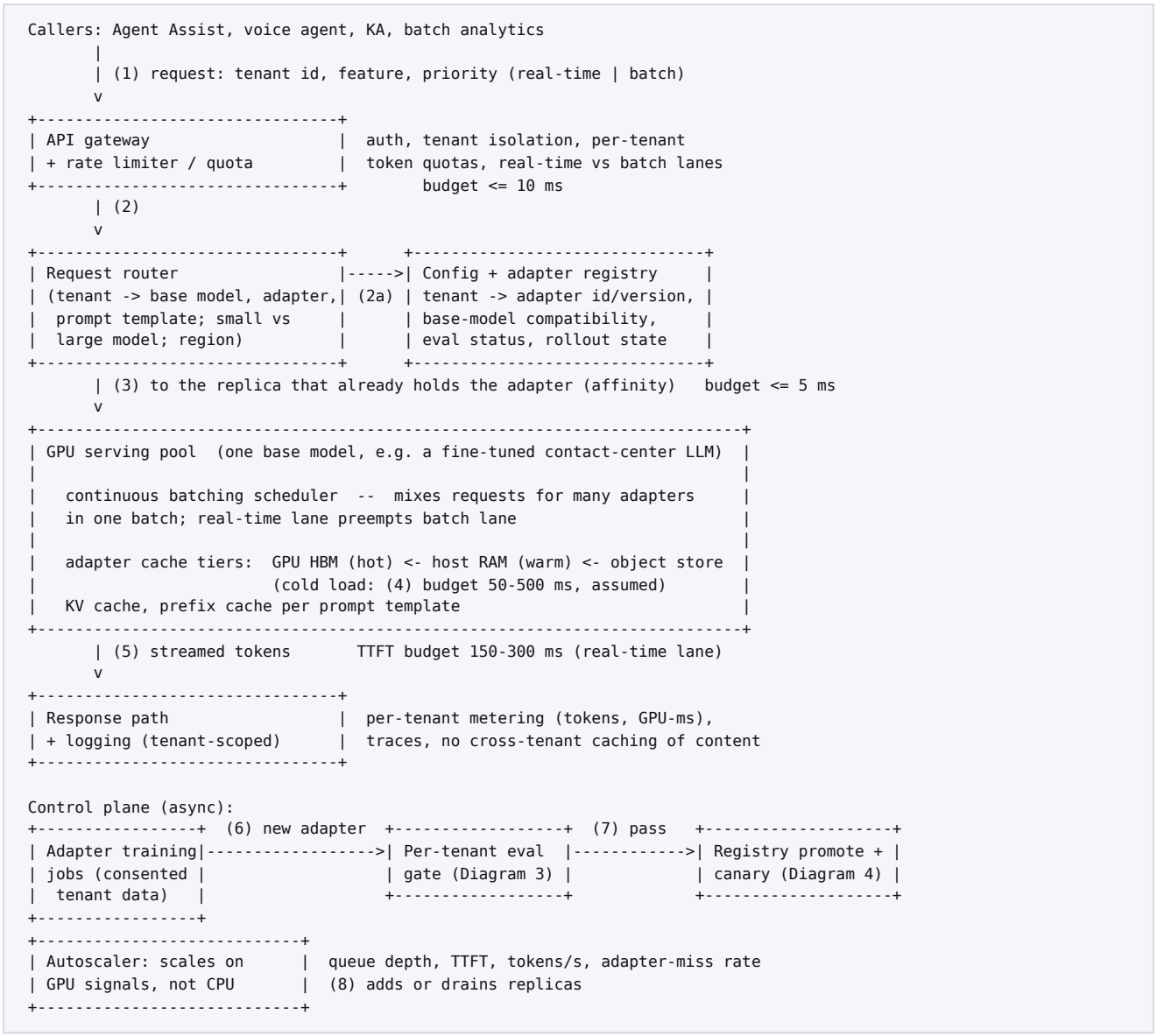
82. Draw the topic-discovery job (arrow 6): clustering, LLM labeling and a hierarchy up to three levels deep. [Likely] [25]
    - How do you keep topic ids stable when the job reruns next week?
    - How do admin edits survive re-clustering?
83. How does the AI analyst (arrows 9-10) answer "Why did cancellations rise last week?" without inventing a cause? [Likely] [25]
    - When does it query structured tables, and when does it read sampled conversations?
    - How do you show statistical confidence and link the evidence?
84. How would you evaluate automated QA scores (in the enrichment box) against human QA analysts? [Likely]
    - What agreement level would make a customer trust automatic scoring of every call?
85. Where in this diagram are PII redaction and data-retention rules enforced? [Likely]

Outcomes, custom extractors and search questions

86. Add a join between conversation data and CRM outcomes (sale closed, churn) to Diagram 6. How would you attribute outcomes to agent behaviors without confusing correlation with cause? [Likely] [25]
87. A customer admin defines a new extractor in plain language ("Did the agent offer the retention discount?"). Draw the path from that definition to scored historical calls, including validation before it runs on a full year of data. [Likely] [25]
88. Add semantic and keyword search over millions of conversations to Diagram 6. Which store serves it, and how does it respect per-tenant access control? [Likely] [25]

Diagram 7. Multi-tenant LoRA / adapter LLM serving [Likely]

Points back to: ch04 Q65 (multi-tenant serving), Q66 (p99 serving stack), Q67 (model router), Q70 (per-customer fine-tuning), Q74 (per-tenant rate limiting), Q75 (real-time vs batch GPU sharing); ch07 Q54 (per-customer adaptation), Q55 (Fireworks multi-LoRA serving), Q56 (routing), Q57 (p99 serving). Product context: [22][23][24].



Serving questions

89. Draw a serving stack where one base model serves hundreds of customer-specific LoRA adapters on shared GPUs. [Likely] [23]
    - Why not one dedicated deployment per tenant? At what tenant count does that break?
    - How does mixed-adapter batching (in the GPU pool) affect throughput and latency?
90. How does the router (arrow 3) place requests so that adapter loads stay rare, without hot-spotting one replica? [Likely]
    - What happens on a cold adapter load (arrow 4) in the middle of a live call?
91. How do you keep real-time traffic within its p99 budget when a batch backfill from one tenant floods the pool? [Likely]
    - Where do preemption, separate lanes and quotas sit in the diagram?
92. Which signal should the autoscaler (arrow 8) use, and why is CPU utilization a poor fit for GPU inference? [Likely]
    - How do you keep scale-up time acceptable when a replica must load a large base model?

93. What happens to every tenant's adapter when the base model is upgraded? [Likely] [24]
    - How does the eval gate (arrow 7) catch one tenant's silent regression?
94. Where could data leak between tenants in this diagram (prefix cache, logs, metering, training data), and how do you prevent each? [Likely]
95. How would you choose between a small fine-tuned model and a frontier API for each request in the router box? [Likely]
    - How do you prove the router does not hurt quality?
96. How would you attribute GPU cost to each tenant when requests share batches? [Likely]
97. How does this diagram change for an EU tenant whose data and model calls must stay in the EU? [Likely]
    - What may the global control plane hold, and what must stay regional?
98. Fireworks describes serving many LoRA adapters on one base-model deployment for Cresta. Which box in your diagram does that remove from Cresta's own engineering scope, and which risks remain Cresta's? [Likely] [22][23]

Decoding and failover questions

99. Where in Diagram 7 would constrained decoding (a JSON schema or grammar) run for structured extraction, and what does it cost in throughput when mixed with free-text requests in one batch? [Likely] [23]
100. Would you add speculative decoding with a draft model to Diagram 7? How does it interact with per-tenant adapters and mixed-adapter batching? [Likely] [23]
101. During a GPU-capacity outage, could the router fail over to an external model provider? Show where that path sits, and which tenants' data and contract terms would forbid it. [Likely] [22]

Cross-cutting whiteboard probes (apply to any diagram above)

102. Before drawing, what functional and non-functional requirements would you confirm, and which one shapes the diagram most? [Likely]
103. Give a back-of-envelope estimate for the diagram you drew: concurrent calls, events per second, tokens per day, GPUs. [Likely]
104. Which single box, if it failed, would hurt customers most, and how does the diagram change to protect it? [Likely]
105. Where are the tenant boundaries in your diagram, and which arrows cross them? [Likely]
106. Which arrows carry PII, and where is it redacted or encrypted? [Likely]
107. Which parts of your diagram are on the latency-critical path, and which could move to async without users noticing? [Likely]
108. Where does evaluation data come from in this diagram, and how does it flow back into training? [Likely]
109. How would the diagram change for 10 tenants compared with 1,000, and for chat compared with voice? [Likely]
110. If you had one quarter and three engineers, which boxes would you build first, which would you buy, and which would you defer? [Likely]

111. How do the seven diagrams connect: which boxes are shared platform services (trace store, model registry, eval gate, serving pool)? [Likely] - Draw the one-page map that links Diagrams 1-7.

References

1. Glassdoor, "Cresta (CA) Interview Questions" (Senior Machine Learning Engineer review: recruiter call, LeetCode-medium, vibe-coding and system design rounds; "How would you deploy a model in production?"), 2025-2026. https://www.glassdoor.co.in/Interview/Cresta-CA-Interview-Questions-E3180909.htm
2. Blind, "Cresta interview process", 2024-2025. https://www.teamblind.com/post/cresta-interview-process-jmfnzjoa
3. Blind, "Anyone interview with Cresta lately?", 2025. https://www.teamblind.com/post/anyone-interview-with-cresta-lately-zvj0uhq2
4. PracHub, "Cresta ML System Design Interview Questions" ("Design an Evaluation Platform for AI Agents Across Prompt and Model Changes"), 2026. https://prachub.com/companies/cresta/categories/ml-system-design
5. PracHub, "Cresta Software Engineer Interview Experience: System Design Round: Build an Eval Platform for AI Agents". https://prachub.com/interview-experiences/cresta-software-engineer-interview-experience-system-design-round-build-an-eval-platform-for-ai-agents
6. Glassdoor, "Cresta (CA) Interview Experience and Questions" (Forward Deployed Engineer technical screen: "What is RAG, and how do you prevent hallucination?"), 2026. https://www.glassdoor.com/Interview/Cresta-CA-Interview-Questions-E3180909.htm
7. Cresta on Greenhouse, "Senior Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4249943008
8. Cresta on Greenhouse, "Staff Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4741283008
9. Cresta on Greenhouse, "Senior Machine Learning Engineer, Voice Experience". https://job-boards.greenhouse.io/cresta/jobs/5155675008
10. Cresta on Greenhouse, "Senior Machine Learning Engineer (ASR)". https://job-boards.greenhouse.io/cresta/jobs/5101011008
11. Built In Vancouver, "Senior Machine Learning Engineer, Cresta" (Agentic Assist track). https://builtinvancouver.org/job/senior-machine-learning-engineer/8619914
12. Cresta Blog, "Understanding Cresta's Voice Platform: The Voice Stack", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-the-voice-stack
13. Cresta Blog, "Understanding Cresta's Voice Platform: ML Services, Inference Graphs and Real-Time Intelligence", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-ml-services-inference-graphs-and-real-time-intelligence
14. Cresta Blog, "Engineering for Real-Time Voice Agent Latency". https://cresta.com/blog/engineering-for-real-time-voice-agent-latency
15. Cresta Blog, "How Cresta Grades Its Agents: Evaluators in an LLM World". https://cresta.com/blog/how-cresta-grades-its-agents-evaluators-in-an-llm-world
16. Cresta Blog, "Why AI Agent Evaluations Fail, and How the Swiss-Cheese Model Prevails". https://cresta.com/blog/why-ai-agent-evaluations-fail----and-how-the-swiss-cheese-model-prevails

17. Cresta Blog, "Cresta Launches Automated AI Agent Testing: Confidence in Every Conversation". https://cresta.com/blog/cresta-launches-automated-ai-agent-testing-confidence-in-every-conversation
18. Cresta Blog, "Cresta Conductor: The Agent for AI Agent Development". https://cresta.com/blog/cresta-conductor-the-agent-for-ai-agent-development
19. Cresta Blog, "Grounding Reality: How Cresta Tackles LLM Hallucinations in Enterprise AI". https://cresta.com/blog/grounding-reality---how-cresta-tackles-llm-hallucinations-in-enterprise-ai
20. Cresta Blog, "AI-Ready Knowledge for Contact Centers: Closing the Gap Between KBs and AI", 2025. https://cresta.com/blog/ai-ready-knowledge-for-contact-centers-closing-the-gap-between-kbs-and-ai
21. Cresta Blog, "How Fine-Tuned LLMs Power Knowledge Assist, Summarization and Chat Suggestions", 2024. https://cresta.com/blog/how-fine-tuned-llms-power-knowledge-assist-summarization-and-chat-suggestions
22. Fireworks AI Blog, "Story: Cresta Knowledge Assist", 2024. https://www.fireworks.ai/blog/story-cresta-knowledge-assist
23. Fireworks AI Blog, "Multi-LoRA", 2024. https://fireworks.ai/blog/multi-lora
24. Cresta Blog, "Introducing Ocean-1: World's First Contact Center Foundation Model", 2023. https://cresta.com/blog/introducing-ocean-1-worlds-first-contact-center-foundation-model
25. Cresta Blog, "Introducing the Next Generation of Cresta Insights: A Complete System for Enterprises to Understand Their CX", 2026. https://cresta.com/blog/introducing-the-next-generation-of-cresta-insights-a-complete-system-for-enterprises-to-understand-their-cx

Source note: No page could be opened in review (DNS failures and proxy 403s on all 10 URLs tried; web-search budget used up). All 25 references are carried over from the reviewed reference lists in chapters 01, 04, 07 and 10, with the same attributions. *Changed in review:* the 1point3acres decoding thread (old reference 6) was removed because no question here cited it, and later references were renumbered. Five follow-up questions (Q35, Q42, Q53, Q54, Q56 in the new numbering) had carried [Reported] tags that their sources do not support, since the sources report only the headline prompt; they are now [Likely] and say which reported prompt they extend. 24 new [Likely] whiteboard questions were added, grouped under one new subsection per diagram (session lifecycle and audit; speech front-end and context; online evaluation and authoring loop; dependencies and change management; routing and safety; outcomes and search; decoding and failover). None is a new candidate report. [Reported] tags remain only on the headline prompts of Diagram 3 [4][5] and Diagram 4 [1] and on question 67 (RAG and hallucination, FDE screen [6]). All latency budgets on the diagrams are illustrative assumptions, except the roughly once-a-second partial transcript cadence attributed to [12] in earlier chapters.

09. Capacity Estimation and Latency Budgets: Back-of-Envelope Numbers for Real-Time Voice, LLM Serving, Knowledge Retrieval and Conversation Storage

Tagging system.

- **[Reported] [n]**: a candidate report or question bank attributes this question, or a close variant, to Cresta. [n] points to the References list.
- **[Likely]**: inferred from Cresta's products, engineering blog and ML job postings. No candidate has reported it.

How to use this chapter. Every question asks you to produce a number, a budget or a trade-off. Where a question supplies assumptions ("Assume ..."), they are inputs for you to calculate from. They are not Cresta's real figures, and they are not answers. Say your assumptions out loud, round hard (powers of ten, 1 day is about 10^5 seconds), and check each result against a sanity bound before moving on.

Diagram prompts. Some questions include a "Sketch" follow-up. It names the latency timeline or capacity diagram to draw on the whiteboard, without giving the numbers.

Research limits. In the October 2026 review pass, every cited domain was again unreachable from this environment (DNS `ENOTFOUND` for cresta.com, fireworks.ai and prachub.com in WebFetch; the egress proxy answered 403 for cresta.com, prachub.com and job-boards.greenhouse.io), and the shared web-search budget was already used up. No page could be opened and no new candidate report could be found, so the 2024-2026 search for new reported questions came back empty. Instead, each [Reported] tag was checked against the excerpt log in chapter 10 (Part 2, tables R1-R7 and C1-C6). Both original [Reported] items match a logged excerpt and keep their tags. The new [Reported] items in section A scale up prompts logged there (a rate limiter, a worker pool, a job dispatcher, RAG and decoding). In every case the base prompt is reported and the sizing probes are inferred. No candidate has reported a capacity-estimation question for Cresta.

Format notes for this round

- **No separate estimation round is reported.** Capacity and latency estimates are follow-ups inside the roughly one-hour system design round. A 2025-26 Senior MLE report lists that round after a recruiter call, an LC-medium round and a "vibe coding"

round [1]. Senior/Staff reports describe one hour of system design next to one hour of coding [2].

- **Reported design prompts that invite sizing.** "How would you deploy a model in production?" (Senior MLE) [1], and "Design an evaluation platform for AI agents across prompt and model changes" [4][5]. Both lead naturally to fleet-size, throughput and cost-per-run questions. The PracHub experience post for the second prompt is labeled as a SWE system design round, not an MLE one [5].
- **Reported coding and screen prompts that a design round can scale up.** A rate limiter and a worker pool with round-robin assignment [1], a job dispatcher and executor (SWE, 2026, title only) [24], calling a public API [25], "What is RAG, and how do you prevent hallucination?" (FDE screen) [23], and step-by-step sequence generation (MLE, AI Agent) [22]. Section A turns each into a sizing exercise. The base prompts are reported, and the numeric follow-ups are inferred.
- **Interviewer (inferred).** Most likely a senior or staff MLE from Agentic Assist, AI Agent, Voice/ASR or Insights [6][7][8][9][10].
- **Senior bar (from postings).** Postings ask candidates to design for latency, cost efficiency, scalability, security and maintainability, and the Voice posting names real-time inference and streaming pipelines [6][7][8]. Postings also list latency and cost next to accuracy and faithfulness as quality metrics [7]. Recruiters have flagged LLM, RAG and agent fundamentals as in scope, so expect serving-cost and latency questions about LLMs specifically [3]. At senior level, expect to be pushed past "it depends" to a concrete number, a stated bottleneck and the lever you would pull first.
- **Product facts you can anchor on.** Cresta publishes both partial and final transcripts, about once per second [12]. The voice stack includes audio and transcript redaction [12]. An inference graph runs several models per conversation [13]. Cresta moved Knowledge Assist, summarization and chat suggestions to fine-tuned models served with multi-LoRA [14][15][16].

Questions

A. Reported prompts, with sizing probes

1. How would you deploy a model in production? [Reported] [1]
    - For a real-time assist model, what is the peak QPS you would size for, and how did you derive it from concurrent conversations?
    - How many GPU replicas do you need at peak, and how much headroom do you keep for canary plus rollback running side by side?
    - What does shadow traffic cost in GPU-hours if you mirror 100 percent of traffic for one week?
    - Which latency percentile do you put in the rollout gate, and how many requests do you need before p95 or p99 is stable?
2. Design an evaluation platform for AI agents that catches regressions across prompt and model changes. [Reported] [4][5]
    - Assume 2,000 scenarios, 5 reruns each, 12 simulated turns per scenario and 3 judge calls per turn. How many LLM calls does one full run make, and how many tokens?

    - At your assumed price per million tokens, what does one run cost? How many runs per day can the team afford?
    - How long does a run take at a given rate limit or GPU budget? What parallelism do you need to finish under 30 minutes for a CI gate?
    - Which cut (fewer reruns, smaller judge, sampled scenarios, cached unchanged turns) saves the most cost for the least loss in detection power?
    - For voice-agent scenarios that run simulated audio through TTS, then ASR, then the agent, how much do cost and wall-clock time per scenario grow compared with text-only simulation [21]?
3. Implement a rate limiter, then scale it to guard calls from real-time features to a shared LLM fleet or a third-party API. [Reported] [1] (coding round; sizing probes inferred)
    - With 500 tenants and 10 features per tenant, each with its own token bucket, how much state does the limiter hold, and can it live in one Redis shard?
    - At 50,000 limiter checks per second at peak, what does a network round-trip per check add to hint latency? When would you switch to local buckets with periodic sync?
    - Do you limit on requests per second or on tokens per minute for an LLM backend, and why does the choice matter for long prompts?
4. Implement a worker pool with an orchestrator that assigns tasks round-robin, then use it for post-call jobs. [Reported] [1] (FDE AI Agent coding round; sizing probes inferred)
    - Assume 1 million calls a day, and post-call jobs (summary, QA, tagging) that take 2-20 seconds of GPU time each. How many workers do you need at steady state and at the evening peak?
    - Why does round-robin break down when job durations vary 10x, and how much does least-loaded or work-stealing assignment cut p95 completion time?
5. Design a job dispatcher and executor, then size it for re-scoring every conversation of a tenant after a scorecard change. [Reported] [24] (SWE phone-screen title, 2026; sizing probes inferred)
    - A tenant with 50 million historical conversations changes one QA criterion. How long does a full backfill take on your batch fleet, and how do you keep it from starving daily jobs?
    - What checkpointing granularity bounds the work lost when an executor dies mid-batch?
6. What is RAG, and how do you prevent hallucination? Then put numbers on the RAG step inside a live call. [Reported] [23] (FDE technical screen, 2026; numeric probes inferred)
    - How many retrieved chunks of 300 tokens can you add to a hint prompt before prefill pushes p95 past 300 ms?
    - What does one extra groundedness-check model call add in latency and cost per hint, and would you run it on every hint or on a sample?
7. Walk through how an AI agent generates a sequence step by step, then estimate per-token decode latency from first principles. [Reported] [22] (MLE AI Agent phone-screen follow-up; numeric probes inferred)
    - For an 8B model in 16-bit weights on a GPU with about 3 TB/s of memory bandwidth, what is the lower bound on time per output token at batch 1? How far above that bound does a real server run?

    - How does that bound change at batch 32, and at what batch size does decode stop being memory-bound?
8. Call a public API and process the results, then size the integration for a voice agent's tool calls. [Reported] [25] (SWE phone screen; sizing probes inferred)
    - If 2,000 concurrent AI Agent calls each make one CRM lookup every 30 seconds, what QPS does the customer's backend see, and what happens when its rate limit is 20 QPS?
    - How large a response cache, with what TTL, would keep you under that limit without serving stale account data?

B. Traffic model: tenants, concurrency and peaks

9. Estimate the peak number of concurrent calls for one large enterprise contact-center tenant. [Likely]
    - Assume 5,000 agents, 70 percent occupancy at peak, and an average handle time of 6 minutes. How many calls are live at the peak hour?
    - How many new calls per second start at peak? How did you use Little's Law?
    - What is the peak-to-average ratio across a day, and across a week (Monday morning, billing cycle, outage day)?
10. Estimate the platform-wide peak concurrency across all tenants. [Likely]
    - Assume a few hundred tenants with a long-tail size distribution. How much do time zones flatten the global peak?
    - Which tenants dominate, and how does that change capacity planning compared with a uniform distribution?
11. How does concurrency change when the AI Agent (autonomous voice agent) handles calls instead of human agents? [Likely] [11]
    - Without an agent-headcount ceiling, what limits concurrency: trunk capacity, the customer's queue or your GPU fleet?
    - How would you size for a marketing campaign that triples inbound volume for two hours?
12. Estimate chat (digital) concurrency for the same tenant when one human agent handles three chats at once. [Likely]
    - How does the request pattern for suggestions differ between chat (one per message) and voice (one per partial)?
13. Convert concurrency into model QPS for each feature in the inference graph. [Likely] [13]
    - If partial transcripts arrive about once per second per call [12], what is the raw trigger rate for 10,000 concurrent calls?
    - Which models run on every partial, which on every final utterance, and which once per call? What QPS does each tier see?
    - How much does debouncing (at most one call per conversation every X ms) cut QPS, and what latency does it add?
14. How would you build a capacity model the team can update each quarter? [Likely]
    - Which inputs (agents, occupancy, handle time, features enabled per tenant) drive the GPU forecast?
    - How do you check the model against real telemetry after launch?
15. Estimate the start-of-shift burst for a tenant whose 3,000 agents log in within 10 minutes. [Likely]

    - How many new streaming sessions per second does that create (WebSocket connects, ASR session setup, adapter and prompt-cache warm-up), and which component fails first?
16. How many hints per minute can a human agent actually absorb, and how does that cap the useful QPS of the hint service? [Likely] [13]
    - If agents act on at most 3-4 hints per minute, how much of the trigger rate from question 13 is wasted compute, and where would you filter it?

C. Audio ingest and streaming ASR

17. Estimate the audio frames per second entering streaming ASR at platform peak. [Likely] [12]
    - Assume telephony audio at 8 kHz, 16-bit PCM after decoding, 20 ms frames, and two channels (agent and customer) per call. How many frames per second per call, and in total for 10,000 calls?
    - What is the ingest bandwidth in Mbit/s, before and after compression (G.711 at 64 kbit/s per channel compared with Opus)?
    - Does resampling to 16 kHz for the ASR model change your network numbers or only your compute numbers?
18. How many audio streams can one GPU serve with a streaming ASR model? [Likely] [9]
    - Assume a real-time factor of 0.02 per stream at batch 1. Why does that not mean 50 streams per GPU in practice?
    - How does chunked (cache-aware) streaming change the batching math compared with offline transcription?
    - How do you size the ASR fleet for 20,000 concurrent channels with N+1 redundancy per zone?
19. Do you transcribe both channels, or only the customer channel for some features? [Likely]
    - What fraction of the ASR fleet does that save, and which downstream features break?
20. What is the cost per call-minute of streaming ASR on your own GPUs compared with a third-party API? [Likely] [9]
    - At what monthly volume does self-hosting break even, once you count on-call and model-maintenance engineers?
21. Estimate the ASR finalization delay: from the end of speech to a final transcript. [Likely] [11][12]
    - How do the voice-activity-detection silence threshold, the chunk size and the right-context lookahead each add to it?
    - What happens to word error rate if you shorten the endpointing silence from 700 ms to 300 ms? How would you measure it?
22. How much memory does a streaming ASR session hold (encoder cache, decoder state, audio buffer)? [Likely]
    - How many sessions fit in one GPU's memory, and is memory or compute the binding limit?
23. How would you handle a reconnect or a pod restart mid-call without losing more than one second of transcript? [Likely] [12]
    - How much audio do you buffer upstream, and what does that buffer cost in memory at peak?

24. Size the transcript event bus between ASR and the downstream models. [Likely] [12]
    - At 20,000 channels emitting a partial about once per second plus finals, how many messages and MB per second does the bus carry? How many partitions do you need, and what do you key them on to keep each call in order?
    - What retention do you set so a downstream model can replay the last 60 seconds after a crash?
25. What does adding diarization to single-channel (mono) recordings cost, compared with stereo telephony where each speaker has a channel? [Likely] [9]
    - How much extra GPU time per call-minute, and what diarization error rate would make QA scoring unreliable?
26. How do the media-server jitter buffer and codec framing add to latency before audio even reaches ASR? [Likely] [11]
    - Put a number on each (packetization, jitter buffer, transcoding), and say which ones you can shrink without more packet-loss artifacts.

D. LLM hint generation for real-time Agent Assist

27. Size the GPU fleet for LLM hint generation at p95 under about 300 ms. [Likely] [13] [14]
    - Assume 10,000 concurrent calls, one hint request per final customer utterance (about one every 8 seconds per call), 1,500 prompt tokens and 40 output tokens. What is the peak request rate and token rate?
    - What prefill and decode throughput per GPU do you assume for a 7-8B model, and where does each number come from?
    - How many GPUs does that give you at 50 percent target utilization? Why not 90 percent?
    - What batch size keeps p95 under 300 ms, and how does that batch size cap per-GPU throughput?
28. Break the 300 ms hint budget into stages. [Likely] [13]
    - Sketch: a timeline from "final utterance emitted" through the transcript bus, trigger/moment detection, retrieval, prompt assembly, queue wait, prefill, decode, post-filter and UI push. Which stages are fixed and which grow with load?
    - How much of the budget does queueing take at 50 percent compared with 80 percent utilization?
29. How does model size change the GPU count for the same SLO? [Likely] [14][19]
    - Compare a 70B general model, a fine-tuned 7-8B model and a distilled 1-3B model on GPUs needed, p95 and quality risk.
    - What evidence would convince you a smaller fine-tuned model is good enough? (Cresta reports fine-tuned models matching a larger general model on its tasks [14][15].)
30. How many per-tenant LoRA adapters can one GPU hold, and what does multi-LoRA serving do to throughput? [Likely] [16]
    - What is the memory footprint of one rank-16 adapter on a 7-8B model compared with the base weights?
    - What happens to p95 when a request needs an adapter that is not loaded? How often can you afford that?
    - When is a per-tenant full fine-tune worth the extra GPUs?

31. Would you trigger hint generation on partial transcripts to save latency? [Likely] [12]
    - How many extra LLM calls does that create, and how many of them are wasted when the partial changes?
    - What hit rate on "partial equals final" would make speculative triggering pay for itself?
32. How do you set the autoscaling policy for the hint service? [Likely]
    - What metric do you scale on (queue depth, in-flight tokens, p95), and why not GPU utilization?
    - How long does a new GPU replica take to become ready (node, image, weights load), and how much warm headroom does that force you to keep?
33. What is the cost per agent-hour of real-time assist? [Likely]
    - Using your GPU count and an assumed GPU-hour price, what does one live agent cost per hour, and is that defensible against a contact-center seat price?
34. How would you degrade gracefully when GPU capacity is short during a spike? [Likely]
    - Which features do you shed first, and what does each shed save in GPU-seconds?
    - Would you serve a smaller fallback model, lengthen the debounce, or drop hints for low-value moments?
35. Estimate prefill time for a 1,500-token hint prompt from FLOPs. [Likely]
    - Using about 2 x parameters x tokens FLOPs, an 8B model, and a GPU at an assumed peak of about 1,000 dense TFLOPS with 40 percent utilization, how long does prefill take? How does that compare with your 300 ms budget?
    - At what prompt length does prefill, not decode, dominate the hint latency?
36. Apply Little's Law to the hint service: how many requests are in flight at peak, and how much KV-cache memory do they need? [Likely]
    - With your request rate from question 27 and a mean service time of 200 ms, how many sequences are in flight across the fleet, and how many per GPU?
37. How long does a new LLM replica take to become ready, and what warm pool does that force? [Likely]
    - Assume 16 GB of weights pulled from object storage at 1-2 GB/s, plus container start and CUDA graph capture. How many minutes until the replica serves traffic, and how much headroom covers a 3x spike that arrives faster than that?

E. Voice-agent turn latency (AI Agent)

38. Draw the latency breakdown of one voice-agent turn, from the customer's last word to the first audio the customer hears. [Likely] [11]
    - Sketch: endpointing/ASR finalization, network hop, orchestration, retrieval or tool call, LLM time to first token, the first sentence chunk, TTS time to first audio, and playout through telephony. Put a number on each.
    - What total target would you set (for example under 800 ms at p50 and under 1.5 s at p95), and why those percentiles?
39. Which single stage would you attack first to cut 200 ms, and how? [Likely] [11]
    - What does each option cost in quality: a shorter endpointing threshold, a smaller LLM, speculative decoding, streaming TTS on partial sentences?
40. How much LLM time to first token can you afford, and what drives it? [Likely]

    - How does TTFT scale with prompt length on a 7-8B model, and with queue depth?
    - How much does prefix caching of the system prompt, tool schemas and conversation history save per turn?
41. How does speculative decoding change latency and GPU count for the voice agent? [Likely]
    - What draft-model acceptance rate do you need for a real speed-up, and how does it vary with temperature and domain?
    - Does speculative decoding help TTFT, time per output token, or both? Why does that matter for voice when TTS starts on the first sentence?
    - With per-token acceptance rate a and draft length k, the expected tokens per target step is (1 - a^(k+1)) / (1 - a). Compute it for a = 0.6 and 0.8 at k = 4, then subtract the draft model's cost. Is it still a win at batch 32?
42. How do you start TTS before the LLM finishes? [Likely] [11]
    - At what chunk boundary (clause, sentence, N tokens) do you hand text to TTS, and what does a bad boundary do to prosody?
    - How many output tokens per second must the LLM sustain so TTS never starves?
43. What does a tool call (for example a CRM lookup) do to the turn budget? [Likely] [11]
    - If the backend API has a p95 of 600 ms, how do you hide it: filler speech, parallel prefetch, or predicting the call from the partial transcript?
    - How many speculative prefetches per call can you afford, and what do wasted ones cost?
44. How do you handle barge-in (the customer interrupts) within the budget? [Likely]
    - How fast must you detect speech and stop TTS playout, and which component owns that timer?
    - What happens to in-flight LLM tokens and the KV cache when the turn is cancelled?
45. Estimate GPU and TTS capacity for 2,000 concurrent voice-agent calls. [Likely]
    - Assume the agent speaks 40 percent of the time and each LLM turn uses 3,000 prompt and 60 output tokens every 10 seconds. What are the token rates and GPU counts?
    - How many TTS streams fit per GPU at real-time factor r, and does TTS or the LLM dominate cost?
46. How do you measure end-to-end turn latency on real calls? [Likely] [8][11]
    - Server timestamps compared with measuring silence gaps in the stereo recording: which do you trust, and why do they disagree?
    - Which percentile do customers feel, and how do you report it per tenant?
47. How do region and network topology affect the budget? [Likely]
    - What round-trip time do you add if the telephony edge, ASR and LLM sit in different regions? When would you co-locate them?
48. A safety or policy guardrail check must run on each voice-agent response. Where does it go in the turn budget? [Likely] [20]
    - Serial check on the full response, streaming check per sentence, or parallel check with the ability to cut TTS: what does each add to time-to-first-audio, and what does each risk?
49. How does prompt length grow over a 30-minute voice-agent call, and what does that do to TTFT and cost per turn? [Likely]

    - Assume 150 turns of about 40 tokens each on top of a 2,500-token system prompt. What is the prompt at the last turn, and the total prefill tokens over the call with and without prefix caching?
    - At what length would you compact history into a running summary, and what does compaction cost?
50. Would you cache synthesized audio for common agent phrases (greetings, disclosures, hold messages)? [Likely] [11]
    - What hit rate is realistic, how much storage does a per-tenant, per-voice cache need, and how many ms does a hit save?
51. Estimate the cost per resolved conversation for the AI Agent, compared with a human agent. [Likely] [11]
    - Add ASR, LLM, TTS and telephony per minute, then divide by the containment rate. At what containment rate does automation stop being cheaper than an assumed fully loaded human cost per hour?

F. Token cost: summarization, QA scoring and insights

52. Estimate the token cost per conversation for post-call summarization. [Likely] [14]
    - Assume a 6-minute call at about 150 spoken words per minute per active speaker and about 1.3 tokens per word. How many transcript tokens is that?
    - Add the system prompt and output. What is the cost per call at a self-hosted cost per million tokens compared with a frontier API price?
    - Multiply out to 1 million calls per day. What is the monthly bill under each option?
53. Estimate the token cost of automated QA scoring with an LLM judge. [Likely] [20]
    - Assume a scorecard with 25 criteria. Do you make one call per criterion or one call for all criteria? How many tokens does each design use per call?
    - How does prompt caching of the shared transcript prefix change the per-criterion design's cost?
    - What accuracy difference would justify the more expensive design?
54. How would you cut QA scoring cost by 10x without losing auditor trust? [Likely] [20]
    - Which criteria can a classifier or rule handle, and which need an LLM?
    - What fraction of calls do you route to the large model, and how do you pick them?
55. Estimate the cost of topic and driver discovery (Insights) across a month of calls. [Likely] [18]
    - Embedding every utterance compared with summarizing every call and then clustering: what are the token and compute costs of each?
    - How often must you recompute, and can you do it incrementally?
56. Should batch post-call work run on the same GPUs as real-time inference? [Likely]
    - How many GPU-hours per day does off-peak batch work need, and how much of it fits into the real-time fleet's night-time trough?
    - How do you keep a batch backlog from hurting real-time p95?
57. What is your total LLM cost per conversation across all features (assist, summary, QA, insights)? [Likely]
    - Which feature dominates, and is that what the business would expect?
    - How would you show cost per tenant so pricing and contracts reflect it?

    - For which features does output length dominate cost, and for which does input length? How do you cap summary length without losing required fields?
58. How do token counts and costs change for non-English calls? [Likely]
    - If Spanish or Japanese transcripts take 1.3-2x more tokens per word than English with your tokenizer, what happens to cost per call, context limits and prefill latency? Would you change the tokenizer or the model?
59. How many human labels do you need to validate an LLM QA judge, and what does that cost in annotator hours? [Likely] [20]
    - To estimate judge-human agreement to within plus or minus 3 points per criterion at 95 percent confidence, how many calls do you label? Multiply by criteria and minutes per call to get the labeling budget.
60. How would you budget tokens for the evaluation of a new summarization model before launch? [Likely] [20][21]
    - How many held-out calls and how many judge calls per call do you need for a confident comparison?

G. Knowledge base and vector index sizing

61. Estimate the vector-index size for one tenant's knowledge base. [Likely] [15][17]
    - Assume 20,000 articles of about 1,500 words each, chunked at 300 tokens with 15 percent overlap. How many chunks?
    - At 768 or 1,024 dimensions in float32, what is the raw vector size? Add HNSW graph overhead and metadata. What is the total?
    - How does that change with int8 or binary quantization, and what recall loss do you accept?
62. Estimate the index size for all tenants, and decide between one shared index with tenant filters and one index per tenant. [Likely] [17]
    - How do noisy-neighbor latency, isolation guarantees and per-tenant deletion requests affect the choice?
    - How much memory does the whole fleet need if every index is kept in RAM?
63. What retrieval latency can Knowledge Assist afford inside the real-time budget? [Likely] [15]
    - Split it into query embedding, approximate nearest-neighbor search, re-ranking and fetching the text. Which step is the largest, and how do you cut it?
    - How many candidates can a cross-encoder re-rank within 50 ms on one GPU?
64. Estimate the cost of re-embedding all knowledge bases when you change embedding models. [Likely]
    - How many tokens, how many GPU-hours, and how do you run it without downtime (dual indexes, backfill, cut-over)?
65. How fresh must the index be, and what update rate must ingestion sustain? [Likely] [17]
    - If a tenant edits 500 articles a day, what is the embedding load, and how quickly must a change be searchable?
66. Would you index past conversations (resolved call transcripts) as well as articles? [Likely]
    - How does that change index size by orders of magnitude, and what retention or PII limits apply?

67. What load does real-time query embedding put on the embedding service? [Likely] [15]
    - If Knowledge Assist embeds a query on each detected customer question (about one every 30 seconds per call) at 10,000 concurrent calls, what QPS and how many GPUs or CPU cores do you need for a 50 ms p95?

H. Storage, retention and PII redaction

68. Estimate transcript storage per year for the platform. [Likely] [12]
    - Assume 1 million calls a day, 6 minutes each, about 1,000 words per call, and word-level timestamps and speaker labels stored as JSON. What is the size per call and per year, raw and compressed?
    - How much do partial-transcript revisions add if you keep them, and should you?
69. Estimate audio storage per year if recordings are kept. [Likely]
    - At 64 kbit/s per channel for two channels, what is the size per call-minute, and per year at the volume above?
    - How do compression choice and retention period (30 days, 1 year, 7 years) move the total?
70. Where in the pipeline does PII redaction run, and what latency does it add? [Likely] [12]
    - For real-time redaction of a token stream, how much lookahead do you need to catch a card number split across partials, and what does that lookahead add to hint latency?
    - What throughput must the redaction model sustain at platform peak, in tokens per second?
71. Do you store both redacted and unredacted transcripts? [Likely]
    - How does each choice multiply storage, encryption-key management and access-audit volume?
    - How do you handle a customer's deletion request across transcripts, audio, embeddings and training sets?
72. Estimate the storage and compute cost of audio redaction (bleeping or silencing PII spans). [Likely] [12]
    - How do you align ASR word timestamps to audio, and what timing error forces you to pad each redacted span?
73. Size the event and analytics store that powers dashboards over conversations. [Likely] [18]
    - How many rows per call (utterances, model outputs, scorecard results), and how many rows per day?
    - Which queries must be interactive, and how does that drive partitioning and pre-aggregation?
74. What does data residency (EU, US, regulated industries) do to your capacity plan? [Likely]
    - How much extra headroom do you hold when each region must absorb its own peak alone?
75. Size the keyword-search index over one year of transcripts for Insights. [Likely] [18]
    - How large is an inverted index over the transcript volume from question 68 compared with the raw text? How does adding a vector index per utterance change the total?

76. Estimate the tracing and logging volume of the real-time pipeline. [Likely]
    - With about 15 spans per hint request and 1 KB per span, how many GB per day at peak QPS? What sampling rate keeps the bill sane while still giving p99 per tenant?

I. Serving trade-offs that move the numbers

77. Walk through how continuous batching changes throughput and latency compared with static batching. [Likely]
    - At what point does adding requests to the batch raise p95 more than it saves cost?
78. When do you separate prefill and decode onto different GPUs? [Likely]
    - For a long-prompt, short-output workload like hint generation, which phase dominates, and does disaggregation pay off?
79. What does weight quantization (FP8, INT8, INT4) buy for each workload? [Likely]
    - How do you measure the quality cost on Cresta-like tasks, not on public benchmarks?
    - Does quantization help a memory-bound decode or a compute-bound prefill more?
80. How do you size the KV cache for long conversations? [Likely]
    - Assume a 7-8B model with grouped-query attention. What is the KV cache per token and per 8,000-token conversation? How many concurrent sequences fit on one 80 GB GPU?
    - When do you truncate history, summarize it, or offload KV to CPU memory?
81. Where does caching help, and what is the hit rate? [Likely]
    - Prefix (prompt) caching, semantic response caching for repeated knowledge questions, and retrieval caching: which is safe for which feature?
    - How do you prevent a cache from serving one tenant's content to another?
82. Distillation: how small can the hint or QA model get before it fails the quality bar? [Likely] [14][19]
    - How many teacher-labelled examples do you need, and what does generating them cost in tokens?
83. Compare a self-hosted fleet with a provider API for each workload. [Likely] [15]
    - Which workloads move to self-hosting first, given latency SLOs, steady volume and data-control needs?
    - How do provider rate limits and tail latency enter your capacity plan when you use an API?
84. What GPU type would you choose for each workload (real-time hints, voice agent, batch QA, embeddings)? [Likely]
    - How do memory size, memory bandwidth and price per hour change the answer?
85. How do you trade beam width against latency and memory in a real-time decoder? [Likely] [22]
    - What does beam width 4 do to KV-cache memory and per-step latency compared with greedy decoding?
86. When does a non-LLM model (classifier, small encoder) beat an LLM on the cost-latency-quality frontier for a real-time trigger? [Likely] [13]

- How many classifier calls cost the same as one LLM call, and does that let you run on every partial?

87. Estimate the GPU-hours to fine-tune one tenant-specific LoRA adapter, and how many tenants the training cluster can serve per week. [Likely] [14][16]
    - Assume 50,000 conversations of 2,000 tokens, 2 epochs, an 8B base model and about 6 x parameters x tokens training FLOPs. How many GPU-hours is one run, and how many runs fit in a 64-GPU cluster per week?
88. Estimate the token and GPU budget for continued pretraining of a contact-center foundation model on transcripts. [Likely] [19]
    - How many tokens does a corpus of 100 million conversations give you after deduplication and PII redaction? How many GPU-days for one epoch on a 7-8B model, and how would you decide whether more epochs pay off?

J. Reliability, headroom and SLO math

89. Set SLOs for each real-time feature, and derive the error budget. [Likely]
    - How do a 99.9 percent and a 99.95 percent monthly availability target translate into minutes of allowed downtime?
    - Is a hint that arrives after 2 seconds an error under the SLO?
90. How much redundancy do you need to survive the loss of a zone at peak? [Likely]
    - With three zones, what utilization must each run at in normal times so two zones can carry the peak?
91. How do you combine per-stage p95s into an end-to-end p95? [Likely]
    - Why is the sum of per-stage p95s a poor estimate, and how would you estimate the real tail?
    - How does fan-out to several models in the inference graph raise the tail [13]?
92. How do you load-test the voice pipeline before a large tenant goes live? [Likely] [21]
    - How do you generate realistic audio traffic at 2x the expected peak, and what do you watch first?
93. What does a retry policy cost in capacity during an incident? [Likely]
    - How do you stop retries from turning a slowdown into an overload (retry budgets, back-off, load shedding)?
94. How do you schedule maintenance and model rollouts when global traffic never truly reaches zero? [Likely]
    - How much spare capacity does a rolling restart of the LLM fleet need?
95. How fast can you move 10,000 live calls out of a failing region? [Likely] [12]
    - How much per-call state (ASR decoder state, conversation history, pending hints) must move, and can a call survive re-establishing ASR from scratch? What gap in transcript do you accept?

K. Applying the estimates in a full design

96. Design Agent Assist for a new 5,000-agent tenant, and present a one-page capacity and cost plan. [Likely] [13][14]
    - Sketch: per-stage throughput, replicas per stage and the end-to-end latency budget on one diagram.
    - Which number in your plan are you least sure of, and how would you measure it in the first week?

97. Design the AI Agent for a tenant moving 30 percent of inbound calls from humans to automation. [Likely] [11]
    - How do GPU, TTS and telephony capacity change, and what is the cost per automated minute compared with a human agent minute?
98. Design Knowledge Assist for a tenant with 100,000 articles in 12 languages. [Likely] [15][17]
    - How do index size, embedding cost and retrieval latency grow, and would you use one multilingual index or one per language?
99. Design post-call summarization and QA scoring for 1 million calls per day with a 15-minute freshness target. [Likely] [14][20]
    - What is the steady-state and catch-up throughput you need, and how large a backlog can you clear after a two-hour outage?
100. Design Insights for a tenant who wants to ask open questions over two years of calls. [Likely] [18] - Which pre-computation (summaries, embeddings, tags) do you pay for up front, and which per query?
101. The CFO asks you to cut inference spend by 40 percent without hurting customer-visible quality. What is your plan? [Likely] - Rank your levers by savings and by risk, and say how you would prove quality held.
102. An enterprise customer asks for a contractual p95 of 200 ms for hints. What do you need to change, and what would you price differently? [Likely] - Which stages must shrink, and would you dedicate GPUs to that tenant?
103. You doubled the context window to improve hint quality, and p95 rose by 40 percent. How do you find out why and fix it? [Likely] - Which metrics separate queueing, prefill and decode growth?
104. Explain to a product manager why a 2 percent quality gain from a larger model is or is not worth 3x the serving cost. [Likely] [7] - How do you express the cost in their units (per seat, per call, per month)?

References

1. Glassdoor, "Cresta (CA) Interview Questions" (Senior Machine Learning Engineer review: recruiter call, LeetCode-medium, vibe-coding and system design rounds; "How would you deploy a model in production?"; rate-limiter coding question; FDE AI Agent worker-pool and round-robin orchestrator question), 2024-2026. https://www.glassdoor.co.in/Interview/Cresta-CA-Interview-Questions-E3180909.htm

2. Blind, "Cresta interview process" (one hour of system design next to one hour of coding), 2024-2025. https://www.teamblind.com/post/cresta-interview-process-jmfnzjoa

3. Blind, "Anyone interview with Cresta lately?" (recruiter flagged LLM, GenAI, RAG and AI-agent fundamentals), 2025. https://www.teamblind.com/post/anyone-interview-with-cresta-lately-zvj0uhq2

4. PracHub, "Cresta ML System Design Interview Questions" ("Design an Evaluation Platform for AI Agents Across Prompt and Model Changes"), 2026. https://prachub.com/companies/cresta/categories/ml-system-design

5. PracHub, "Cresta Software Engineer Interview Experience: System Design Round: Build an Eval Platform for AI Agents". https://prachub.com/interview-experiences/cresta-software-engineer-interview-experience-system-design-round-build-an-eval-platform-for-ai-agents

6. Cresta on Greenhouse, "Senior Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4249943008

7. Cresta on Greenhouse, "Staff Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4741283008

8. Cresta on Greenhouse, "Senior Machine Learning Engineer, Voice Experience". https://job-boards.greenhouse.io/cresta/jobs/5155675008

9. Cresta on Greenhouse, "Senior Machine Learning Engineer (ASR)". https://job-boards.greenhouse.io/cresta/jobs/5101011008

10. Built In Vancouver, "Senior Machine Learning Engineer, Cresta" (Agentic Assist track). https://builtinvancouver.org/job/senior-machine-learning-engineer/8619914

11. Cresta Blog, "Engineering for Real-Time Voice Agent Latency". https://cresta.com/blog/engineering-for-real-time-voice-agent-latency

12. Cresta Blog, "Understanding Cresta's Voice Platform: The Voice Stack", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-the-voice-stack

13. Cresta Blog, "Understanding Cresta's Voice Platform: ML Services, Inference Graphs and Real-Time Intelligence", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-ml-services-inference-graphs-and-real-time-intelligence

14. Cresta Blog, "How Fine-Tuned LLMs Power Knowledge Assist, Summarization and Chat Suggestions", 2024. https://cresta.com/blog/how-fine-tuned-llms-power-knowledge-assist-summarization-and-chat-suggestions

15. Fireworks AI Blog, "Story: Cresta Knowledge Assist", 2024. https://www.fireworks.ai/blog/story-cresta-knowledge-assist

16. Fireworks AI Blog, "Multi-LoRA", 2024. https://fireworks.ai/blog/multi-lora

17. Cresta Blog, "AI-Ready Knowledge for Contact Centers: Closing the Gap Between KBs and AI", 2025. https://cresta.com/blog/ai-ready-knowledge-for-contact-centers-closing-the-gap-between-kbs-and-ai

18. Cresta Blog, "Introducing the Next Generation of Cresta Insights", 2026. https://cresta.com/blog/introducing-the-next-generation-of-cresta-insights-a-complete-system-for-enterprises-to-understand-their-cx

19. Cresta Blog, "Introducing Ocean-1: World's First Contact Center Foundation Model", 2023. https://cresta.com/blog/introducing-ocean-1-worlds-first-contact-center-foundation-model

20. Cresta Blog, "How Cresta Grades Its Agents: Evaluators in an LLM World". https://cresta.com/blog/how-cresta-grades-its-agents-evaluators-in-an-llm-world

21. Cresta Blog, "Cresta Launches Automated AI Agent Testing: Confidence in Every Conversation". https://cresta.com/blog/cresta-launches-automated-ai-agent-testing-confidence-in-every-conversation

22. 1point3acres, "Cresta Machine Learning Engineer Tech Phone Interview Experience" (MLE, AI Agent direction; greedy and beam-search decoding), 2025. https://www.1point3acres.com/interview/thread/1140481

23. Glassdoor, "Cresta (CA) Interview Experience and Questions" (Forward Deployed Engineer technical screen: "What is RAG, and how do you prevent hallucination?"), 2026. https://www.glassdoor.com/Interview/Cresta-CA-Interview-Questions-E3180909.htm

24. 1point3acres, "Cresta Fulltime Software Engineer Tech Phone Screen: Job Dispatcher-Executor System Implementation" (title seen in a listing; full text not read), 2026. https://www.1point3acres.com/interview/thread/1172433

25. 1point3acres, "cresta interview questions" (company page; SWE and MLE phone screens, including calling a public API such as Wikipedia and processing the results), 2024-2026. https://www.1point3acres.com/interview/company/cresta

Source note: No page could be opened in either research pass. In the October 2026 review, WebFetch failed DNS for cresta.com, fireworks.ai and prachub.com, curl through the egress proxy got 403 for cresta.com, prachub.com and job-boards.greenhouse.io, and the shared web-search budget was used up. So no URL could be confirmed live or dead here, and none was removed for that reason. References 1-22 are reused from chapters 01, 04 and 07, where their excerpts were reviewed. References 23-25 are reused from chapter 10 (entries 3, 15 and 16), with the same attributions and strength notes. Reference 24 is weak: only the thread title was seen. Glassdoor mirrors [1] and [23] show one review pool, so they are not independent reports. The product facts in the format notes (partial and final transcripts about once per second, redaction in the voice stack, inference graphs, fine-tuned models with multi-LoRA) come from those reviewed excerpts. All numeric assumptions inside the questions (hardware throughput, prices, volumes) are illustrative inputs, not Cresta figures.

10. Tagging System and Source Index: What Each Tag Means, Every Reported ML System Design Prompt by Portal, and Where Each Likely Prompt Comes From

> **Why this chapter exists.** You asked for details on the tagging system. Chapters 01-09 (there is no chapter 06) each repeat the [Reported]/[Likely] definitions in their header, keep their own reference lists, and scatter research caveats. This chapter puts all of it in one place.
>
> - Part 1 defines each tag and the evidence needed to earn it.
> - Part 2 lists every [Reported] system design prompt with its portal, date, role, round and evidence strength.
> - Part 3 maps each [Likely] design prompt to the job posting, blog post or product it is inferred from.
> - Part 4 lists questions to confirm the loop with your recruiter.
>
> Back-pointers such as "ch04 Q5" mean question 5 in chapter 04. Reference numbers in this chapter use the single consolidated list at the end, not the per-chapter lists.

Research limits (stated once, applies to the whole bank)

- **No source page was ever opened.** During research and every review pass, the egress proxy refused, or DNS failed for, every cited domain: Glassdoor, Blind (teamblind.com), 1point3acres, PracHub, Greenhouse, Lever, Built In, CoderPad, cresta.com, fireworks.ai and CX Today. While this chapter was being written, a fetch of the PracHub ML system design page failed again (DNS lookup `ENOTFOUND prachub.com`). In the adversarial review of this chapter, 12 more cited URLs were tried, and every one failed DNS lookup (`ENOTFOUND`): PracHub [10], 1point3acres [14], Glassdoor [1], Blind [6], Greenhouse [19], Lever [24], Built In [23], CoderPad [17], a16z Jobs [18], cresta.com [27], fireworks.ai [35] and CX Today [39]. A failed lookup means the page could not be checked. It does not show that the URL is dead or invented, so no reference was removed for that reason.
- **Live web search was exhausted.** The shared web-search budget for this run was already used up. A test search while writing this chapter was refused, and so was a search for Cresta MLE system design reports during the review. No new candidate reports could be found, and no new [Reported] prompts were added. Everything here comes from re-reading chapters 01-09 and the search-result excerpts they recorded, not from new searches.

- **What a [Reported] tag therefore rests on.** It rests on a search-result excerpt (title plus snippet) gathered while the bank was first researched, never on a full read of the page. Dates, roles and wording come from those excerpts and may be incomplete.
- **What you should do.** Treat the loop description as provisional. Before the onsite, ask your recruiter to confirm the round list, which round is ML system design, how long it lasts, who conducts it, and whether the AI-assisted coding round is in your loop (see Part 4).

Format notes for this round

- **ML system design slot.** About 60 minutes. One 2025-26 Senior MLE report lists it after a recruiter call, a LeetCode-medium round and a "vibe coding" round [1]. Senior/Staff SWE reports describe one hour of system design next to one hour of coding and a hiring-manager chat [6].
- **Interviewer (inferred).** Most likely a senior or staff MLE, or an engineering manager, from the track you are matched to: Agentic Assist, Agent and System Quality, Voice/ASR, Chat Agent or Insights [19][20][21][22][23][24].
- **Senior bar (from postings, not candidate reports).**
  - End-to-end ownership from research to production [23].
  - Evaluation of non-deterministic systems with offline benchmarks, online experiments and LLM-as-a-judge [19][20].
  - Diagnosis of hallucination, retrieval errors, tool misuse, context drift and prompt brittleness [19][20].
  - Design for latency, cost, scale and security, including real-time streaming for voice [21].
- **Process length.** About 25 days on average across Cresta engineering roles [2].

Part 1. The tags and their evidence thresholds

1.1 Tag definitions

Tag	Meaning	Minimum evidence to earn it	What it does not mean
[Reported] [n]	A candidate report or a question bank attributes this question, or a close variant, to Cresta.	A search-result excerpt from an interview portal (Glassdoor, Blind, 1point3acres, PracHub) that names Cresta and states the question or its title. The citation [n] must point to that excerpt's page.	It does not mean the question was asked in the Senior MLE system design round. Where the report is for another role (SWE, FDE) or round (phone screen, coding), the tag says so.
[Reported format]	A source reports that a round of this type exists at Cresta, not the exact wording.	An excerpt that describes the round (for example "project presentation"), with no question text.	It does not mean the wording in this bank was reported.
[Likely]	A well-grounded inference from Cresta's products, engineering blog, partner write-ups and ML job postings. No candidate has reported it.	A specific Cresta posting, blog post or product that makes the topic central to the role. A citation on a [Likely] item points to that source, not to a candidate report.	It never means someone was asked it.

Tag	Meaning	Minimum evidence to earn it	What it does not mean
[Likely scale-up]	A design version of a question reported for a Cresta coding round (rate limiter, worker pool, job dispatcher, decoder, API call). The citation points to the coding report.	The coding question must itself be [Reported] (table 2.2 or R4).	The design version was not reported.
[Self-check]	A study prompt about the evidence behind a reported item, used only in section 3.9.	None. The citation names the reported item it examines.	It is not an interview question and was not reported.

1.2 Evidence-strength scale used in Part 2

- **Strong:** the role (MLE) and the round (system design) both match, and the excerpt contains the question text.
- **Moderate:** the excerpt contains the question text, but the role or round differs, or the source is an aggregator listing with no role or date detail.
- **Weak:** only a listing title, a topic list (for example a recruiter's prep email), or a description of the round format.
- **Verified:** the full page was read and the question confirmed. **No item in this bank reaches this level**, because no page could be opened.

1.3 Source-handling rules applied across chapters

- **Glassdoor regional mirrors are one source.** The .co.in, .ca, .com and .sg pages (employer ID E3180909) show the same review pool. Two mirrors citing one review do not count as two reports.
- **PracHub is an aggregator.** Its pages are user-submitted question banks. A PracHub title counts as [Reported], but it is weaker than a first-person account.
- **Removed sources.** The Scoutify guide (a generic template, not candidate reports), the Blind "Cresta AI SWE AMA" thread (no excerpt supported any question) and the second 1point3acres thread 1173094 (no excerpt supported it) were removed.
- **Process claims from non-portal sources are [Likely].** A vendor case study (CoderPad [17]) or a job posting that describes the hiring process (a16z Jobs [18]) is not a candidate report, so a claim resting only on them is [Likely], not [Reported].
- **Restricted source.** The Blind "Cresta senior backend SWE interview" thread [9] supports only an online assessment, so it must not be cited for the system design round.
- **Open clean-up items.** Chapter 02's reference 12 still lists Scoutify. Chapter 01 has two questions numbered 34 and none numbered 39. Chapters 01, 02 and 05 still cite [9] for loop shape. Chapters 01 and 05 cite a JobsByCulture culture summary, and chapter 02 cites a ZeroG Talent repost of the Senior MLE posting [19]. Neither is an interview portal or supports a design prompt, so neither is indexed here.

Part 2. Every [Reported] prompt, by portal

2.1 Index table: reported system design and design-adjacent prompts

#	Prompt	Portal	Date	Role	Round	Strength
R 1	How would you deploy a model in production?	Glassdoor [1]	2025-26	Senior MLE	Systems design	Strong (one review, excerpt only)
R 2	Design an evaluation platform for AI agents across prompt and model changes	PracHub [10][11]	2026 (listing); experience undated	SWE (experience post); listed under ML system design	System design	Moderate (aggregator; role is SWE)
R 3	What is RAG, and how do you prevent hallucination?	Glassdoor [3]	2026	Forward Deployed Engineer	Technical screen (conceptual)	Moderate for content, weak as a design prompt
R 4	Design sequence decoding with greedy and beam search	PracHub [12][13]; 1point3acres [14]	2025 (Aug 2025 thread)	MLE, AI Agent direction	Phone screen (coding)	Moderate (role matches, round differs)
R 5	Walk through how an AI agent generates a sequence step by step	1point3acres [14]	2025	MLE, AI Agent direction	Phone screen follow-up	Moderate
R 6	Key concepts of LLMs, GenAI, RAG and AI agents	Blind [7]	2025	Not stated (engineering)	Recruiter prep email (topic list)	Weak
R 7	One hour of system design in the loop	Blind [6]	2024-25	Senior/Staff SWE	Format only	Weak (format)

2.2 Index table: reported coding prompts that design rounds may scale up

#	Prompt	Portal	Date	Role	Round	Strength
C 1	Implement a rate limiter	Glassdoor [1]	2024-26	Not stated	Coding	Moderate
C 2	Worker pool plus orchestrator with round-robin assignment	Glassdoor [1]	2024-26	FDE, AI Agent	Coding (OOP)	Moderate
C 3	Job dispatcher/executor system	1point3acres [15]	2026	Full-time SWE	Phone screen	Weak (title seen in a listing; text not read)
C 4	Call a public API (Wikipedia) and process results	1point3acres [16]	2024-26	SWE	Phone screen	Moderate
C 5	Project presentation	Glassdoor [2]	2025-26	SWE	Project presentation	Weak (format only)
C 6	1.5-hour practical with AI tools allowed; "vibe coding" with an LLM tool	Blind [8]; Glassdoor [1]	2025; 2025-26	FDE; Senior MLE	AI-assisted practical	Moderate

2.3 Reported prompts as practice questions

1. How would you deploy a model in production? [Reported] [1] (Senior MLE systems design round, Glassdoor 2025-26; ch04 Q2, ch08 Diagram 4)
    - What path does the model take from registry through shadow, canary and full rollout, and where is the rollback path?

    - What would you change for a real-time assist model compared with a batch post-call QA model?
    - Which signals in the first 24 hours trigger an automatic rollback, and which only page a human?
    - How do you pin one tenant to an old version while the others upgrade?
2. Design an evaluation platform for AI agents that catches regressions when prompts or models change. [Reported] [10][11] (PracHub; experience post labeled SWE system design round; ch04 Q1, ch08 Diagram 3)
    - What must a run pin (prompt, model ID, tools, knowledge snapshot, test set) so it can be reproduced a month later?
    - Which checks are deterministic, and which need an LLM judge?
    - How many reruns per scenario, and which statistical test declares a regression?
    - What does a full run cost across hundreds of customer-specific agents?
3. What is RAG, and how do you prevent hallucination? [Reported] [3] (FDE technical screen, Glassdoor 2026; conceptual, not a design prompt; ch04 Q3, ch08 Q67)
    - If the interviewer asks you to draw it, where in the pipeline does each type of hallucination start, and which check catches it?
    - What do you do when the groundedness check fails in the middle of a live voice turn?
4. Design sequence decoding with greedy and beam search as a reusable component. [Reported] [12][14] (PracHub title; the matching 1point3acres thread is a 2025 MLE AI Agent phone screen; ch04 Q4, ch02 Q3)
    - What interface lets greedy, beam and sampling share code?
    - How does beam width trade quality against latency and memory inside a real-time service?
5. Walk through how an AI agent generates a sequence step by step. Where can the decoding strategy change the agent's behavior? [Reported] [14] (MLE AI Agent phone-screen follow-up; ch03 Q2, ch07 Q24)
    - How would constrained decoding change what the agent can emit as a tool call?
6. Explain the key concepts and fundamentals of LLMs, GenAI, RAG and AI agents. [Reported] [7] (recruiter prep email; a topic list, not a question; ch01 Q57, ch03 Q3)
    - Can you draw how the four fit together inside one Cresta product?

Part 3. [Likely] design prompts mapped to their sources

Each prompt below is [Likely]: no candidate has reported it. Where a prompt has an "Inferred from" line, that line names the posting, blog post or product behind it. Every other prompt is mapped in table 3.0. The citation is context only.

3.0 Source map: which source each [Likely] prompt is inferred from

- **Direct:** the topic appears in the source's title or in the excerpt recorded in this bank.
- **Indirect:** the source states only a general requirement (latency, cost, security, "tool misuse", "end-to-end ownership"), and the prompt is one reasonable reading of it. Rank indirect prompts lower when you prepare.
- No source was re-opened to check this mapping (see Research limits).

Ref	Source	Type	Direct prompts	Indirect prompts
[19]	Senior MLE posting (Greenhouse)	Job posting	21, 22, 28, 35	26, 29, 51, 54, 55, 69, 71
[20]	Staff MLE posting (Greenhouse)	Job posting	21, 22, 35	69, 71
[21]	Senior MLE, Voice Experience (Greenhouse)	Job posting	9	36, 37, 51, 53, 70, 74
[22]	Senior MLE (ASR) (Greenhouse)	Job posting	10, 11, 12, 13	none
[23]	Senior MLE, Agentic Assist (Built In)	Job posting	75	34
[24]	Senior MLE (Chat Agent) (Lever)	Job posting	38	74
[25]	"How Cresta Grades Its Agents"	Blog	21, 60, 61	62
[26]	"Swiss-Cheese Model" evaluations post	Blog	20, 24, 27	none
[27]	"Real-Time Voice Agent Latency"	Blog	9, 16, 72	none
[28]	Voice platform: inference graphs	Blog	7, 8, 15, 17	none
[29]	Voice platform: voice stack	Blog	7, 10, 14	none
[30]	Ocean-1	Product (foundation model)	48, 49, 52	none
[31]	"AI-Ready Knowledge"	Blog / Knowledge Assist	39, 41, 42	none
[32]	Next-generation Insights (2026)	Product launch	23, 56, 57, 58, 59	63
[33]	Fine-tuned LLMs for Knowledge Assist, summarization, chat suggestions	Blog	39, 40, 44, 45	none
[34]	Fireworks "Story: Cresta Knowledge Assist"	Partner write-up	39, 40, 46	43
[35]	Fireworks "Multi-LoRA"	Partner write-up	46, 47	none
[36]	"Grounding Reality" hallucination post	Blog	32, 33	73
[37]	Automated AI Agent Testing launch	Product launch	18, 19, 25	none
[38]	Cresta Conductor	Product	28, 30, 31	none
[39]	CX Today interview with the CEO	Press interview	49, 50	none

Prompts 64-68 are [Likely scale-up] items built on reported coding questions (section 3.7). Prompts 76-78 are [Self-check] study prompts (section 3.9).

3.1 Inferred from the voice platform posts and the Voice Experience and ASR postings

7. Design real-time Agent Assist that listens to a live call and shows the human agent hints and suggested responses. [Likely] [29][28] (ch04 Q5, ch08 Diagram 1)
    - Inferred from: the voice-stack post (partial and final transcripts about once per second) and the inference-graph post.
    - What is your end-to-end latency budget from spoken words to on-screen hint?
    - Do you trigger on partial transcripts or wait for finals?
8. Design an inference graph that runs many models per conversation (ASR, redaction, intent, sentiment, summarization) with per-customer configuration. [Likely] [28] (ch04 Q9)
    - Inferred from: "Understanding Cresta's Voice Platform: ML Services, Inference Graphs and Real-Time Intelligence".
    - How do you stop one slow node from blocking real-time outputs?
9. Design a voice AI agent that answers inbound calls with responses in under a second. [Likely] [27][21] (ch04 Q19, ch08 Diagram 2)
    - Inferred from: "Engineering for Real-Time Voice Agent Latency" and the Voice Experience posting.
    - Which per-hop metrics do you track (ASR final latency, LLM time to first token, TTS time to first byte)?
    - How do you detect end of turn and handle barge-in?
10. Design a streaming ASR service for thousands of concurrent contact-center calls. [Likely] [22][29] (ch04 Q78)
    - Inferred from: the Senior MLE (ASR) posting and the voice-stack post.
    - How do you balance chunk size and lookahead against accuracy?
11. Design customer-specific vocabulary adaptation for ASR. [Likely] [22] (ch04 Q79, ch01 Q15)
    - Inferred from: the ASR posting.
    - Contextual biasing, fine-tuning or LLM post-correction: which, and why?
12. Design an ASR quality program across accents, noise and domains. [Likely] [22] (ch04 Q80)
    - How does WER relate to downstream task quality?
13. Design a test harness that measures how robust the LLM layer is to ASR errors. [Likely] [22] (ch04 Q41)
    - How do you inject realistic ASR noise on entities and spelled data?
14. Design real-time translation for a live call. [Likely] [29] (ch04 Q18)
    - Where does translation sit relative to ASR, hints and the agent's reply?
15. Design a real-time sentiment and escalation-risk detector. [Likely] [28] (ch04 Q10)
    - How do you combine acoustic and lexical signals?
16. Keep a voice agent's latency stable during peak traffic. [Likely] [27] (ch04 Q25)
    - What degrades first, and what is your graceful-degradation plan?
17. Design PII redaction for audio and transcripts before storage and training. [Likely] [28] (ch04 Q81)
    - What recall target does the cost of a missed card number set?

3.2 Inferred from the agent-evaluation posts and the Agent and System Quality track

18. Design a customer simulator that generates synthetic callers to test an AI agent before launch. [Likely] [37] (ch04 Q33)
    - Inferred from: "Cresta Launches Automated AI Agent Testing".
    - How do you stop the simulator and the agent from sharing blind spots?
19. Extend the simulator to voice: accents, background noise, interruptions and silence. [Likely] [37] (ch04 Q40)
    - Which failures appear only in audio?
20. Design a layered ("Swiss-cheese") reliability program for a production AI agent. [Likely] [26] (ch04 Q34)
    - Inferred from: "Why AI Agent Evaluations Fail, and How the Swiss-Cheese Model Prevails".
    - What does each layer catch, and what does it miss?
21. Design an LLM-as-a-judge evaluator service used for both testing and production scoring. [Likely] [25][19][20] (ch04 Q35)
    - Inferred from: "How Cresta Grades Its Agents: Evaluators in an LLM World" and the LLM-as-a-judge requirement in the Senior and Staff postings.
    - How do you calibrate judges against expert labels and detect judge drift?
22. Design production monitoring for AI agents across hundreds of enterprise deployments. [Likely] [19][20] (ch04 Q36)
    - Which label-free signals indicate a quality problem for one customer or one intent?
23. Measure whether an AI agent resolved a call when there is no survey or disposition code. [Likely] [32] (ch04 Q37)
    - How do you validate an inferred-outcome model without ground truth?
24. Design a shadow-mode launch in which a new AI agent runs silently beside human agents. [Likely] [26] (ch04 Q39)
    - Which disagreements are agent errors, and which are policy differences?
25. Design the curation pipeline for golden evaluation sets drawn from production conversations. [Likely] [37] (ch04 Q42)
    - How do you keep the set from going stale or leaking into training data?
26. Design an audit and replay system that reproduces why an agent did something three months ago. [Likely] [19] (ch04 Q43)
    - What must you log, and at what storage cost?
27. Design the release process for a prompt change that ships to 50 customer agents at once. [Likely] [26] (ch04 Q38)
    - How do you stage the rollout and roll back when a customer has customized the prompt?

3.3 Inferred from the AI Agent product, Conductor and the grounding post

28. Design the tool-use layer for an AI agent that looks up orders and issues refunds. [Likely] [38][19] (ch04 Q20)
    - Inferred from: Cresta Conductor (flight-rebooking example) and "tool misuse" in the Senior MLE posting.
    - How do you stop the agent from making unauthorized commitments?
29. Design the handoff from an AI agent to a human agent. [Likely] [19] (ch04 Q21)
    - What context do you pass to the human, and in what format?

30. Design an agent-building system that turns a customer's SOPs into a working AI agent. [Likely] [38] (ch04 Q22)
    - Inferred from: "Cresta Conductor: The Agent for AI Agent Development".
    - Structured flow, free-form LLM with tools, or a hybrid?
31. Design tool onboarding from a customer's API spec, with a sandbox and tests. [Likely] [38] (ch04 Q89)
    - How do you detect tool misuse after launch?
32. Design guardrails against prompt injection, PII leakage and policy violations. [Likely] [36] (ch04 Q23)
    - Inferred from: "Grounding Reality: How Cresta Tackles LLM Hallucinations in Enterprise AI".
    - Where do guardrails run, and what latency does each add?
33. Design a grounding checker that flags responses not supported by the knowledge base or tool outputs. [Likely] [36] (ch01 Q10)
    - Claim extraction plus entailment, or a single judge call?
34. Design a multi-agent architecture with a router and specialized sub-agents. [Likely] [23] (ch04 Q24)
    - When does it beat one agent with many tools?
35. Design context management for a 30-60 minute call with many tool calls. [Likely] [19] [20] (ch04 Q27)
    - Inferred from: "context drift" in the Senior and Staff postings.
    - What do you keep verbatim, summarize, or store as structured state?
36. Design identity verification inside a voice agent before sensitive actions. [Likely] [21] (ch04 Q28)
    - How do you enforce "verify first" in code rather than in the prompt?
37. Design an outbound voice agent with answering-machine and IVR detection. [Likely] [21] (ch04 Q30)
    - How do consent records and calling-window rules enter the design?
38. Design an AI agent for asynchronous messaging (SMS, WhatsApp) where one conversation spans days. [Likely] [24] (ch04 Q32)
    - Inferred from: the Chat Agent posting on Lever.
    - What changes in latency targets and evaluation compared with voice?

3.4 Inferred from Knowledge Assist, the fine-tuning post and the Fireworks write-ups

39. Design Knowledge Assist: surface answers from a customer's knowledge base during a live conversation. [Likely] [33][34][31] (ch04 Q58, ch08 Diagram 5; extends reported R3)
    - Inferred from: "How Fine-Tuned LLMs Power Knowledge Assist, Summarization and Chat Suggestions", Fireworks "Story: Cresta Knowledge Assist" and "AI-Ready Knowledge for Contact Centers".
    - How do you turn implicit conversation context into a retrieval query?
40. Serve Knowledge Assist with a fine-tuned model at low latency and cost. [Likely] [33] [34] (ch04 Q59)
    - When does fine-tuning beat prompting a frontier model?
41. Design the ingestion pipeline that makes enterprise knowledge "AI-ready" (duplicates, stale and contradictory articles). [Likely] [31] (ch04 Q60)

    - How do you surface conflicting articles to the content owner?
42. Design retrieval evaluation with no labeled query-document pairs. [Likely] [31] (ch04 Q62)
    - How far can you trust LLM-bootstrapped labels?
43. Plan an embedding-model upgrade that re-embeds billions of vectors without downtime. [Likely] [34] (ch04 Q64)
    - How do you cut over per tenant?
44. Design suggested-response generation for chat agents. [Likely] [33] (ch04 Q7)
    - Inferred from: the fine-tuning post (chat suggestions).
    - Which offline metric best predicts acceptance rate?
45. Design live summarization that is ready when the call ends and writes to the customer's CRM fields. [Likely] [33] (ch04 Q8, ch01 Q23)
    - How do you measure factual consistency at scale?
46. Design multi-tenant LLM serving: one base model with per-customer LoRA adapters. [Likely] [35][34] (ch04 Q65, ch08 Diagram 7)
    - Inferred from: Fireworks "Multi-LoRA" and the Knowledge Assist story.
    - How do you prevent cross-tenant leakage in caches, logs and training data?
47. Design per-customer fine-tuning as a product feature, with evaluation and rollback per tenant. [Likely] [35] (ch04 Q70)
    - How do you stop a tenant's adapter from regressing silently after a base-model upgrade?

3.5 Inferred from Ocean-1, the CEO interview and the serving requirements in postings

48. Design the domain adaptation of a base LLM for contact centers, in the spirit of Ocean-1. [Likely] [30] (ch04 Q68, ch01 Q24)
    - Inferred from: "Introducing Ocean-1: World's First Contact Center Foundation Model".
    - How do you show it beats a frontier API on quality, latency and cost?
49. Plan the migration of one high-volume feature from a frontier API to a distilled, self-hosted model. [Likely] [30][39] (ch04 Q69)
    - Inferred from: Ocean-1 and the CX Today interview with Cresta's CEO.
    - What is the break-even volume?
50. Design a model router between a self-hosted small model and a frontier API. [Likely] [39] (ch04 Q67)
    - How do you prove the router does not hurt quality?
51. Design an LLM serving stack that meets p99 latency targets for real-time features. [Likely] [19][21] (ch04 Q66)
    - Inferred from: latency, cost and model-optimization requirements in the postings.
    - What signal would you autoscale GPUs on?
52. Design a distillation pipeline in which a frontier LLM labels data for small real-time classifiers. [Likely] [30] (ch04 Q76)
    - How do you refresh the student when the customer's taxonomy changes?
53. Design scheduling that shares a GPU fleet between real-time inference and batch work. [Likely] [21] (ch04 Q75)
    - How do you pre-empt batch work without losing progress?

54. Design cost accounting for LLM usage per customer and per feature. [Likely] [19] (ch04 Q71)
    - How do you attribute GPU cost on shared, batched infrastructure?
55. Design for data residency and isolation on a shared LLM platform. [Likely] [19] (ch04 Q72)
    - What changes in serving, storage and training?

3.6 Inferred from the Insights launch and QA and coaching products

56. Design topic discovery over millions of unlabeled conversations, with an editable taxonomy. [Likely] [32] (ch04 Q44, ch08 Diagram 6)
    - Inferred from: "Introducing the Next Generation of Cresta Insights" (2026).
    - How do you keep assignments stable when admins merge or rename topics?
57. Design an "AI analyst" that answers open-ended business questions over a conversation corpus. [Likely] [32] (ch04 Q45)
    - How do you stop it from presenting a weak correlation as a finding?
58. Design a driver-analysis system that finds which agent behaviors correlate with good outcomes. [Likely] [32] (ch04 Q46)
    - How do you control for call type and customer mood?
59. Design an emerging-issue detector that flags spikes within minutes. [Likely] [32] (ch04 Q47)
    - How do you separate a true spike from seasonality?
60. Design automated QA that scores 100% of calls against each customer's rubric. [Likely] [25] (ch04 Q51)
    - Inferred from: Cresta's QA and evaluator products (the evaluators post).
    - How do you turn a free-text rubric item into an auditable judgment?
61. Design a system that lets a customer add a QA criterion and have it live within an hour. [Likely] [25] (ch04 Q53)
    - What does a backfill over a year of calls cost?
62. Design compliance monitoring with near-zero miss tolerance. [Likely] [25] (ch04 Q56)
    - Which checks run in real time, and which post-call?
63. Design storage and indexing for searching billions of utterances by keyword, meaning and metadata. [Likely] [32] (ch04 Q49)
    - How do you enforce per-tenant isolation in the index?

3.7 Scale-ups of reported coding questions

These are design prompts tagged [Likely scale-up]. The coding question underneath each one is [Reported] (see table 2.2), and the citation points to that coding report. The design version itself was never reported.

64. Scale the reported rate limiter into per-tenant quota enforcement in front of shared LLM serving. [Likely scale-up] [1] (ch04 Q74; built on C1)
    - Token-based or request-based limits?
    - How do you protect real-time traffic from a tenant's batch backfill?
65. Scale the reported worker pool and job dispatcher into an orchestrator for agent and evaluation jobs. [Likely scale-up] [1][15] (ch04 Q88; built on C2 and C3)
    - How do you handle stuck jobs, retries and idempotency?
    - How do you give each tenant a fair share?

66. Scale the reported decoder into a structured-output layer that guarantees valid tool-call JSON. [Likely scale-up] [12][14] (ch04 Q77, ch07 Q25; built on R4)
    - Constrained decoding, or validate-and-retry? What does each cost in latency?
67. Scale the reported "call a public API" exercise into a resilient connector layer for customer CRMs and knowledge bases. [Likely scale-up] [16] (built on C4)
    - How do you handle pagination, rate limits, schema drift and outages?
68. Redraw a project you shipped as an evaluation platform for weekly prompt and model changes. [Likely scale-up] [2][10] (ch05 Q35; combines C5 with R2)
    - Which parts of your old evaluation stack would survive, and which would you rebuild?

3.8 Cross-cutting probes inferred from the senior bar in postings

69. Which non-functional requirement dominates your design: latency, cost, quality or isolation? [Likely] [19][20] (ch04 Q90)
70. Give a back-of-envelope estimate: concurrent calls, tokens per day, GPUs needed. [Likely] [21] (ch04 Q91)
71. What is your offline evaluation, and how do you know it predicts online results? [Likely] [19][20] (ch04 Q93)
72. What happens when the LLM provider has an outage or rate-limits you mid-call? [Likely] [27] (ch04 Q95)
73. Where could this system hallucinate or mislead a human agent, and how do you contain the blast radius? [Likely] [36] (ch04 Q97)
74. How does the design change for chat versus voice, and for 10 customers versus 1,000? [Likely] [24][21] (ch04 Q96)
75. What would make you choose a classic ML model over an LLM for one component? [Likely] [23] (ch04 Q99)
    - Inferred from: the Agentic Assist track's call for strong ML foundations from before LLMs.

3.9 Testing yourself on the tags

These three are study prompts about the evidence, not interview questions, so they carry the [Self-check] tag (see table 1.1). The citation names the reported item each one examines.

76. For the prompt "How would you deploy a model in production?", what exactly did the source report, and which follow-ups in this bank are your own extrapolation? [Self-check] [1]
    - Can you prepare an answer that works whether the interviewer means an LLM feature or a classic model?
77. The agent-evaluation prompt is reported for an SWE system design round. How would an MLE interviewer likely push deeper than an SWE interviewer on the same prompt? [Self-check] [10][11]
    - Would they spend more time on judge calibration and statistical gates than on storage and APIs?
78. The RAG question was a conceptual FDE screen question. If it becomes a design prompt in your loop, which diagram would you draw first? [Self-check] [3]

Part 4. Questions to confirm the loop with your recruiter

The tag on each question is the tag of the claim it checks.

79. Does my onsite include a dedicated ML system design round, and is it about 60 minutes? [Reported] [1][6]
    - The round's existence is reported for a Senior MLE [1]. The one-hour length is reported only for Senior/Staff SWE loops [6].
    - Is it ML system design specifically, or general backend system design?
80. Is the round more likely to centre on deploying and operating a model, or on LLM agents and evaluation? [Reported] [1][10]
81. Will there be an AI-assisted ("vibe coding") round, and which tools may I use? [Reported] [1][8]
    - Is it scored on the final product, or on how I direct and verify the tool?
82. Is the technical screen a LeetCode-style round, an ML coding problem such as decoding, or both? [Reported] [1][14]
83. Will an asynchronous CoderPad assessment come before the live rounds? [Likely] [17] [18]
    - Retagged from [Reported]: [17] is a vendor case study and [18] is a job posting. Neither is a candidate report on an interview portal, and neither is for an MLE role.
84. Should I expect questions on LLM, GenAI, RAG and AI-agent fundamentals, as other candidates were told? [Reported] [7]
85. Is there a project presentation or deep-dive round for the MLE loop, as there is for some SWE loops? [Reported format] [2]
86. Which team or track is the system design interviewer from: Agentic Assist, Agent and System Quality, Voice/ASR, Chat Agent or Insights? [Likely] [19][21][22][23][24]
87. Will the design round focus on real-time voice, chat or batch analytics? [Likely] [21] [28]
88. Is there a separate hiring-manager or behavioral round, and how long is it? [Reported] [6]
89. What is the expected timeline from onsite to decision? [Reported] [2]

References

1. Glassdoor, "Cresta (CA) Interview Questions" (Senior MLE loop: recruiter call, LeetCode-medium, vibe-coding and systems design rounds; "How would you deploy a model in production?"; CoderPad and Zoom Python round; rate limiter; FDE AI Agent worker-pool question), 2024-2026. https://www.glassdoor.co.in/Interview/Cresta-CA-Interview-Questions-E3180909.htm
2. Glassdoor, "Cresta (CA) Interview Questions" (about 25-day average; SWE project presentation; Senior FDE loop), 2025-2026. https://www.glassdoor.ca/Interview/Cresta-CA-Interview-Questions-E3180909.htm
3. Glassdoor, "Cresta (CA) Interview Experience and Questions" (FDE technical screen: "What is RAG, and how do you prevent hallucination?"), 2026. https://www.glassdoor.com/Interview/Cresta-CA-Interview-Questions-E3180909.htm

4. Glassdoor, "Cresta (CA) Senior Software Engineer Interview Questions" (three LeetCode-medium questions in 70 minutes), 2026. https://www.glassdoor.ca/Interview/Cresta-CA-Senior-Software-Engineer-Interview-Questions-EI_IE3180909.0,9_KO10,34.htm
5. Glassdoor, "Cresta (CA) Software Engineer Interview Questions" (autocomplete, list-to-tree, grid puzzle, SQL joins), 2024-2026. https://www.glassdoor.sg/Interview/Cresta-CA-Software-Engineer-Interview-Questions-EI_IE3180909.0,9_KO10,27.htm
6. Blind, "Cresta interview process", 2024-2025. https://www.teamblind.com/post/cresta-interview-process-jmfnzjoa
7. Blind, "Anyone interview with Cresta lately?" (recruiter email: "key concepts and fundamentals around LLMs, GenAI, RAG, and AI Agents"), 2025. https://www.teamblind.com/post/anyone-interview-with-cresta-lately-zvj0uhq2
8. Blind, "Interview for forward deployment engineer at Cresta" (1.5-hour practical with AI allowed), 2025. https://www.teamblind.com/post/interview-for-forward-deployment-engineer-at-cresta-ngswbj44
9. Blind, "Cresta senior backend SWE interview" (supports an online assessment only; not to be cited for the design round), 2025. https://www.teamblind.com/post/cresta-senior-backend-swe-interview-6v1tmjld
10. PracHub, "Cresta ML System Design Interview Questions" ("Design an Evaluation Platform for AI Agents Across Prompt and Model Changes"), 2026. https://prachub.com/companies/cresta/categories/ml-system-design
11. PracHub, "Cresta Software Engineer Interview Experience: System Design Round: Build an Eval Platform for AI Agents". https://prachub.com/interview-experiences/cresta-software-engineer-interview-experience-system-design-round-build-an-eval-platform-for-ai-agents
12. PracHub, "Cresta Interview Questions" (company page; "Design sequence decoding with greedy and beam search"), 2025. https://prachub.com/companies/cresta
13. PracHub, "Cresta Machine Learning Interview Questions" ("Implement greedy and beam decoding"). https://prachub.com/companies/cresta/categories/machine-learning
14. 1point3acres, "Cresta Machine Learning Engineer Tech Phone Interview Experience" (MLE, AI Agent direction; greedy and beam-search decoding; how an agent generates a sequence), 2025. https://www.1point3acres.com/interview/thread/1140481
15. 1point3acres, "Cresta Fulltime Software Engineer Tech Phone Screen: Job Dispatcher-Executor System Implementation" (title seen in a listing; full text not read), 2026. https://www.1point3acres.com/interview/thread/1172433
16. 1point3acres, "cresta interview questions" (company page; SWE/MLE phone screens and online tests, including the API phone screen), 2024-2026. https://www.1point3acres.com/interview/company/cresta
17. CoderPad, "How Cresta consolidated all technical hiring on CoderPad to scale eng hiring" (case study). https://coderpad.io/case-study/how-cresta-consolidated-all-technical-hiring-on-coderpad-to-scale-eng-hiring/
18. a16z Jobs, "Cresta: Senior Product Engineer" (posting describing the self-paced CoderPad assessment). https://jobs.a16z.com/jobs/cresta/5288477008--senior-product-engineer
19. Cresta on Greenhouse, "Senior Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4249943008
20. Cresta on Greenhouse, "Staff Machine Learning Engineer". https://job-boards.greenhouse.io/cresta/jobs/4741283008

21. Cresta on Greenhouse, "Senior Machine Learning Engineer, Voice Experience". https://job-boards.greenhouse.io/cresta/jobs/5155675008
22. Cresta on Greenhouse, "Senior Machine Learning Engineer (ASR)". https://job-boards.greenhouse.io/cresta/jobs/5101011008
23. Built In Vancouver, "Senior Machine Learning Engineer, Cresta" (Agentic Assist track). https://builtinvancouver.org/job/senior-machine-learning-engineer/8619914
24. Cresta on Lever, "Senior Machine Learning Engineer (Chat Agent)". https://jobs.lever.co/cresta/172fe276-9415-4bae-9b0b-af72cf7cf014
25. Cresta Blog, "How Cresta Grades Its Agents: Evaluators in an LLM World". https://cresta.com/blog/how-cresta-grades-its-agents-evaluators-in-an-llm-world
26. Cresta Blog, "Why AI Agent Evaluations Fail, and How the Swiss-Cheese Model Prevails". https://cresta.com/blog/why-ai-agent-evaluations-fail----and-how-the-swiss-cheese-model-prevails
27. Cresta Blog, "Engineering for Real-Time Voice Agent Latency". https://cresta.com/blog/engineering-for-real-time-voice-agent-latency
28. Cresta Blog, "Understanding Cresta's Voice Platform: ML Services, Inference Graphs and Real-Time Intelligence", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-ml-services-inference-graphs-and-real-time-intelligence
29. Cresta Blog, "Understanding Cresta's Voice Platform: The Voice Stack", 2025. https://cresta.com/blog/understanding-crestas-voice-platform-the-voice-stack
30. Cresta Blog, "Introducing Ocean-1: World's First Contact Center Foundation Model", 2023. https://cresta.com/blog/introducing-ocean-1-worlds-first-contact-center-foundation-model
31. Cresta Blog, "AI-Ready Knowledge for Contact Centers: Closing the Gap Between KBs and AI", 2025. https://cresta.com/blog/ai-ready-knowledge-for-contact-centers-closing-the-gap-between-kbs-and-ai
32. Cresta Blog, "Introducing the Next Generation of Cresta Insights: A Complete System for Enterprises to Understand Their CX", 2026. https://cresta.com/blog/introducing-the-next-generation-of-cresta-insights-a-complete-system-for-enterprises-to-understand-their-cx
33. Cresta Blog, "How Fine-Tuned LLMs Power Knowledge Assist, Summarization and Chat Suggestions", 2024. https://cresta.com/blog/how-fine-tuned-llms-power-knowledge-assist-summarization-and-chat-suggestions
34. Fireworks AI Blog, "Story: Cresta Knowledge Assist", 2024. https://www.fireworks.ai/blog/story-cresta-knowledge-assist
35. Fireworks AI Blog, "Multi-LoRA", 2024. https://fireworks.ai/blog/multi-lora
36. Cresta Blog, "Grounding Reality: How Cresta Tackles LLM Hallucinations in Enterprise AI". https://cresta.com/blog/grounding-reality---how-cresta-tackles-llm-hallucinations-in-enterprise-ai
37. Cresta Blog, "Cresta Launches Automated AI Agent Testing: Confidence in Every Conversation". https://cresta.com/blog/cresta-launches-automated-ai-agent-testing-confidence-in-every-conversation
38. Cresta Blog, "Cresta Conductor: The Agent for AI Agent Development". https://cresta.com/blog/cresta-conductor-the-agent-for-ai-agent-development
39. CX Today, "Cresta CEO Ping Wu: Many Contact Centers Misunderstand the Value of LLMs". https://www.cxtoday.com/contact-center/cresta-ceo-ping-wu-many-contact-centers-misunderstand-the-value-of-llms/

Source note: All 39 references are carried over from the reviewed reference lists in chapters 01-09, with the same attributions. None was opened for this chapter. A fetch of prachub.com failed DNS lookup while it was written, 12 more fetches failed the same way in review (see Research limits), and the web-search budget was exhausted. References 4, 5 and 9 support coding or online-assessment claims in other chapters. They are listed here so this index covers every interview-portal source in the bank, but no system design prompt rests on them.

Changed in review: Every cited URL tried (12) failed DNS lookup, and web search was refused, so no reference could be verified, removed or added, and no new candidate-reported prompts were added. Changes come from reading this chapter against chapters 01-09. Recruiter question "Will an asynchronous CoderPad assessment come before the live rounds?" was retagged from [Reported] to [Likely], because its sources are a vendor case study and a job posting, not portal reports. Added a rule that process claims from non-portal sources are [Likely]. Section 3.7 items now use a named [Likely scale-up] tag. Section 3.9 items are now [Self-check], defined in table 1.1, because they are study prompts and not interview questions. The former Q79 ("which [Likely] areas map to your track") was removed as a duplicate of recruiter question 86, and Part 4 was renumbered 79-89. Added table 3.0, which maps every [Likely] prompt to its source and marks it direct or indirect; a script confirmed the table matches every inline citation. Recruiter question 79 now notes that the 60-minute length comes from SWE reports only. Chapter coverage was corrected from 01-08 to 01-09, and the JobsByCulture and ZeroG Talent citations in other chapters are recorded as not indexed.